

EXP NO: 1**CREATION OF LARGE DATASET****DATE:**

AIM: To create large datasets and store it in different file formats such as .csv, .json, .xml.

QUESTIONS:

1. Create a 100-record employee dataset of a hospital with the following attributes and save it as csv file (Emp_id, Name, Age, Salary, Designation, Gender, Experience, City and Year of Joining).

CODE:

```
import pandas as pd
import numpy as np
from faker import Faker
import random

fake = Faker()

num_rows = 100

titles = ["Dr.", "Mr.", "Ms."]
designations = [
    "Doctor",
    "Nurse",
    "Radiologist",
    "Pharmacist",
    "Manager",
    "Lab Technician",
    "Paramedic",
    "System Administrator"
]

cities = ["Chennai", "Bangalore", "Hyderabad", "Pune"]

genders = ["Male", "Female"]

data = {
    "Emp_ID": np.arange(1, num_rows + 1),

    "Name": [
        random.choice(titles) + " " + fake.first_name() + " " + fake.last_name()
        for _ in range(num_rows)
    ],

    "Age": np.random.randint(25, 66, num_rows),

    "Salary": np.random.randint(30000, 1000001, num_rows),
```

```

"Designation": np.random.choice(designations, num_rows),

"Gender": np.random.choice(genders, num_rows),

"Experience": np.random.randint(1, 26, num_rows),

"City": np.random.choice(cities, num_rows),

"Year_of_Joining": np.random.randint(2000, 2027, num_rows)
}

df = pd.DataFrame(data)

print(df.head(10))

df.to_csv("Hospital_Employee_Dataset.csv", index=False)

print("CSV File Saved Successfully!")

```

OUTPUT:

```

...   Emp_ID      Name  Age  Salary  Designation  Gender \
0      1  Ms. Matthew Gutierrez  50   42404      Doctor  Female
1      2    Dr. Mary Bennett  62  710563      Doctor  Female
2      3    Ms. Roy Odom  43  920218  Lab Technician  Female
3      4    Dr. James Bell  30  136747      Manager  Female
4      5    Ms. Thomas Hall  54  212726    Radiologist  Female
5      6    Ms. Jose Anderson  42  709740      Nurse    Male
6      7    Mr. John Pratt  25  234730    Radiologist  Female
7      8  Dr. Audrey Castillo  30  476610      Nurse    Male
8      9    Mr. Andrea Holt  52  962177      Doctor  Female
9     10  Ms. Kenneth Harrison  50  773144      Doctor  Female

   Experience      City  Year_of_Joining
0          11   Chennai          2017
1          12   Chennai          2011
2          25  Bangalore          2003
3          11     Pune          2000
4          13  Bangalore          2022
5          18   Chennai          2014
6           8  Bangalore          2008
7           9  Bangalore          2004
8          13  Hyderabad          2006
9           5   Chennai          2009
CSV File Saved Successfully!

```

Hospital_Employee_Dataset

Emp_ID	Name	Age	Salary	Designation	Gender	Experience	City	Year_of_Joining
1	Ms. Matthew Gutierrez	50	42404	Doctor	Female	11	Chennai	2017
2	Dr. Mary Bennett	62	710563	Doctor	Female	12	Chennai	2011
3	Ms. Roy Odom	43	920218	Lab Technician	Female	25	Bangalore	2003
4	Dr. James Bell	30	136747	Manager	Female	11	Pune	2000
5	Ms. Thomas Hall	54	212726	Radiologist	Female	13	Bangalore	2022
6	Ms. Jose Anderson	42	709740	Nurse	Male	18	Chennai	2014
7	Mr. John Pratt	25	234730	Radiologist	Female	8	Bangalore	2008
8	Dr. Audrey Castillo	30	476610	Nurse	Male	9	Bangalore	2004
9	Mr. Andrea Holt	52	962177	Doctor	Female	13	Hyderabad	2006
10	Ms. Kenneth Harrison	50	773144	Doctor	Female	5	Chennai	2009
11	Dr. Misty Pierce	29	481378	Doctor	Male	15	Bangalore	2019
12	Dr. Rachel Perry	59	231853	Manager	Female	4	Hyderabad	2011
13	Mr. Maria McBride	51	352674	System Administrator	Female	14	Chennai	2008
14	Ms. David Peterson	46	234515	Lab Technician	Female	12	Pune	2024
15	Dr. Lauren Morgan	56	181042	Pharmacist	Male	22	Chennai	2012

2. Create a 100-record news article dataset with the following attributes and save it as json file (article_id, title, author, publisher, category, published_date, content, views, likes, comments, shares, reading_time_minutes).

CODE:

```
import pandas as pd
import numpy as np
from faker import Faker
import random
```

```
fake = Faker()
```

```
num_rows = 100
```

```
publishers = [
    "Daily News",
    "Global Times",
    "Tech Today",
    "Sports Weekly",
    "Health Chronicle",
    "Business World"
]
```

```
categories = [
    "Politics",
    "Technology",
    "Sports",
    "Health",
    "Business",
]
```

```

"Entertainment",
"Education",
"Science"
]

data = {
    "article_id": [f"ART{str(i).zfill(3)}" for i in range(1, num_rows + 1)],
    "title": [fake.sentence(nb_words=6) for _ in range(num_rows)],
    "author": [fake.name() for _ in range(num_rows)],
    "publisher": np.random.choice(publishers, num_rows),
    "category": np.random.choice(categories, num_rows),
    "published_date": [
        fake.date_between(start_date='-2y', end_date='today').isoformat()
        for _ in range(num_rows)
    ],
    "content": [fake.text(max_nb_chars=300) for _ in range(num_rows)],
    "views": np.random.randint(100, 100001, num_rows),
    "likes": np.random.randint(10, 10001, num_rows),
    "comments": np.random.randint(0, 2001, num_rows),
    "shares": np.random.randint(0, 5001, num_rows),
    "reading_time_minutes": np.random.randint(2, 16, num_rows)
}

df = pd.DataFrame(data)

print(df.head())

df.to_json("News_Articles.json", orient="records", lines=True)

print("JSON File Saved Successfully!")

```

OUTPUT:

```

...  article_id                                     title \
0    ART001      Trade recognize beautiful how matter any.
1    ART002                South month him car follow.
2    ART003  Out own computer camera rise member there hist...
3    ART004                Firm product box specific east.
4    ART005      Toward blood PM ten born.

...  author      publisher      category  published_date \
0    Willie Smith    Daily News      Science    2025-10-03
1    Breanna Mcguire  Sports Weekly      Health    2026-03-14
2    Summer Williams  Business World    Technology    2025-03-04
3    Luis Day        Global Times      Education    2026-03-24
4    Kelly Lewis     Business World    Entertainment  2026-02-04

...  content      views  likes  comments \
0    Black important test. Page century situation w...  73442  5012  336
1    Open shake name detail agreement radio informa...  4986  6807  1188
2    Statement program data above woman somebody ne...  76975  6056  1224
3    Activity difficult young financial computer us...  72287  4625  1834
4    Identify camera work still then. Knowledge dea...  76990  865  1674

...  shares  reading_time_minutes
0    3040      10
1    4990      12
2     20       9
3    461       3
4   1415      13
JSON File Saved Successfully!

```

```

() News_Articles.json 1 X
Users > john > Desktop > () News_Articles.json > ...
1 {"article_id":"ART001","title":"Trade recognize beautiful how matter any.,"author":"Willie Smith"
2 {"article_id":"ART002","title":"South month him car follow.,"author":"Breanna Mcguire","publisher
3 {"article_id":"ART003","title":"Out own computer camera rise member there history.,"author":"Summ
4 {"article_id":"ART004","title":"Firm product box specific east.,"author":"Luis Day","publisher":"
5 {"article_id":"ART005","title":"Toward blood PM ten born.,"author":"Kelly Lewis","publisher":"Bus
6 {"article_id":"ART006","title":"Especially lay later case eight.,"author":"Karen Caldwell","publi
7 {"article_id":"ART007","title":"Off safe since meeting ability.,"author":"Samantha Espinoza","pub
8 {"article_id":"ART008","title":"Dog finally family second catch teach.,"author":"Stephanie Maynar
9 {"article_id":"ART009","title":"Animal performance social.,"author":"Jenny Cole","publisher":"Glo
10 {"article_id":"ART010","title":"Certain list down seat level situation.,"author":"Lisa Miller","p
11 {"article_id":"ART011","title":"Hold little east usually behavior piece process produce.,"author"
12 {"article_id":"ART012","title":"Night economy ok prepare true believe election.,"author":"Carolín
13 {"article_id":"ART013","title":"Attention least modern election total theory hard.,"author":"Mich
14 {"article_id":"ART014","title":"Evidence rest friend nice establish marriage rate street.,"author
15 {"article_id":"ART015","title":"Coach despite drop environmental check.,"author":"Sarah Franklin"

```

3. Create a 100-record Purchase dataset with the following attributes and save it as xml file (DeviceID, OwnerName, IPAddress, OS, OSVersion, PurchaseDate, Warranty Status, BatteryHealth, IsOnline).

CODE:

```

import pandas as pd
from faker import Faker
import random

fake = Faker()

num_rows = 100

os_list = ["Windows", "Linux", "Android", "iOS", "macOS"]
warranty = ["Valid", "Expired"]

data = {
    "DeviceID": [fake.uuid4() for _ in range(num_rows)],
    "OwnerName": [fake.name() for _ in range(num_rows)],
    "IPAddress": [fake.ipv4() for _ in range(num_rows)],
    "OS": [random.choice(os_list) for _ in range(num_rows)],
    "OSVersion": [fake.numerify(text="###") for _ in range(num_rows)],
    "PurchaseDate": [
        fake.date_between(start_date='-5y', end_date='today').isoformat()
        for _ in range(num_rows)
    ],
    "WarrantyStatus": [random.choice(warranty) for _ in range(num_rows)],
    "BatteryHealth": [random.randint(50, 100) for _ in range(num_rows)],
    "IsOnline": [random.choice([True, False]) for _ in range(num_rows)]
}

df = pd.DataFrame(data)

print(df.head())

```

```
df.to_xml(
    "Purchase_Dataset.xml",
    root_name="Devices",
    row_name="Device",
    index=False
)

print("XML File Saved Successfully!")
```

OUTPUT:

```
...
      DeviceID      OwnerName      IPAddress \
0  56609355-2510-4ac0-94ca-27fbc4079dd1      Tina West      221.212.19.52
1  ef1e7691-3425-484e-b713-7fe629437ad7      Nancy Fleming      28.125.221.95
2  69fc31b2-4ffa-4dc4-9740-b9b19115f8a6      Eric Spence      82.250.169.129
3  32217122-5308-4829-9e7b-8c44f3bb485b      Mr. Michael Leach      1.172.235.215
4  b4571dac-8678-4eae-ba7c-440f27d95a4d      Carolyn Hill      202.93.75.124

      OS OSVersion PurchaseDate WarrantyStatus BatteryHealth IsOnline
0   Linux    9.8.7    2024-06-10         Valid           83      False
1    iOS     0.0.4    2024-10-06        Expired           95      False
2  Windows    6.8.0    2026-04-19        Expired           66      False
3    iOS     6.0.0    2024-11-12         Valid           51      False
4    iOS     7.2.0    2025-01-02         Valid           72       True
XML File Saved Successfully!
```

```
Purchase_Dataset.xml X
Users > john > Desktop > Purchase_Dataset.xml
1  <?xml version='1.0' encoding='utf-8'?>
2  <Devices>
3    <Device>
4      <DeviceID>56609355-2510-4ac0-94ca-27fbc4079dd1</DeviceID>
5      <OwnerName>Tina West</OwnerName>
6      <IPAddress>221.212.19.52</IPAddress>
7      <OS>Linux</OS>
8      <OSVersion>9.8.7</OSVersion>
9      <PurchaseDate>2024-06-10</PurchaseDate>
10     <WarrantyStatus>Valid</WarrantyStatus>
11     <BatteryHealth>83</BatteryHealth>
12     <IsOnline>False</IsOnline>
13   </Device>
14   <Device>
15     <DeviceID>ef1e7691-3425-484e-b713-7fe629437ad7</DeviceID>
```

RESULT: Hence, large datasets were successfully created and exported using Python libraries.

EXP NO: 2**DATA PREPROCESSING****DATE:**

AIM: To perform various data preprocessing techniques

QUESTIONS:**1. Import and Explore Dataset****1.1 Import the required libraries.****1.2 Load the dataset into a Pandas DataFrame.****1.3 Display the first 5 and last 5 records of the dataset.****1.4 Determine the number of rows and columns in the dataset.****1.5 Display the data types of all attributes.****1.6 Generate summary statistics for the numerical features.****CODE:**

```
# 1.1 Import Libraries
import pandas as pd
import numpy as np

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt

# 1.2 Load Dataset
df = pd.read_csv("Bank_Marketing_Synthetic_500.csv")

# 1.3 Display First and Last 5 Records
print("First 5 Records")
print(df.head())

print("\nLast 5 Records")
print(df.tail())

# 1.4 Number of Rows and Columns
print("\nDataset Shape")
print(df.shape)

print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

# 1.5 Display Data Types
print("\nData Types")
print(df.dtypes)
```

```
# 1.6 Summary Statistics
print("\nSummary Statistics")
print(df.describe())
```

OUTPUT:

```
First 5 Records
  age  job  marital  education  default  balance  housing  loan  contact  \
0  56  retired  divorced  unknown  no  33262.0  no  yes  unknown
1  69  unknown  single  secondary  no  10677.0  yes  no  unknown
2  46  unknown  married  secondary  no  49871.0  yes  no  telephone
3  32  unknown  married  secondary  no  16268.0  no  no  telephone
4  60  retired  married  unknown  no  10330.0  yes  no  cellular

  day month  duration  campaign  pdays  previous  poutcome  y
0  26  dec  1387.0  11  244  5  other  no
1  4  may  145.0  11  310  5  unknown  no
2  10  mar  2125.0  11  58  8  failure  no
3  5  dec  1831.0  14  277  3  unknown  no
4  21  mar  2176.0  9  101  0  unknown  no

Last 5 Records
  age  job  marital  education  default  balance  housing  loan  \
505  47  services  divorced  primary  no  44026.0  yes  yes
506  29  blue-collar  divorced  primary  no  48019.0  no  no
507  52  retired  divorced  primary  no  39674.0  no  no
508  31  blue-collar  married  tertiary  yes  5758.0  yes  no
509  57  housemaid  single  unknown  no  40821.0  no  no

  contact  day month  duration  campaign  pdays  previous  poutcome  y
505  unknown  23  aug  1501.0  1  326  2  unknown  no
506  unknown  28  apr  166.0  9  403  1  other  no
507  telephone  27  sep  307.0  7  313  9  success  no
508  cellular  4  may  659.0  13  465  1  unknown  yes
509  unknown  27  nov  302.0  9  339  5  failure  no

Dataset Shape
(510, 17)
Rows: 510
Columns: 17

Data Types
age      int64
job      object
marital  object
education  object
default  object
balance  float64
housing  object
loan     object
contact  object
day      int64
month    object
duration  float64
campaign  int64
pdays   int64
previous  int64
poutcome  object
y         object
dtype: object

Summary Statistics
  count  age  balance  day  duration  campaign  \
mean    46.129412  24796.196040  16.945098  1438.736634  7.747059
std     16.162331  14632.699203  8.759666  840.784070  3.945695
min     18.000000  -1839.000000  1.000000  37.000000  1.000000
25%     33.000000  12289.000000  10.000000  682.000000  4.000000
50%     46.000000  25605.000000  17.000000  1439.000000  8.000000
75%     59.000000  36787.000000  24.000000  2131.000000  11.000000
max     74.000000  49871.000000  31.000000  2995.000000  14.000000

  pdays  previous
count  510.000000  510.000000
mean    248.596078  4.741176
std     145.958298  2.963377
min     -1.000000  0.000000
25%     113.250000  2.000000
50%     245.500000  5.000000
75%     376.500000  7.000000
max     498.000000  9.000000
```

2. Identify and Handle Missing Values**2.1 Identify missing values using Pandas functions.****2.2 Treat the value "unknown" as a missing value.****2.3 Calculate the percentage of missing values in each column.****2.4 Handle the missing values.****2.5 Verify that all missing values have been addressed.****CODE:**

```

# 2.1 Identify Missing Values
print("\nMissing Values")
print(df.isnull().sum())

# 2.2 Treat "unknown" as Missing Value
df.replace("unknown", np.nan, inplace=True)

print("\nMissing Values After Replacing Unknown")
print(df.isnull().sum())

# 2.3 Percentage of Missing Values
missing_percent = (df.isnull().sum()/len(df))*100

print("\nPercentage of Missing Values")
print(missing_percent)

# 2.4 Handle Missing Values
# Numerical Columns
num_cols = df.select_dtypes(include=np.number).columns

for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)

# Categorical Columns
cat_cols = df.select_dtypes(include='object').columns

for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

# 2.5 Verify Missing Values

print("\nMissing Values After Handling")
print(df.isnull().sum())

```

OUTPUT:

Missing Values		Missing Values After Replacing Unknown	
age	0	age	0
job	0	job	62
marital	0	marital	0
education	0	education	128
default	0	default	0
balance	5	balance	5
housing	0	housing	0
loan	0	loan	0
contact	0	contact	178
day	0	day	0
month	0	month	0
duration	5	duration	5
campaign	0	campaign	0
pdays	0	pdays	0
previous	0	previous	0
poutcome	0	poutcome	138
y	0	y	0
dtype: int64		dtype: int64	

Percentage of Missing Values		Missing Values After Handling	
age	0.000000	age	0
job	12.156863	job	0
marital	0.000000	marital	0
education	25.098039	education	0
default	0.000000	default	0
balance	0.980392	balance	0
housing	0.000000	housing	0
loan	0.000000	loan	0
contact	34.901961	contact	0
day	0.000000	day	0
month	0.000000	month	0
duration	0.980392	duration	0
campaign	0.000000	campaign	0
pdays	0.000000	pdays	0
previous	0.000000	previous	0
poutcome	27.058824	poutcome	0
y	0.000000	y	0
dtype: float64		dtype: int64	

3. Remove Duplicate Records**3.1 Find duplicate records.****3.2 Count the number of duplicate records.****3.3 Remove duplicate records.****3.4 Compare the dataset dimensions before and after duplicate removal.****CODE:**

```
before = df.shape
```

```
# 3.1 Find Duplicate Records
```

```
duplicates = df[df.duplicated()]
```

```
print("\nDuplicate Records")  
print(duplicates)
```

```
# 3.2 Count Duplicate Records
```

```
print("\nNumber of Duplicate Records")  
print(df.duplicated().sum())
```

```
# 3.3 Remove Duplicate Records
```

```
df = df.drop_duplicates()
```

```
# 3.4 Compare Dataset Dimensions
```

```
after = df.shape
```

```
print("\nBefore Removing Duplicates:", before)  
print("After Removing Duplicates :", after)
```

OUTPUT:

```

Duplicate Records
  age      job      marital  education  default  balance  housing  loan  \
500  58    retired  divorced   primary    no  14029.0    yes   no
501  21  housemaid  married   tertiary    no   3542.0    yes   no
502  34  housemaid  divorced  secondary    no  35861.0    no    no
503  22    admin.  married   primary    no   8142.0    no    no
504  42  unemployed  divorced  secondary    no  28620.0    no    no
506  29 blue-collar  divorced   primary    no  48019.0    no    no
507  52    retired  divorced   primary    no  39674.0    no    no
508  31 blue-collar  married   tertiary    yes   5758.0    yes   no
509  57  housemaid   single   primary    no  40821.0    no    no

  contact  day month  duration  campaign  pdays  previous  poutcome  y
500  telephone    8  feb    1962.0        7     69         7    other    no
501  telephone   20  apr    1835.0        2    230         9  failure  yes
502   cellular   28  mar    1079.0        9    368         3    other  yes
503   cellular   24  nov    2393.0        3     85         5  failure  no
504  telephone   29  mar    2298.0        6    329         7  failure  yes
506   cellular   28  apr     166.0        9    403         1    other  no
507  telephone   27  sep     307.0        7    313         9  success  no
508   cellular    4  may     659.0       13    465         1    other  yes
509   cellular   27  nov     302.0        9    339         5  failure  no

Number of Duplicate Records
9

Before Removing Duplicates: (510, 17)
After Removing Duplicates : (501, 17)

```

4. Perform Categorical Data Encoding**4.1 Identify the categorical attributes.****4.2 Apply Label Encoding to binary categorical variables.****4.3 Apply One-Hot Encoding to multi-categorical variables.****4.4 Compare the dataset before and after encoding.****CODE:**

```
# 4.1 Identify Categorical Attributes
```

```
categorical = df.select_dtypes(include='object').columns
```

```
print("\nCategorical Columns")
```

```
print(categorical)
```

```
encoded_df = df.copy()
```

```
# 4.2 Label Encoding for Binary Variables
```

```
le = LabelEncoder()
```

```
for col in categorical:
```

```
if encoded_df[col].nunique()==2:  
    encoded_df[col] = le.fit_transform(encoded_df[col])
```

4.3 One-Hot Encoding

```
multi_cols = []
```

```
for col in categorical:
```

```
    if df[col].nunique()>2:  
        multi_cols.append(col)
```

```
encoded_df = pd.get_dummies(encoded_df,  
                             columns=multi_cols,  
                             drop_first=True)
```

4.4 Compare Dataset

```
print("\nOriginal Dataset")  
print(df.head())
```

```
print("\nEncoded Dataset")  
print(encoded_df.head())
```

OUTPUT:

```

Categorical Columns
Index(['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact',
      'month', 'poutcome', 'y'],
      dtype='object')

Original Dataset
  age  job  marital  education  default  balance  housing  loan  \
0  56  retired  divorced  primary     no  33262.0     no   yes
1  69  housemaid  single  secondary  no  10677.0     yes  no
2  46  housemaid  married  secondary  no  49871.0     yes  no
3  32  housemaid  married  secondary  no  16268.0     no   no
4  60  retired  married  primary     no  10330.0     yes  no

  contact  day  month  duration  campaign  pdays  previous  poutcome  y
0  cellular  26  dec    1387.0      11    244         5    other  no
1  cellular   4  may     145.0      11    310         5    other  no
2  telephone 10  mar    2125.0      11     58         8  failure  no
3  telephone  5  dec    1831.0      14    277         3    other  no
4  cellular  21  mar    2176.0       9    101         0    other  no

Encoded Dataset
  age  default  balance  housing  loan  contact  day  duration  campaign  \
0  56         0  33262.0        0     1         0  26    1387.0      11
1  69         0  10677.0        1     0         0   4     145.0      11
2  46         0  49871.0        1     0         1  10    2125.0      11
3  32         0  16268.0        0     0         1   5    1831.0      14
4  60         0  10330.0        1     0         0  21    2176.0       9

  pdays  ...  month_jan  month_jul  month_jun  month_mar  month_may  \
0    244  ...      False      False      False      False      False
1    310  ...      False      False      False      False      True
2     58  ...      False      False      False      True      False
3    277  ...      False      False      False      False      False
4    101  ...      False      False      False      True      False

  month_nov  month_oct  month_sep  poutcome_other  poutcome_success
0      False      False      False          True          False
1      False      False      False          True          False
2      False      False      False          False          False
3      False      False      False          True          False
4      False      False      False          True          False

[5 rows x 39 columns]

```

5. Normalize and Standardize Numerical Values**5.1 Identify the numerical features.****5.2 Apply Min-Max Normalization.****5.3 Apply Z-score Standardization.****5.4 Compare the transformed values with the original dataset.**

CODE:

5.1 Numerical Features

```
numerical = encoded_df.select_dtypes(include=np.number).columns
```

```
print("\nNumerical Features")  
print(numerical)
```

5.2 Min-Max Normalization

```
minmax = MinMaxScaler()
```

```
normalized = encoded_df.copy()
```

```
normalized[numerical] = minmax.fit_transform(normalized[numerical])
```

```
print("\nMin-Max Normalized Data")  
print(normalized.head())
```

5.3 Standardization (Z-score)

```
standard = StandardScaler()
```

```
standardized = encoded_df.copy()
```

```
standardized[numerical] = standard.fit_transform(standardized[numerical])
```

```
print("\nStandardized Data")  
print(standardized.head())
```

5.4 Compare

```
print("\nOriginal Values")  
print(encoded_df[numerical].head())
```

```
print("\nNormalized Values")  
print(normalized[numerical].head())
```

```
print("\nStandardized Values")  
print(standardized[numerical].head())
```

OUTPUT:

```

Numerical Features
Index(['age', 'default', 'balance', 'housing', 'loan', 'contact', 'day',
      'duration', 'campaign', 'pdays', 'previous', 'y'],
      dtype='object')

Min-Max Normalized Data
   age  default  balance  housing  loan  contact  day  duration \
0  0.678571    0.0  0.678805    0.0    1.0    0.0  0.833333  0.456389
1  0.910714    0.0  0.242042    1.0    0.0    0.0  0.100000  0.036511
2  0.500000    0.0  1.000000    1.0    0.0    1.0  0.300000  0.705882
3  0.250000    0.0  0.350164    0.0    0.0    1.0  0.133333  0.606491
4  0.750000    0.0  0.235332    1.0    0.0    0.0  0.666667  0.723124

   campaign  pdays  ...  month_jan  month_jul  month_jun  month_mar \
0  0.769231  0.490982  ...    False    False    False    False
1  0.769231  0.623246  ...    False    False    False    False
2  0.769231  0.118236  ...    False    False    False    True
3  1.000000  0.557114  ...    False    False    False    False
4  0.615385  0.204409  ...    False    False    False    True

   month_may  month_nov  month_oct  month_sep  poutcome_other \
0    False    False    False    False    True
1    True    False    False    False    True
2    False    False    False    False    False
3    False    False    False    False    True
4    False    False    False    False    True

   poutcome_success
0    False
1    False
2    False
3    False
4    False

[5 rows x 39 columns]

Standardized Data
   age  default  balance  housing  loan  contact  day \
0  0.602435 -0.224309  0.582861 -1.134756  2.053960 -0.684988  1.047323
1  1.407124 -0.224309 -0.973098  0.881247 -0.486864 -0.684988 -1.473662
2 -0.016556 -0.224309  1.727113  0.881247 -0.486864  1.459880 -0.786121
3 -0.883144 -0.224309 -0.587914 -1.134756 -0.486864  1.459880 -1.359072
4  0.850032 -0.224309 -0.997004  0.881247 -0.486864 -0.684988  0.474372

   duration  campaign  pdays  ...  month_jan  month_jul  month_jun \
0 -0.066623  0.820333 -0.026507  ...    False    False    False
1 -1.554135  0.820333  0.425526  ...    False    False    False
2  0.817261  0.820333 -1.300421  ...    False    False    False
3  0.465145  1.579078  0.199510  ...    False    False    False
4  0.878343  0.314503 -1.005914  ...    False    False    False

```

```

2      True      False      False      False      False      False      False
3      False     False     False     False     False     False      True
4      True      False     False     False     False     False      True

  poutcome_success
0           False
1           False
2           False
3           False
4           False

[5 rows x 39 columns]

Original Values
  age  default  balance  housing  loan  contact  day  duration  campaign \
0   56         0  33262.0         0     1         0   26   1387.0        11
1   69         0  10677.0         1     0         0    4    145.0        11
2   46         0  49871.0         1     0         1   10   2125.0        11
3   32         0  16268.0         0     0         1    5   1831.0        14
4   60         0  10330.0         1     0         0   21   2176.0         9

  pdays  previous  y
0    244         5  0
1    310         5  0
2     58         8  0
3    277         3  0
4    101         0  0

Normalized Values
  age  default  balance  housing  loan  contact  day  duration \
0 0.678571    0.0  0.678805    0.0    1.0    0.0  0.833333  0.456389
1 0.910714    0.0  0.242042    1.0    0.0    0.0  0.100000  0.036511
2 0.500000    0.0  1.000000    1.0    0.0    1.0  0.300000  0.705882
3 0.250000    0.0  0.350164    0.0    0.0    1.0  0.133333  0.606491
4 0.750000    0.0  0.235332    1.0    0.0    0.0  0.666667  0.723124

  campaign  pdays  previous  y
0 0.769231  0.490982  0.555556  0.0
1 0.769231  0.623246  0.555556  0.0
2 0.769231  0.118236  0.888889  0.0
3 1.000000  0.557114  0.333333  0.0
4 0.615385  0.204409  0.000000  0.0

Standardized Values
  age  default  balance  housing  loan  contact  day \
0 0.602435 -0.224309  0.582861 -1.134756  2.053960 -0.684988  1.047323
1 1.407124 -0.224309 -0.973098  0.881247 -0.486864 -0.684988 -1.473662
2 -0.016556 -0.224309  1.727113  0.881247 -0.486864  1.459880 -0.786121
3 -0.883144 -0.224309 -0.587914 -1.134756 -0.486864  1.459880 -1.359072
4 0.850032 -0.224309 -0.997004  0.881247 -0.486864 -0.684988  0.474372

  duration  campaign  pdays  previous  y
0 -0.066623  0.820333 -0.026507  0.090333 -0.42615
1 -1.554135  0.820333  0.425526  0.090333 -0.42615
2 0.817261  0.820333 -1.300421  1.103547 -0.42615
3 0.465145  1.579078  0.199510 -0.585143 -0.42615
4 0.878343  0.314503 -1.005914 -1.598356 -0.42615

```

6. Detect and Handle Outliers**6.1 Plot boxplots for the numerical attributes.****6.2 Detect outliers using the IQR method.****6.3 Remove or cap the outliers.****6.4 Compare the dataset before and after outlier treatment.****CODE:**

Add some outliers

df.loc[0, "balance"] = 150000

df.loc[1, "balance"] = 200000

df.loc[2, "balance"] = -50000

df.loc[3, "duration"] = 10000

df.loc[4, "duration"] = 12000

df.loc[5, "campaign"] = 50

df.loc[6, "campaign"] = 60

df.loc[7, "age"] = 99

df.loc[8, "age"] = 100

numeric = df.select_dtypes(include=np.number).columns

6.1 Boxplots

for col in numeric:

```
plt.figure(figsize=(5,2))
plt.boxplot(df[col], vert=False)
plt.title(col)
plt.show()
```

6.2 Detect Outliers using IQR

outlier_count = {}

for col in numeric:

Q1 = df[col].quantile(0.25)

Q3 = df[col].quantile(0.75)

IQR = Q3-Q1

lower = Q1 - 1.5*IQR

upper = Q3 + 1.5*IQR

outliers = df[(df[col]<lower)|(df[col]>upper)]

```

    outlier_count[col] = len(outliers)

print("\nOutlier Count")

for k,v in outlier_count.items():
    print(k,":",v)

# 6.3 Cap Outliers

treated_df = df.copy()

for col in numeric:

    Q1 = treated_df[col].quantile(0.25)
    Q3 = treated_df[col].quantile(0.75)

    IQR = Q3-Q1

    lower = Q1 - 1.5*IQR
    upper = Q3 + 1.5*IQR

    treated_df[col] = np.where(
        treated_df[col]<lower,
        lower,
        treated_df[col]
    )

    treated_df[col] = np.where(
        treated_df[col]>upper,
        upper,
        treated_df[col]
    )

# 6.4 Compare

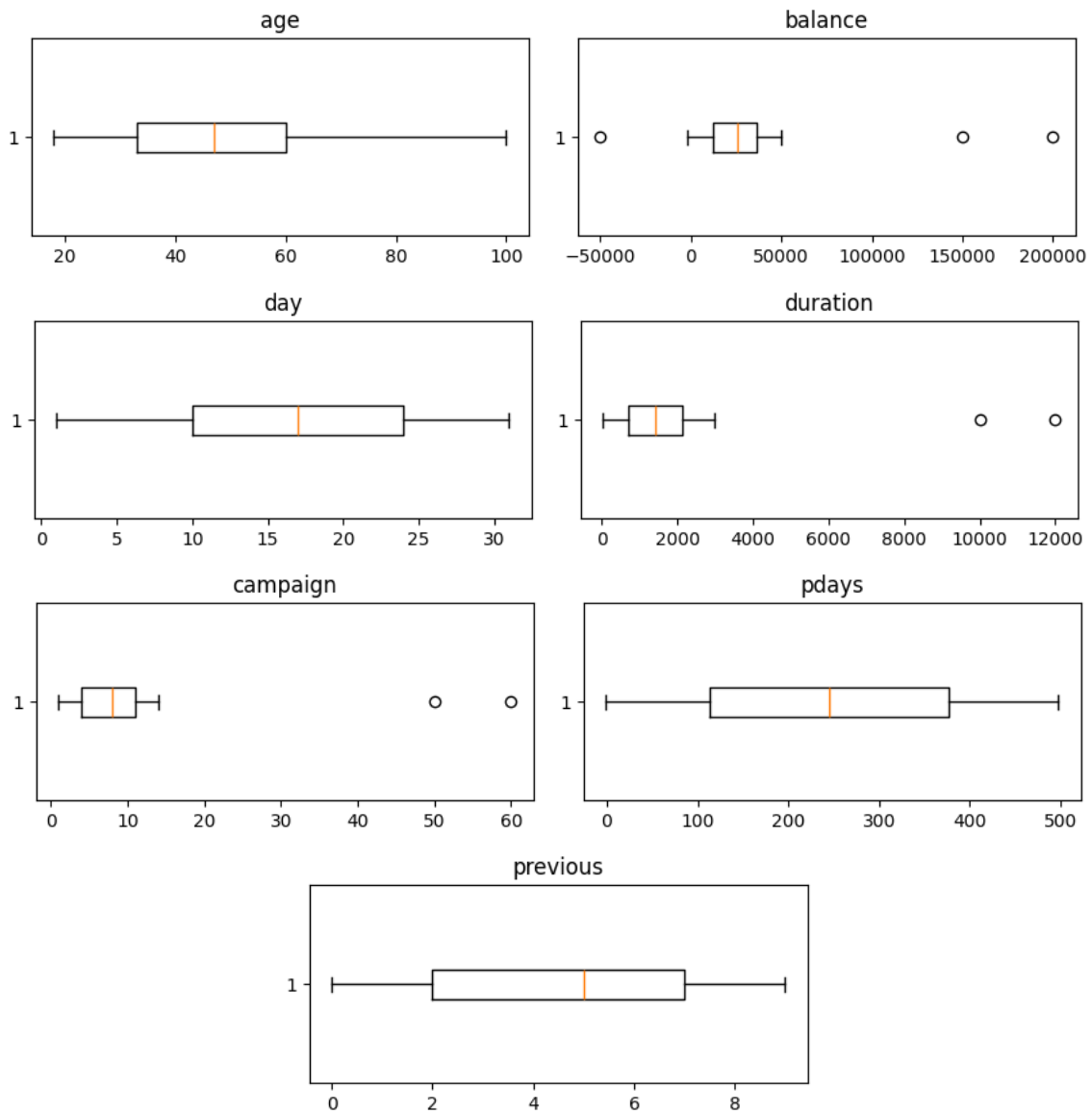
print("\nOriginal Dataset Shape")
print(df.shape)

print("\nAfter Outlier Treatment")
print(treated_df.shape)

print("\nOriginal Data")
print(df.head())

print("\nAfter Outlier Treatment")
print(treated_df.head())

```

OUTPUT:

Outlier Count

age : 0
 balance : 3
 day : 0
 duration : 2
 campaign : 2
 pdays : 0
 previous : 0

Original Dataset Shape
 (501, 17)

After Outlier Treatment
 (501, 17)

Original Data

	age	job	marital	education	default	balance	housing	loan	\
0	56	retired	divorced	primary	no	150000.0	no	yes	
1	69	housemaid	single	secondary	no	200000.0	yes	no	
2	46	housemaid	married	secondary	no	-50000.0	yes	no	
3	32	housemaid	married	secondary	no	16268.0	no	no	
4	60	retired	married	primary	no	10330.0	yes	no	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	cellular	26	dec	1387.0	11	244	5	other	no
1	cellular	4	may	145.0	11	310	5	other	no
2	telephone	10	mar	2125.0	11	58	8	failure	no
3	telephone	5	dec	10000.0	14	277	3	other	no
4	cellular	21	mar	12000.0	9	101	0	other	no

After Outlier Treatment

	age	job	marital	education	default	balance	housing	loan	\
0	56.0	retired	divorced	primary	no	73077.0	no	yes	
1	69.0	housemaid	single	secondary	no	73077.0	yes	no	
2	46.0	housemaid	married	secondary	no	-23891.0	yes	no	
3	32.0	housemaid	married	secondary	no	16268.0	no	no	
4	60.0	retired	married	primary	no	10330.0	yes	no	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	cellular	26.0	dec	1387.0	11.0	244.0	5.0	other	no
1	cellular	4.0	may	145.0	11.0	310.0	5.0	other	no
2	telephone	10.0	mar	2125.0	11.0	58.0	8.0	failure	no
3	telephone	5.0	dec	4274.5	14.0	277.0	3.0	other	no
4	cellular	21.0	mar	4274.5	9.0	101.0	0.0	other	no

7. Perform Feature Engineering

7.1 Create age groups such as Young, Middle-aged, Seniors.

7.2 Categorise account balances into Low, Medium, and High.

7.3 Display the updated dataset.

CODE:

#7.1 Age groups

```
age_bins = [0, 35, 55, np.inf]
```

```
age_labels = ['Young', 'Middle-aged', 'Senior']
```

```
df['AgeGroup'] = pd.cut(df['age'], bins=age_bins, labels=age_labels, right=True)
```

#7.2 Account Balance groups

```
balance_bins = [0, 5000, 15000, np.inf]
```

```
balance_labels = ['Low', 'Medium', 'High']
```

```
df['BalanceCategory'] = pd.cut(df['balance'], bins=balance_bins, labels=balance_labels, right=True)
```

#7.3 Display

```
df
```

OUTPUT:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y	AgeGroup	BalanceCategory
0	56	retired	divorced	primary	no	150000.0	no	yes	cellular	26	dec	1387.0	11	244	5	other	no	Senior	High
1	69	housemaid	single	secondary	no	200000.0	yes	no	cellular	4	may	145.0	11	310	5	other	no	Senior	High
2	46	housemaid	married	secondary	no	-50000.0	yes	no	telephone	10	mar	2125.0	11	58	8	failure	no	Middle-aged	NaN
3	32	housemaid	married	secondary	no	16268.0	no	no	telephone	5	dec	10000.0	14	277	3	other	no	Young	High
4	60	retired	married	primary	no	10330.0	yes	no	cellular	21	mar	12000.0	9	101	0	other	no	Senior	Medium
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
496	60	student	single	secondary	no	28682.0	yes	no	cellular	6	oct	949.0	9	432	8	other	no	Senior	High
497	54	services	single	tertiary	no	30262.0	no	no	cellular	31	jul	2369.0	4	123	3	success	no	Middle-aged	High
498	50	student	married	tertiary	no	29358.0	no	no	cellular	6	sep	1999.0	7	79	8	other	no	Middle-aged	High
499	25	unemployed	single	primary	no	12289.0	no	no	cellular	24	sep	785.0	2	483	1	success	no	Young	Medium
505	47	services	divorced	primary	no	44026.0	yes	yes	cellular	23	aug	1501.0	1	326	2	other	no	Middle-aged	High

501 rows x 19 columns

8. Perform Feature Selection

8.1 Compute the correlation matrix for numerical variables.

8.2 Visualize correlations using a heatmap.

8.3 Remove highly correlated features, if any.

8.4 Identify the most relevant features for predicting customer subscription.

CODE:

#8.1 Correlation matrix

```
corr_matrix = df.corr(numeric_only=True)
```

```
print("\nCorrelation Matrix:\n", corr_matrix)
```

#8.2 Heatmap visualisation

```
!pip install seaborn
```

```
import seaborn as sns
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

#8.3 Removal of high;y correlated features

```
threshold = 0.5
```

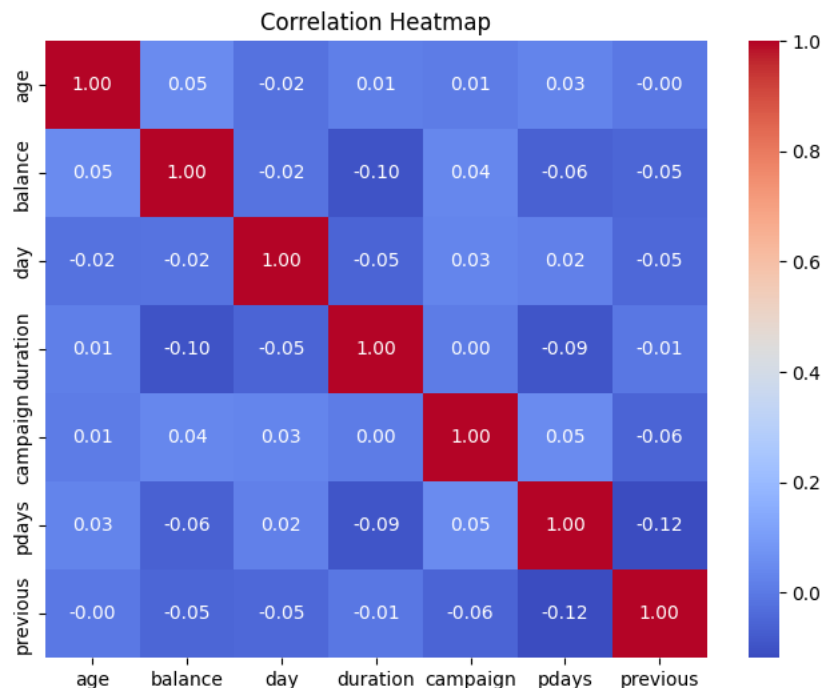
```
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [column for column in upper_tri.columns if any(upper_tri[column].abs() >
threshold)]
print("Highly correlated features to drop:", to_drop)
```

#8.4 Most relevant features for subscription

```
y_numeric = df_reduced['y'].map({'yes': 1, 'no': 0})
numeric_df = df_reduced.select_dtypes(include=['number'])
correlations = numeric_df.apply(lambda col: col.corr(y_numeric)).abs()
relevant_features = correlations.sort_values(ascending=False)
print("\nMost Relevant Numerical Features for Predicting Subscription:")
print(relevant_features)
```

OUTPUT:

```
Correlation Matrix:
          age  balance  day  duration  campaign  pdays  previous
age      1.000000  0.050481 -0.018648  0.012800  0.005759  0.027218 -0.004520
balance  0.050481  1.000000 -0.016768 -0.098274  0.038116 -0.062032 -0.048985
day      -0.018648 -0.016768  1.000000 -0.053413  0.033793  0.021051 -0.046791
duration  0.012800 -0.098274 -0.053413  1.000000  0.004680 -0.094637 -0.006718
campaign  0.005759  0.038116  0.033793  0.004680  1.000000  0.051114 -0.056918
pdays   0.027218 -0.062032  0.021051 -0.094637  0.051114  1.000000 -0.118763
previous -0.004520 -0.048985 -0.046791 -0.006718 -0.056918 -0.118763  1.000000
```



Highly correlated features to drop: []

Most Relevant Numerical Features for Predicting Subscription:

```
previous    0.068048
day         0.047717
pdays      0.041544
campaign    0.025719
balance     0.021007
duration    0.019670
age         0.007371
dtype: float64
```

9. Build a Complete Data Preprocessing Pipeline

9.1 Load the dataset. Handle missing values.

9.2 Remove duplicates.

9.3 Encode categorical variables.

9.4 Normalize numerical attributes.

9.5 Detect and treat outliers.

9.6 Select relevant features.

9.7 Split the dataset into training and testing sets.

9.8 Save the preprocessed dataset as a new CSV file.

CODE:

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

def remove_duplicates_and_outliers(df, numeric_cols):
    df = df.drop_duplicates()
    for col in numeric_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
    return df

df = pd.read_csv("Bank_Marketing_Synthetic_500_1.csv")

numeric_features = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = df.select_dtypes(exclude=[np.number]).columns.tolist()
```

```

target_column = "y" if "y" in df.columns else df.columns[-1]

if target_column in numeric_features:
    numeric_features.remove(target_column)
if target_column in categorical_features:
    categorical_features.remove(target_column)

df_cleaned = remove_duplicates_and_outliers(df, numeric_features)

X = df_cleaned[numeric_features + categorical_features]
y = df_cleaned[target_column]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

numeric_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="mean")),
        ("scaler", StandardScaler()),
    ]
)

categorical_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_pipeline, numeric_features),
        ("cat", categorical_pipeline, categorical_features),
    ]
)

k_features = min(5, len(numeric_features) + len(categorical_features))

full_pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("feature_selection", SelectKBest(score_func=f_classif, k=k_features)),
    ]
)

X_train_processed = full_pipeline.fit_transform(X_train, y_train)
X_test_processed = full_pipeline.transform(X_test)

```

```
feature_names = full_pipeline.named_steps["preprocessor"].get_feature_names_out()
selected_mask = full_pipeline.named_steps["feature_selection"].get_support()
selected_feature_names = feature_names[selected_mask]
```

```
df_train_processed = pd.DataFrame(
    X_train_processed, columns=selected_feature_names
)
df_train_processed[target_column] = y_train.values
```

```
df_test_processed = pd.DataFrame(
    X_test_processed, columns=selected_feature_names
)
df_test_processed[target_column] = y_test.values
```

```
df_train_processed.to_csv("train_preprocessed.csv", index=False)
df_test_processed.to_csv("test_preprocessed.csv", index=False)
```

OUTPUT:

	A	B	C	D	E	F
	test_preprocessed					
	num_campaign	num_previous	cat_job_blue-collar	cat_month_jun	cat_month_sep	y
1						
2	-0.9266922008397330	-0.5431806344167470	0.0	1.0	0.0	no
3	0.5590474422243790	-0.20786282545396800	0.0	0.0	0.0	no
4	0.31142416838036	1.4687262193599300	0.0	0.0	0.0	no
5	-0.679068926995714	0.4627727924715910	0.0	0.0	0.0	no
6	-0.4314456531516960	0.7980906014343700	0.0	1.0	0.0	no
7	0.31142416838036	0.7980906014343700	0.0	0.0	0.0	yes
8	0.5590474422243790	-0.878498443379526	0.0	0.0	0.0	no
9	0.31142416838036	0.4627727924715910	1.0	0.0	0.0	no
10	1.0542939899124200	0.4627727924715910	0.0	0.0	0.0	no
11	-0.679068926995714	1.1334084103971500	0.0	0.0	0.0	no
12	-1.174315474683750	-0.878498443379526	0.0	0.0	0.0	no
13	0.806670716068397	1.4687262193599300	0.0	0.0	0.0	no
14	0.31142416838036	1.4687262193599300	0.0	1.0	0.0	no
15	1.0542939899124200	-0.5431806344167470	0.0	0.0	0.0	no
16	-1.669562022371790	-1.213816252342310	0.0	0.0	0.0	no
17	-1.4219387485277700	1.4687262193599300	0.0	0.0	0.0	yes
18	-0.183822379307677	0.7980906014343700	1.0	0.0	0.0	no
19	0.5590474422243790	-0.20786282545396800	0.0	0.0	0.0	no
20	0.31142416838036	-1.5491340613050800	0.0	0.0	0.0	no

	A	B	C	D	E	F
	train_preprocessed					
1	num_campaign	num_previous	cat_job_blue-collar	cat_month_jun	cat_month_sep	y
2	1.301917263756430	1.4687262193599300	0.0	0.0	0.0	no
3	1.5495405376004500	1.1334084103971500	0.0	0.0	0.0	no
4	-1.4219387485277700	-0.878498443379526	0.0	0.0	0.0	no
5	-1.4219387485277700	-1.213816252342310	0.0	0.0	1.0	no
6	1.0542939899124200	-1.5491340613050800	0.0	0.0	0.0	no
7	-0.9266922008397330	1.4687262193599300	0.0	0.0	0.0	no
8	-0.4314456531516960	-0.878498443379526	0.0	0.0	0.0	no
9	-0.679068926995714	0.4627727924715910	0.0	0.0	0.0	no
10	-0.679068926995714	-0.878498443379526	0.0	0.0	0.0	no
11	0.06380089453634150	1.1334084103971500	0.0	0.0	0.0	no
12	0.31142416838036	0.7980906014343700	0.0	0.0	1.0	no
13	-1.4219387485277700	0.4627727924715910	0.0	0.0	0.0	no
14	1.0542939899124200	1.4687262193599300	0.0	0.0	0.0	no
15	-1.174315474683750	-0.878498443379526	1.0	0.0	0.0	no
16	0.31142416838036	-0.5431806344167470	0.0	0.0	0.0	no
17	-0.9266922008397330	-0.20786282545396800	0.0	0.0	0.0	no
18	-1.174315474683750	-0.20786282545396800	0.0	0.0	0.0	no
19	0.31142416838036	1.4687262193599300	0.0	0.0	0.0	no
20	1.301917263756430	-0.5431806344167470	0.0	0.0	0.0	no

RESULT: Hence, various data preprocessing techniques were performed.

EXP NO: 3**WEB SCRAPING AND VISUALISATION****DATE:**

AIM: To scrape headlines from the given website and analyse the most frequent words.

QUESTIONS:

1. Fetch HTML content from <https://www.espncricinfo.com/> and extract all headlines present in <h2> and <h3> tags. Print the first 5 headlines.

CODE:

```

pip install requests beautifulsoup4

pip install curl-cffi beautifulsoup4

from bs4 import BeautifulSoup
from curl_cffi import requests

url = "https://www.espncricinfo.com/"

try:
    response = requests.get(url, impersonate="chrome", timeout=10)
    response.raise_for_status()

    soup = BeautifulSoup(response.text, "html.parser")

    headlines = []
    for tag in soup.find_all(["h2", "h3"]):
        text = tag.get_text(strip=True)

        if text and text not in headlines:
            headlines.append(text)

    print("--- Top 5 Headlines from Cricinfo ---")
    for i, headline in enumerate(headlines[:5], 1):
        print(f'{i}. {headline}')

except Exception as e:
    print(f'An error occurred while fetching the page: {e}')

```

OUTPUT:

```

--- Top 5 Headlines from Cricinfo ---
1. Match Coverage
2. Salt: 'Sometimes you've got to fight harder'
3. Gambhir: 'Middle order needs to play the situation'
4. Watch: India capitulate in powerplay
5. Watch – India all out for 76

```

2. From extracted headlines:

- **Convert text to lowercase**
- **Remove numbers, punctuations and special characters**
- **Remove common stop words**
- **Display first 20 cleaned words**

CODE:

```

import re
import string
from bs4 import BeautifulSoup
from curl_cffi import requests

STOPWORDS = {
    "the", "and", "to", "of", "in", "for", "on", "at", "with", "a", "an", "by", "is", "are", "this",
    "that"
}
all_text = ".join(headlines)
if all_text:
    lowercase_text = all_text.lower()

    clean_text = re.sub(r"[^a-z\s]", "", lowercase_text)

    words = clean_text.split()

    filtered_words = [word for word in words if word not in STOPWORDS]

    print("--- First 20 Cleaned Words ---")
    print(filtered_words[:20])

```

OUTPUT:

```

--- First 20 Cleaned Words ---

```

```

['match', 'coverage', 'salt', 'sometimes', 'youve', 'got', 'fight', 'harder', 'gambhir', 'middle', 'order',
'needs', 'play', 'situation', 'watch', 'india', 'capitulate', 'powerplay', 'watch', 'india']

```

```

--- First 20 Cleaned Words ---
['match', 'coverage', 'salt', 'sometimes', 'youve', 'got', 'fight', 'harder', 'gambhir', 'middle', 'order', 'needs', 'play', 'situation', 'watch',

```

3. Count the frequency of all words in cleaned text and display the Top 10 most frequent words along with their count.

CODE:

```
pip install matplotlib wordcloud

import collections
import re
from bs4 import BeautifulSoup
from curl_cffi import requests
import matplotlib.pyplot as plt
from wordcloud import WordCloud

word_counts = collections.Counter(filtered_words)
sorted_word_counts = word_counts.most_common()

print("--- Word Frequencies (Descending Order) ---")
for word, count in sorted_word_counts[:10]:
    print(f'{word}: {count}')
```

OUTPUT:

```
--- Word Frequencies (Descending Order) ---
watch: 11
out: 6
two: 4
india: 3
play: 2
all: 2
archer: 2
sooryavanshi: 2
ti: 2
pretorius: 2
```

4. Plot the frequency and words as a Bar Chart and a Word Cloud Image. Save it as an image.

CODE:

```
top_words = [item[0] for item in sorted_word_counts[:10]]
top_counts = [item[1] for item in sorted_word_counts[:10]]

plt.figure(figsize=(10, 5))
plt.bar(top_words, top_counts, color="skyblue", edgecolor="black")
plt.title("Top 10 Most Common Words")
plt.xlabel("Words")
plt.ylabel("Frequency")
```


5. Visualise the Word Cloud Image using matplotlib.**CODE:**

```

import random
if all_text:
    word_counts = collections.Counter(filtered_words)
    top_words = word_counts.most_common(40)
    max_count = top_words[0][1] if top_words else 1

fig, ax = plt.subplots(figsize=(10, 6), facecolor="white")
colors = [
    "#1f77b4",
    "#ff7f0e",
    "#2ca02c",
    "#d62728",
    "#9467bd",
    "#8c564b",
    "#e377c2",
    "#17becf",
]

for word, count in top_words:
    font_size = 12 + int((count / max_count) * 36)
    x = random.uniform(0.1, 0.9)
    y = random.uniform(0.1, 0.9)

    ax.text(
        x,
        y,
        word,
        fontsize=font_size,
        color=random.choice(colors),
        ha="center",
        va="center",
        alpha=0.85,
        weight="bold",
    )

ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.axis("off")

plt.title("Pure Matplotlib Word Cloud", fontsize=16, pad=20, weight="bold")
plt.tight_layout()
plt.savefig("word_cloud.png", dpi=300, bbox_inches="tight", facecolor="white")
plt.show()

```


EXP NO: 4
DATE: 16/7/26

DATA VISUALISATION IN DIFFERENT DIMENSIONS

AIM: To visualise data in different dimensions (1D, 2D, 3D and higher) using python libraries and to extract meaningful patterns and insights.

QUESTIONS:

I. Data Visualization Using the Iris Dataset

1.1 Plot a histogram for the sepal length feature of the Iris dataset. Analyse the shape of its distribution.

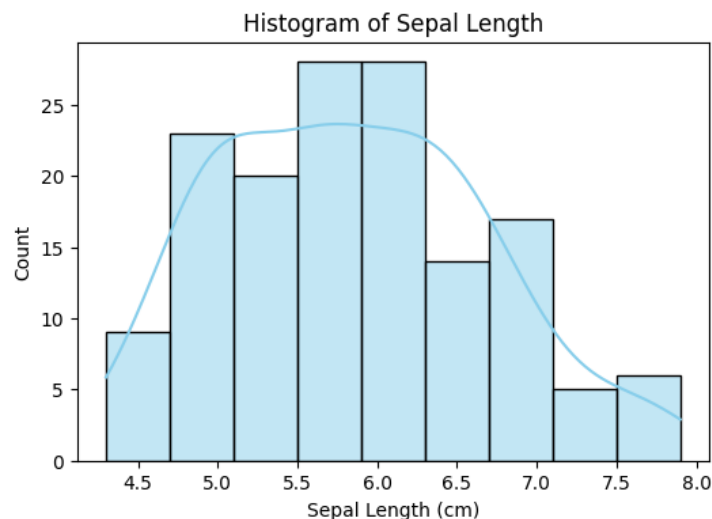
CODE:

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

plt.figure(figsize=(6, 4))
sns.histplot(
    data=df, x="sepal length (cm)", kde=True, color="skyblue"
)
plt.title("Histogram of Sepal Length")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Count")
plt.show()
```

OUTPUT:



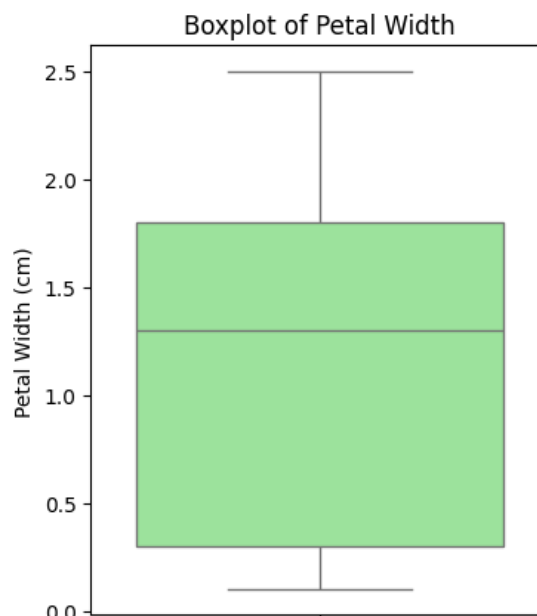
INFERENCE: The histogram shows that the sepal length is approximately normally distributed with a slight positive (right) skew. The distribution is nearly symmetric and bell-shaped, indicating that most values are concentrated around the mean, with a few observations extending toward higher sepal lengths.

1.2 Draw a box plot for the petal width feature of the Iris dataset. Identify any outliers, if present.

CODE:

```
plt.figure(figsize=(4, 5))
sns.boxplot(data=df, y="petal width (cm)", color="lightgreen")
plt.title("Boxplot of Petal Width")
plt.ylabel("Petal Width (cm)")
plt.show()
```

OUTPUT:



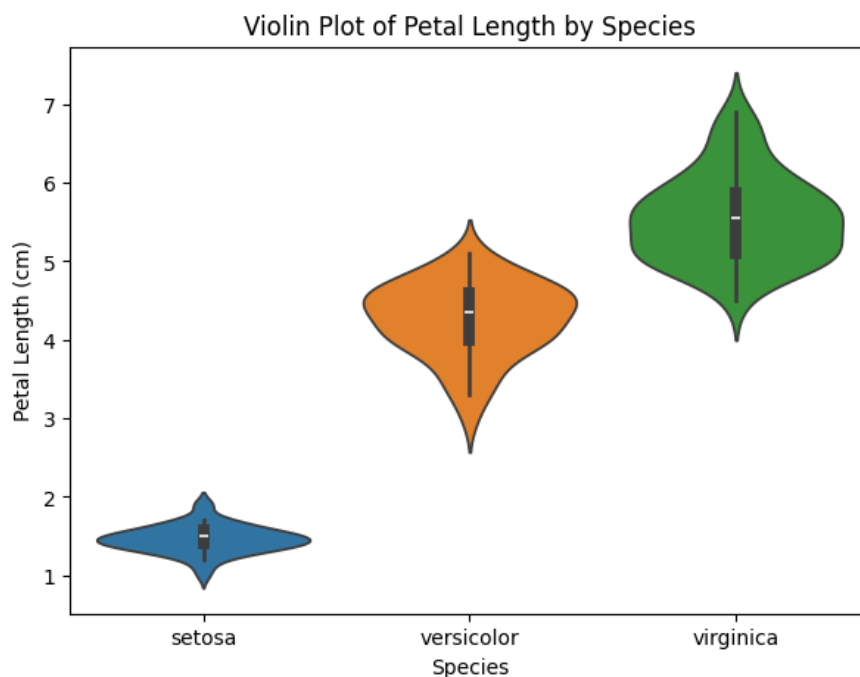
INFERENCE: The box plot indicates that there are no significant outliers in the petal width feature. All observations lie within the upper and lower whiskers, showing that the data is well distributed without any extreme values.

1.3 Create a violin plot for the petal length feature categorized by species. Interpret the observations from the plot.

CODE:

```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
species_map = {0: "setosa", 1: "versicolor", 2: "virginica"}
df["species"] = [species_map[x] for x in iris.target]
```

```
plt.figure(figsize=(7, 5))
sns.violinplot(
    data=df,
    x="species",
    y="petal length (cm)",
    hue="species",
    legend=False,
)
plt.title("Violin Plot of Petal Length by Species")
plt.xlabel("Species")
plt.ylabel("Petal Length (cm)")
plt.show()
```

OUTPUT:

INFERENCE: The violin plot shows that Setosa has a distinct and compact distribution of petal lengths, making it clearly separable from the other two species. Versicolor and Virginica exhibit slightly overlapping distributions, indicating that petal length alone can effectively distinguish Setosa but may require additional features to accurately differentiate Versicolor and Virginica.

II. Two-Dimensional Visualization Using the Iris Dataset

1.4 Generate a scatter plot of sepal length versus sepal width, color-coded by species. What pattern can be observed from the plot?

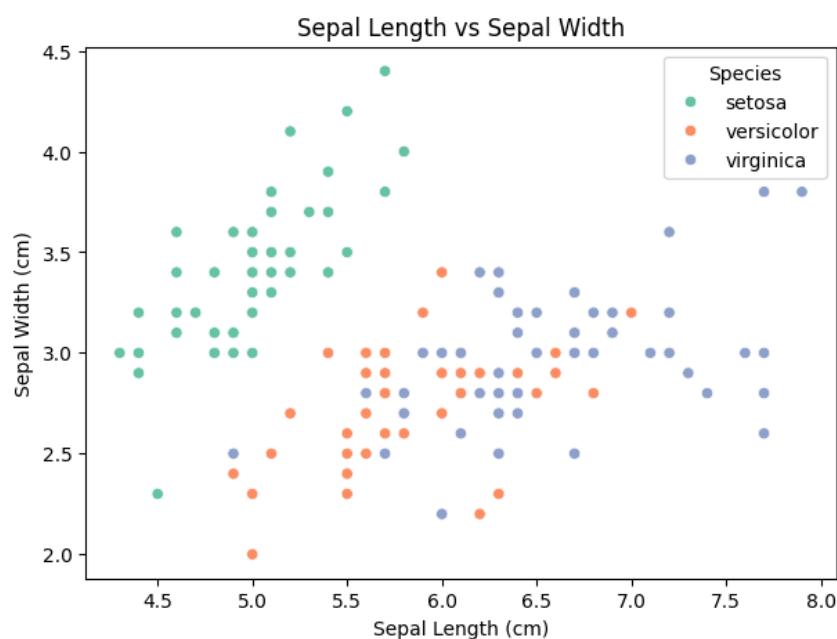
CODE:

```
plt.figure(figsize=(7, 5))
sns.scatterplot(
```

```

data=df,
x="sepal length (cm)",
y="sepal width (cm)",
hue="species",
palette="Set2",
)
plt.title("Sepal Length vs Sepal Width")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.legend(title="Species")
plt.show()

```

OUTPUT:

INFERENCE: The scatter plot shows that Setosa forms a distinct cluster with shorter sepal lengths and wider sepal widths. In contrast, Versicolor and Virginica exhibit considerable overlap, indicating that sepal measurements alone are not sufficient to clearly distinguish these two species.

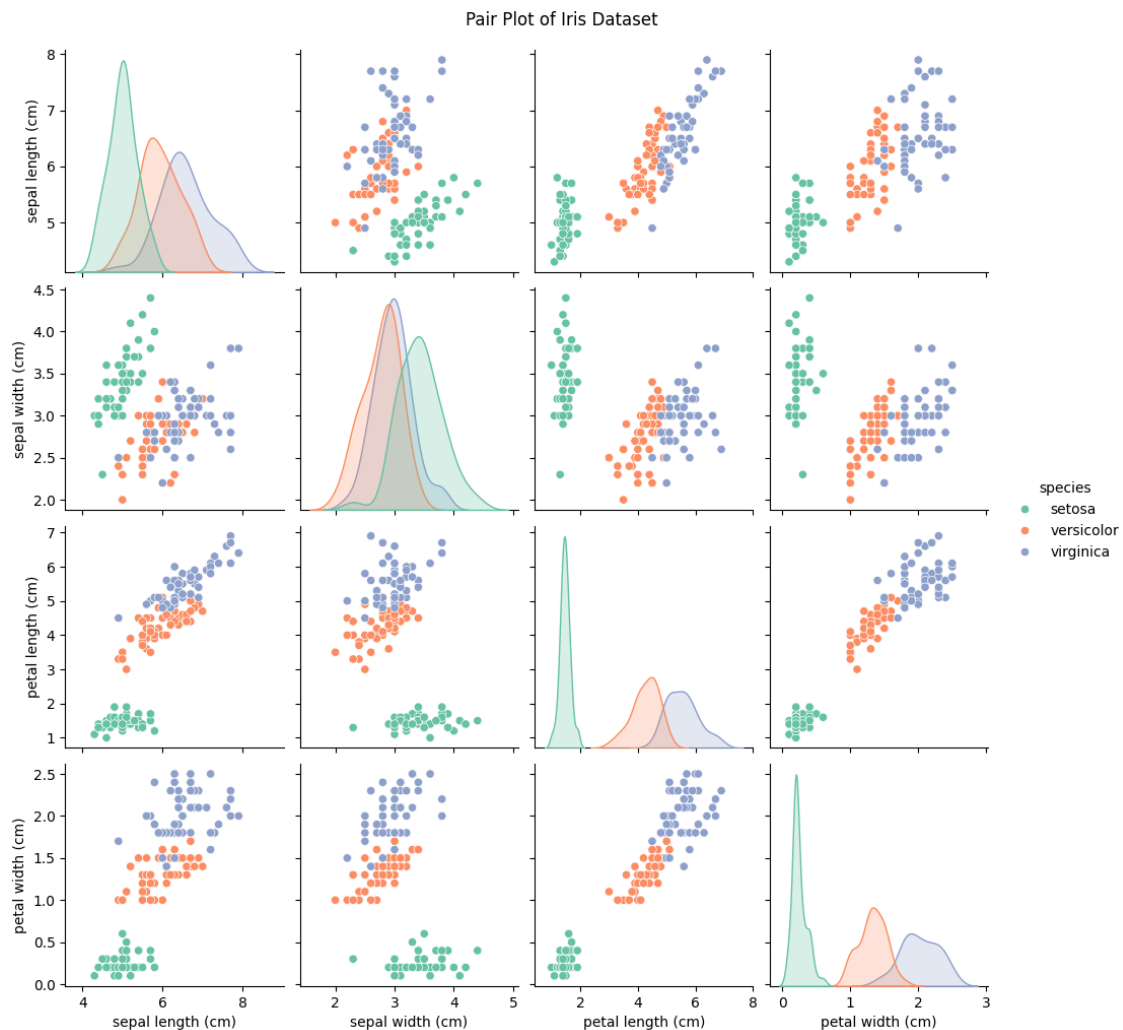
1.5 Create a pair plot (scatter matrix) for the Iris dataset. How does this visualization help in understanding the relationships among features?

CODE:

```

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
species_map = {0: "setosa", 1: "versicolor", 2: "virginica"}
df["species"] = [species_map[x] for x in iris.target]
sns.pairplot(data=df, hue="species", palette="Set2")
plt.suptitle("Pair Plot of Iris Dataset", y=1.02)
plt.show()

```

OUTPUT:

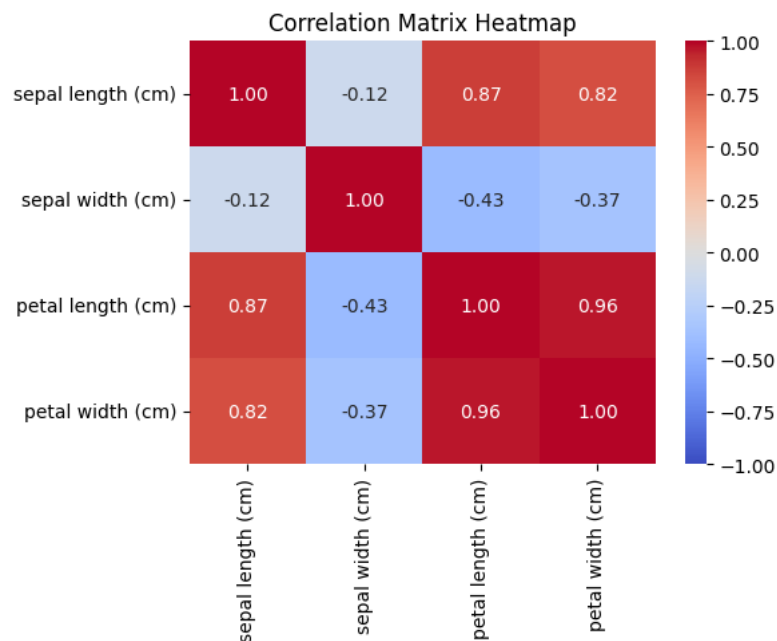
INFERENCE: The pair plot provides a comprehensive view of the relationships between all pairs of features. It helps identify feature distributions, correlations, and class separations, making it easier to determine which feature combinations are most effective for distinguishing the different Iris species.

1.6 Plot a heatmap of the correlation matrix for the Iris dataset. Which features are most strongly correlated?

CODE:

```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
corr_matrix = df.corr()

plt.figure(figsize=(6, 5))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", vmin=-1, vmax=1)
plt.title("Correlation Matrix Heatmap")
plt.tight_layout()
plt.show()
```

OUTPUT:

INFERENCE: The correlation heatmap shows that petal length and petal width have the strongest positive correlation. It also indicates that sepal length is positively correlated with both petal length and petal width, suggesting that petal-related features are the most informative for analysing and classifying the Iris dataset.

III. Three-Dimensional Visualization Using the Iris Dataset

1.7 Plot a three-dimensional scatter plot using sepal length, sepal width, and petal length. Use different colors to represent each Iris species.

CODE:

```
colors = {0: "teal", 1: "orange", 2: "purple"}
species_names = {0: "setosa", 1: "versicolor", 2: "virginica"}
df["color"] = [colors[x] for x in iris.target]
```

```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
```

```
scatter = ax.scatter(
    df["sepal length (cm)"],
    df["sepal width (cm)"],
    df["petal length (cm)"],
    c=df["color"],
    s=40,
    edgecolor="k",
)
```

```
ax.set_title("3D Scatter Plot of Iris Dataset")
```

```

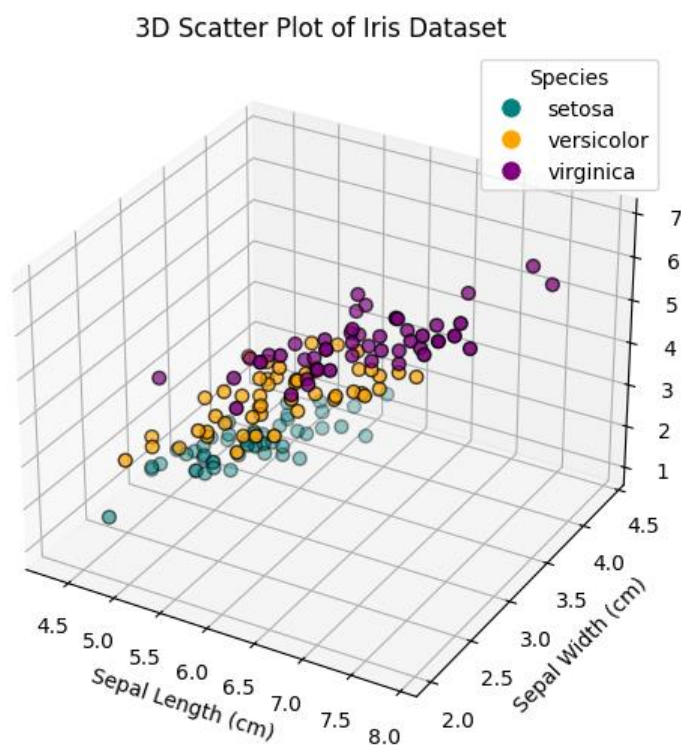
ax.set_xlabel("Sepal Length (cm)")
ax.set_ylabel("Sepal Width (cm)")
ax.set_zlabel("Petal Length (cm)")

markers = [
    plt.Line2D([0], [0], marker="o", color="w", markerfacecolor=col, markersize=10)
    for col in colors.values()
]
ax.legend(markers, species_names.values(), title="Species")

plt.show()

```

OUTPUT:



INFERENCE: The 3D scatter plot provides a clearer visualization of the relationships among the three features. Setosa forms a well-separated cluster, while Versicolor and Virginica are closer together with slight overlap. The use of three dimensions enhances the visualization of feature variations and improves the understanding of species-wise data distribution.

IV. Data Visualization Using the Titanic Dataset

2.1 Plot a count plot for the *survived* column. Analyse the distribution of passenger survival.

CODE:

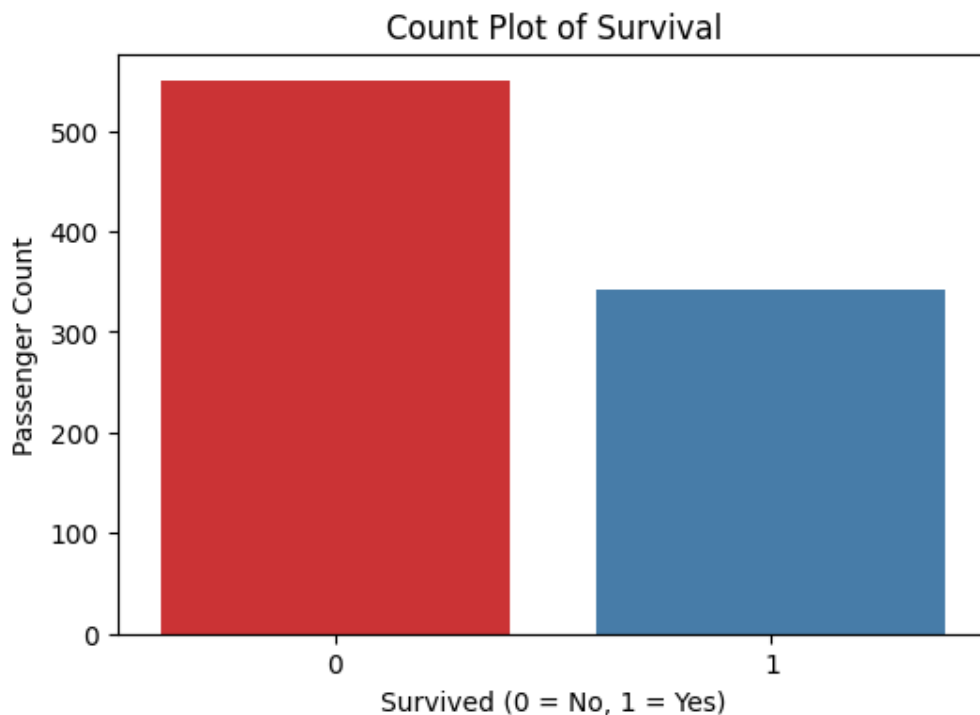
```

import matplotlib.pyplot as plt
import seaborn as sns

```

```
titanic = sns.load_dataset("titanic")

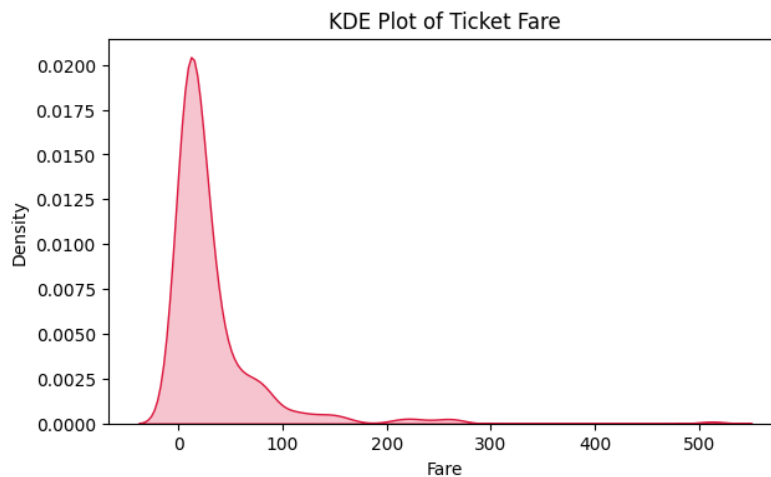
plt.figure(figsize=(6, 4))
sns.countplot(data=titanic, x="survived", hue="survived", palette="Set1", legend=False)
plt.title("Count Plot of Survival")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Passenger Count")
plt.show()
```

OUTPUT:

INFERENCE: The count plot shows that the survival distribution is imbalanced, with a greater number of passengers not surviving than surviving. Approximately 61% of the passengers perished, while only about 39% survived, indicating that fatalities were more common in the Titanic disaster.

2.2 Visualize the distribution of ticket fares using a Kernel Density Estimation (KDE) plot. Is the distribution skewed?**CODE:**

```
plt.figure(figsize=(7, 4))
sns.kdeplot(data=titanic, x="fare", fill=True, color="crimson")
plt.title("KDE Plot of Ticket Fare")
plt.xlabel("Fare")
plt.ylabel("Density")
plt.show()
```

OUTPUT:

INFERENCE: The KDE plot reveals that the fare distribution is positively (right) skewed. Most passengers paid relatively low fares, while a small number of passengers purchased very expensive tickets, resulting in a long tail toward higher fare values.

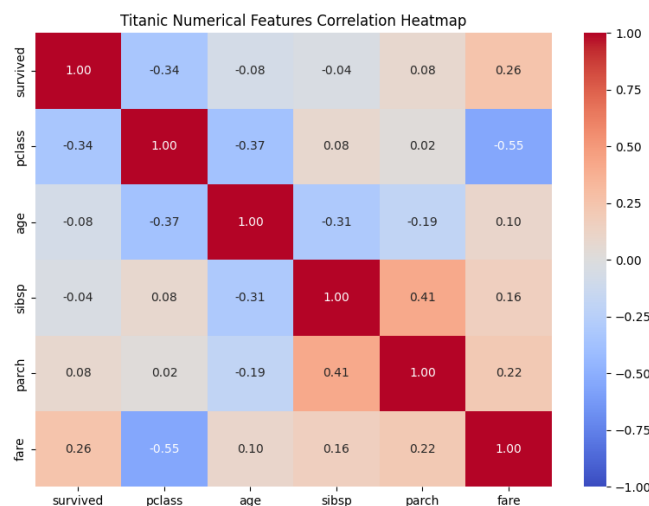
2.3 Display a correlation heatmap for the numerical features of the Titanic dataset. Which variables show the strongest correlations?

CODE:

```
numeric_df = titanic.select_dtypes(include=["float64", "int64"])

corr_matrix = numeric_df.corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", vmin=-1, vmax=1)
plt.title("Titanic Numerical Features Correlation Heatmap")
plt.tight_layout()
plt.show()
```

OUTPUT:

INFERENCE: The heatmap indicates that passenger class (Pclass) and fare have a strong negative correlation, showing that higher-class passengers generally paid higher fares. It also reveals a moderate positive correlation between SibSp and Parch, suggesting that family members often travelled together. Additionally, Pclass and Age exhibit a mild negative correlation.

2.4 Create a three-dimensional scatter plot using *fare*, *age*, and *Pclass*, with data points color-coded by the *survived* column. Interpret the observed pattern.

CODE:

```
titanic = sns.load_dataset("titanic").dropna(subset=["age", "fare", "pclass"])

colors = {0: "crimson", 1: "teal"}
passenger_colors = [colors[x] for x in titanic["survived"]]

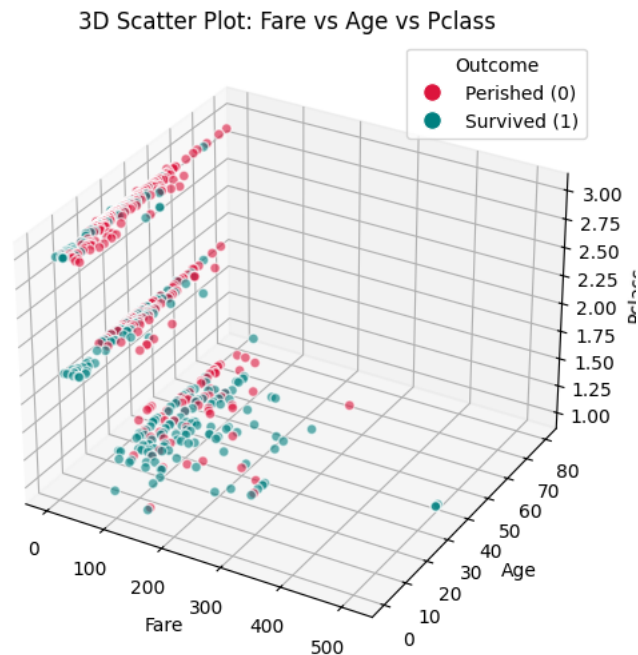
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")

ax.scatter(
    titanic["fare"],
    titanic["age"],
    titanic["pclass"],
    c=passenger_colors,
    alpha=0.6,
    edgecolor="w",
    s=30
)

ax.set_title("3D Scatter Plot: Fare vs Age vs Pclass")
ax.set_xlabel("Fare")
ax.set_ylabel("Age")
ax.set_zlabel("Pclass")

markers = [
    plt.Line2D([0], [0], marker="o", color="w", markerfacecolor=col, markersize=10)
    for col in colors.values()
]
ax.legend(markers, ["Perished (0)", "Survived (1)"], title="Outcome")

plt.show()
```

OUTPUT:

INFERENCE: The 3D scatter plot shows that passengers in first class with higher ticket fares had a greater likelihood of survival. In contrast, most third-class passengers with lower fares are clustered among the non-survivors. The visualization also indicates that younger passengers had relatively better survival rates, reflecting the historical rescue priority given to women and children.

2.5 Apply Principal Component Analysis (PCA) to reduce the dataset dimensions and visualize the transformed data using a three-dimensional scatter plot.

CODE:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

titanic = sns.load_dataset("titanic")
features = ["pclass", "age", "sibsp", "parch", "fare"]

df_pca = titanic[features + ["survived"]].dropna()

X = df_pca[features]
y = df_pca["survived"]

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=3)
X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(data=X_pca, columns=["PC1", "PC2", "PC3"])
pca_df["survived"] = y.values
```

```

colors = {0: "crimson", 1: "teal"}
dot_colors = [colors[x] for x in pca_df["survived"]]

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")

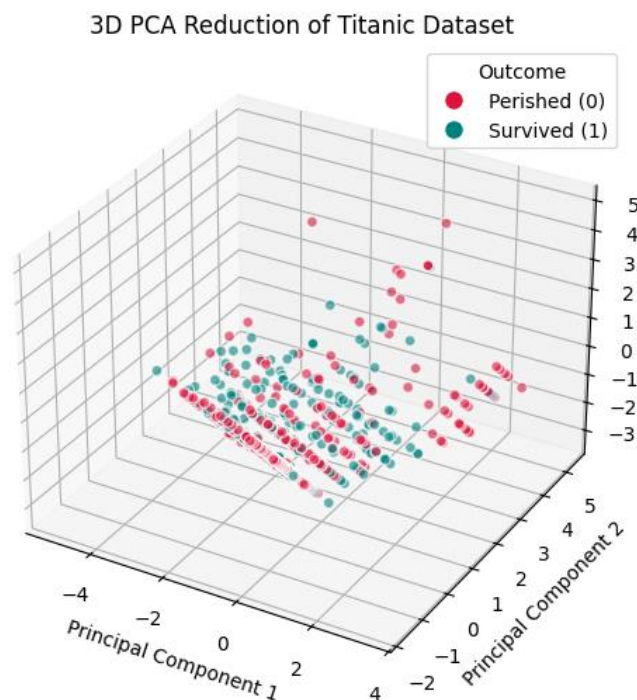
ax.scatter(
    pca_df["PC1"],
    pca_df["PC2"],
    pca_df["PC3"],
    c=dot_colors,
    alpha=0.6,
    edgecolor="w",
    s=30
)

ax.set_title("3D PCA Reduction of Titanic Dataset")
ax.set_xlabel("Principal Component 1")
ax.set_ylabel("Principal Component 2")
ax.set_zlabel("Principal Component 3")

markers = [
    plt.Line2D([0], [0], marker="o", color="w", markerfacecolor=col, markersize=10)
    for col in colors.values()
]
ax.legend(markers, ["Perished (0)", "Survived (1)"], title="Outcome")

plt.show()

```

OUTPUT:

INFERENCE: The PCA plot successfully reduces the original numerical features into three principal components while preserving most of the dataset's variance. Although the dimensionality is reduced, the survivor and non-survivor classes still overlap considerably, indicating that the data is not linearly separable and may require more advanced machine learning models for accurate classification.

V. Multidimensional Visualization Using the Wine Dataset

3.1 Create a scatter plot of *Alcohol* versus *Colour Intensity*, using different colours to represent the wine classes. Interpret the observed pattern.

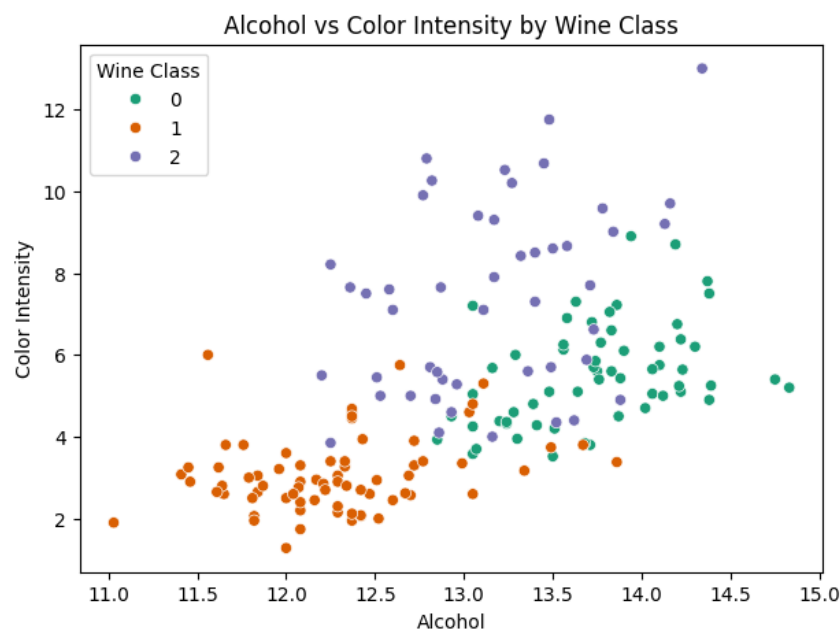
CODE:

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_wine

wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)
df["class"] = wine.target

plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df, x="alcohol", y="color_intensity", hue="class", palette="Dark2"
)
plt.title("Alcohol vs Color Intensity by Wine Class")
plt.xlabel("Alcohol")
plt.ylabel("Color Intensity")
plt.legend(title="Wine Class")
plt.show()
```

OUTPUT:



INFERENCE: The scatter plot shows that the wine classes form distinct clusters based on alcohol content and colour intensity. Class 0 generally exhibits higher alcohol content, while the remaining classes occupy different regions of the plot. This indicates that these features are effective in distinguishing between the wine classes.

3.2 Create a parallel coordinates plot for all features, with lines color-coded according to the wine class. What observations can be made from the visualization?

CODE:

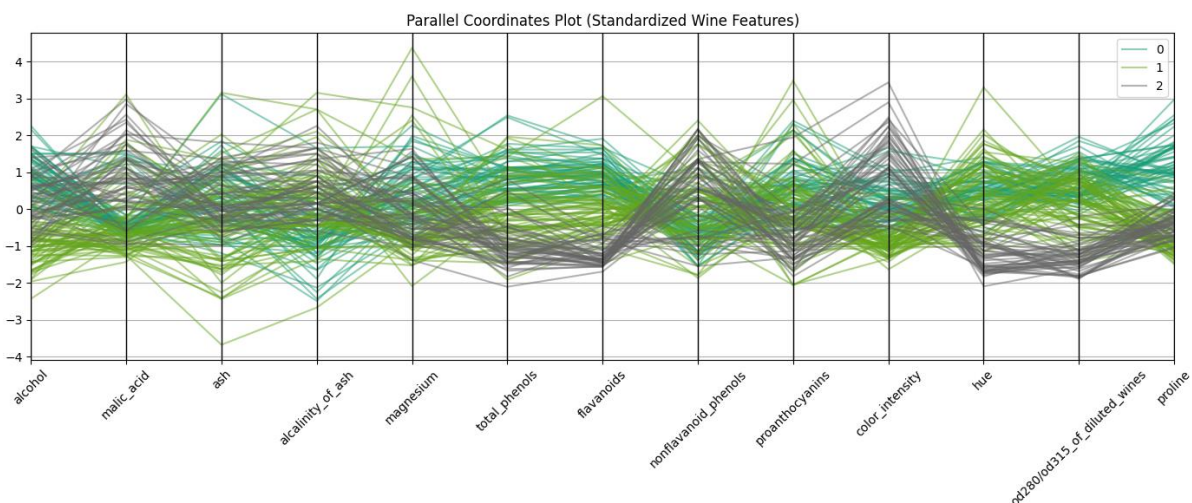
```
from pandas.plotting import parallel_coordinates
from sklearn.preprocessing import StandardScaler

wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)

scaler = StandardScaler()
scaled_features = scaler.fit_transform(df)
df_scaled = pd.DataFrame(scaled_features, columns=wine.feature_names)
df_scaled["class"] = wine.target

plt.figure(figsize=(14, 6))
parallel_coordinates(df_scaled, "class", colormap="Dark2", alpha=0.5)
plt.title("Parallel Coordinates Plot (Standardized Wine Features)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:



INFERENCE: The parallel coordinates plot reveals that samples belonging to the same wine class follow similar patterns across multiple features. Certain attributes, such as Proline, Flavanoids, and Alcohol, clearly separate the classes, while features like Ash and Magnesium show greater overlap. This visualization highlights the multidimensional relationships among

the features and demonstrates that several chemical properties effectively distinguish the wine classes.

3.3 Plot a radar chart showing the average feature values for each wine class. What observations can be made from the chart?

CODE:

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)
df["class"] = wine.target

scaler = MinMaxScaler()
df_scaled = pd.DataFrame(scaler.fit_transform(df[wine.feature_names]),
                          columns=wine.feature_names)
df_scaled["class"] = df["class"]

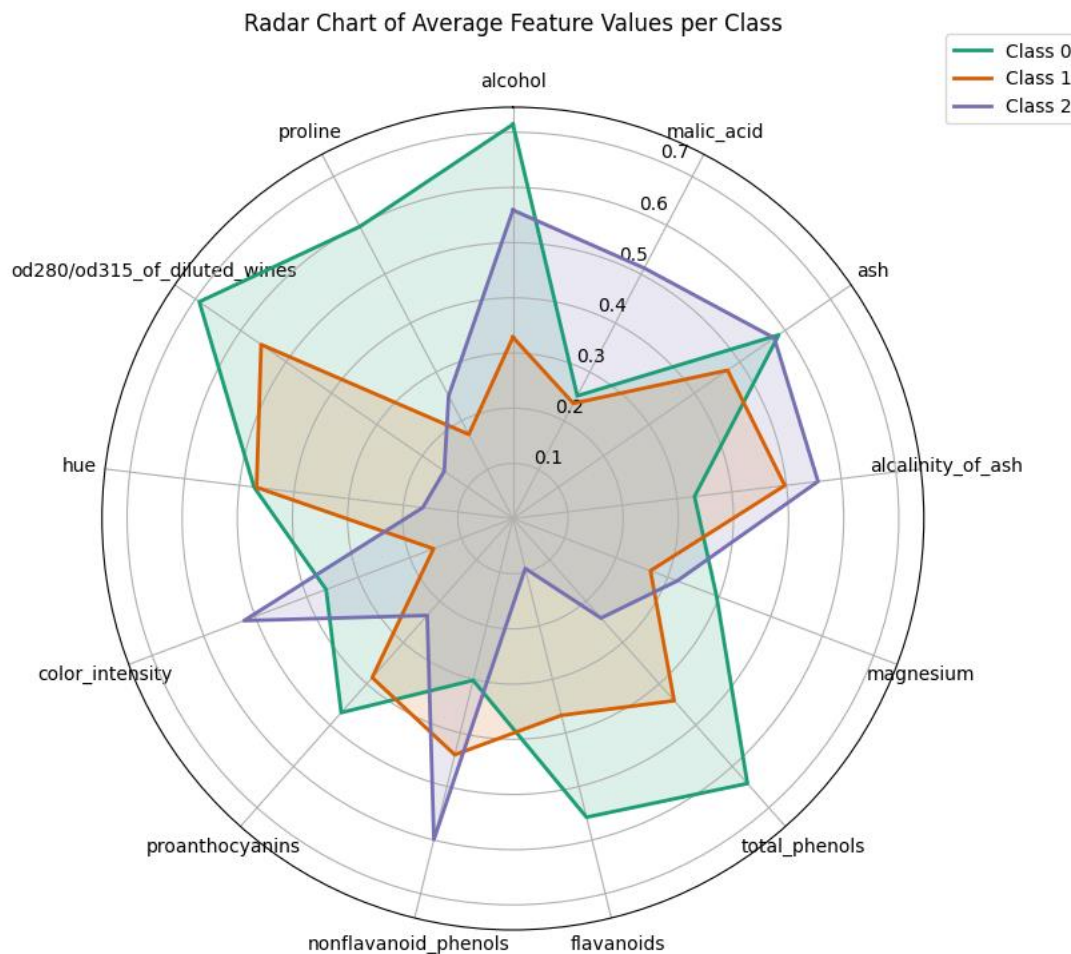
class_means = df_scaled.groupby("class").mean()
features = list(class_means.columns)
num_vars = len(features)

angles = np.linspace(0, 2 * np.pi, num_vars, endpoint=False).tolist()
angles += angles[:1]

fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(polar=True))
colors = ["#1b9e77", "#d95f02", "#7570b3"]

for i, class_label in enumerate(class_means.index):
    values = class_means.loc[class_label].tolist()
    values += values[:1]
    ax.plot(angles, values, color=colors[i], linewidth=2, label=f"Class {class_label}")
    ax.fill(angles, values, color=colors[i], alpha=0.15)

ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
ax.set_thetagrids(np.degrees(angles[:-1]), features)
plt.title("Radar Chart of Average Feature Values per Class", y=1.08)
plt.legend(loc="upper right", bbox_to_anchor=(1.2, 1.1))
plt.show()
```

OUTPUT:

INFERENCE: The radar chart illustrates that each wine class has a unique chemical profile. Class 0 is characterized by higher values of Alcohol and Proline, while Class 2 exhibits higher Malic Acid and Alkalinity of Ash. The distinct shapes of the radar plots indicate that the wine classes possess different characteristics, making them suitable for accurate classification.

VI. Map-Based Visualization Using the COVID-19 Dataset

4.1 Using the dataset containing COVID-19 cases and ISO country codes, create a choropleth map to visualize the number of confirmed COVID-19 cases across the world.

CODE:

```

pip install plotly
pip install pycountry
import pandas as pd
import plotly.express as px
import plotly.io as pio
import pycountry

pio.renderers.default = "iframe"

```

```

def get_iso3(country_name):
    try:
        return pycountry.countries.lookup(country_name).alpha_3
    except LookupError:
        mapping = {
            "US": "USA",
            "USA": "USA",
            "United States": "USA",
            "UK": "GBR",
            "United Kingdom": "GBR",
            "South Korea": "KOR",
            "Korea, South": "KOR",
            "Taiwan*": "TWN",
            "Congo (Kinshasa)": "COD",
            "Congo (Brazzaville)": "COG",
        }
        return mapping.get(country_name, None)

df = pd.read_csv("covid dataset.csv")
df["iso_alpha"] = df["Country/Region"].apply(get_iso3)

fig = px.choropleth(
    data_frame=df,
    locations="iso_alpha",
    locationmode="ISO-3",
    color="Confirmed",
    hover_name="Country/Region",
    hover_data=["Deaths", "Recovered", "Active"],
    color_continuous_scale=px.colors.sequential.Plasma,
    title="Global Distribution of Confirmed COVID-19 Cases",
)

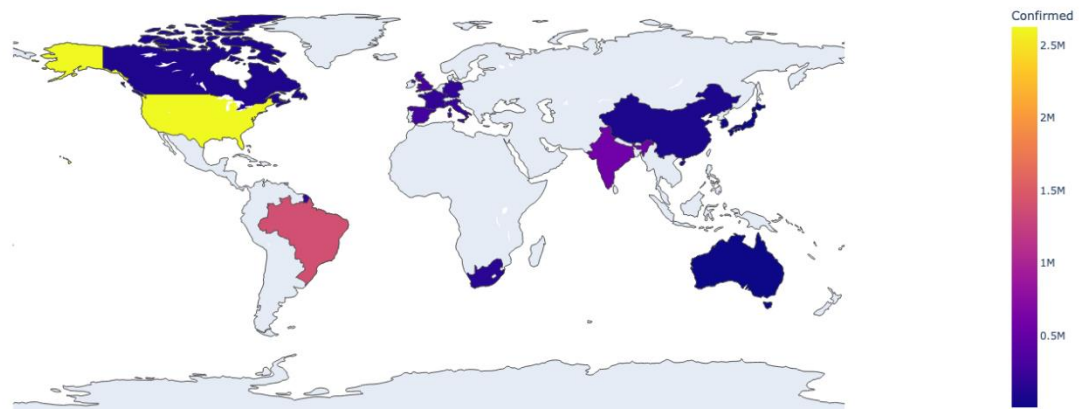
fig.update_layout(
    geo=dict(showframe=False, showcoastlines=True, projection_type="equiangular"),
    margin={"r": 0, "t": 40, "l": 0, "b": 0},
)

fig.show()

```

OUTPUT:

Global Distribution of Confirmed COVID-19 Cases



INFERENCE: The choropleth map clearly illustrates the global distribution of confirmed COVID-19 cases. Countries with a higher number of confirmed cases appear in darker shades, making it easy to identify the regions most affected by the pandemic. The visualization provides an effective overview of the geographical spread and severity of COVID-19 worldwide.

4.2 Build an interactive map using Plotly to visualize the confirmed, recovered, and death counts for each country. Which regions show higher mortality rates?

CODE:

```
import pandas as pd
import plotly.express as px
import plotly.io as pio
import pycountry

pio.renderers.default = "iframe"

def get_iso3(country_name):
    try:
        return pycountry.countries.lookup(country_name).alpha_3
    except LookupError:
        mapping = {
            "US": "USA",
            "USA": "USA",
            "United States": "USA",
            "UK": "GBR",
            "United Kingdom": "GBR",
            "South Korea": "KOR",
            "Korea, South": "KOR",
            "Taiwan*": "TWN",
            "Congo (Kinshasa)": "COD",
            "Congo (Brazzaville)": "COG",
        }
        return mapping.get(country_name, None)
```

```

df = pd.read_csv("covid dataset.csv")
df["iso_alpha"] = df["Country/Region"].apply(get_iso3)

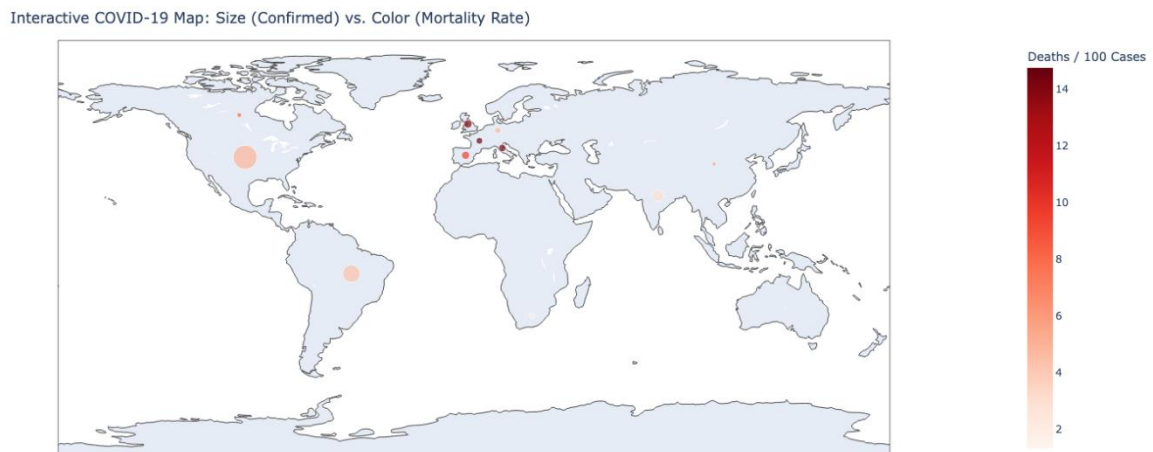
fig = px.scatter_geo(
    data_frame=df,
    locations="iso_alpha",
    locationmode="ISO-3",
    size="Confirmed",
    color="Deaths / 100 Cases",
    hover_name="Country/Region",
    hover_data=[
        "Confirmed",
        "Deaths",
        "Recovered",
        "Deaths / 100 Cases",
    ],
    color_continuous_scale=px.colors.sequential.Reds,
    title="Interactive COVID-19 Map: Size (Confirmed) vs. Color (Mortality Rate)",
)

fig.update_layout(margin={"r": 0, "t": 40, "l": 0, "b": 0})

fig.show()

```

OUTPUT:



INFERENCE: The interactive map enables easy comparison of COVID-19 statistics across countries by displaying confirmed cases, recoveries, and mortality rates. Countries with higher mortality rates are represented using darker shades, while larger markers indicate a greater number of confirmed cases. The visualization helps identify regions that experienced relatively severe outcomes and supports comparative analysis of the pandemic's global impact.

RESULT: Hence, data was visualised in different dimensions and meaningful patterns and insights were extracted successfully.

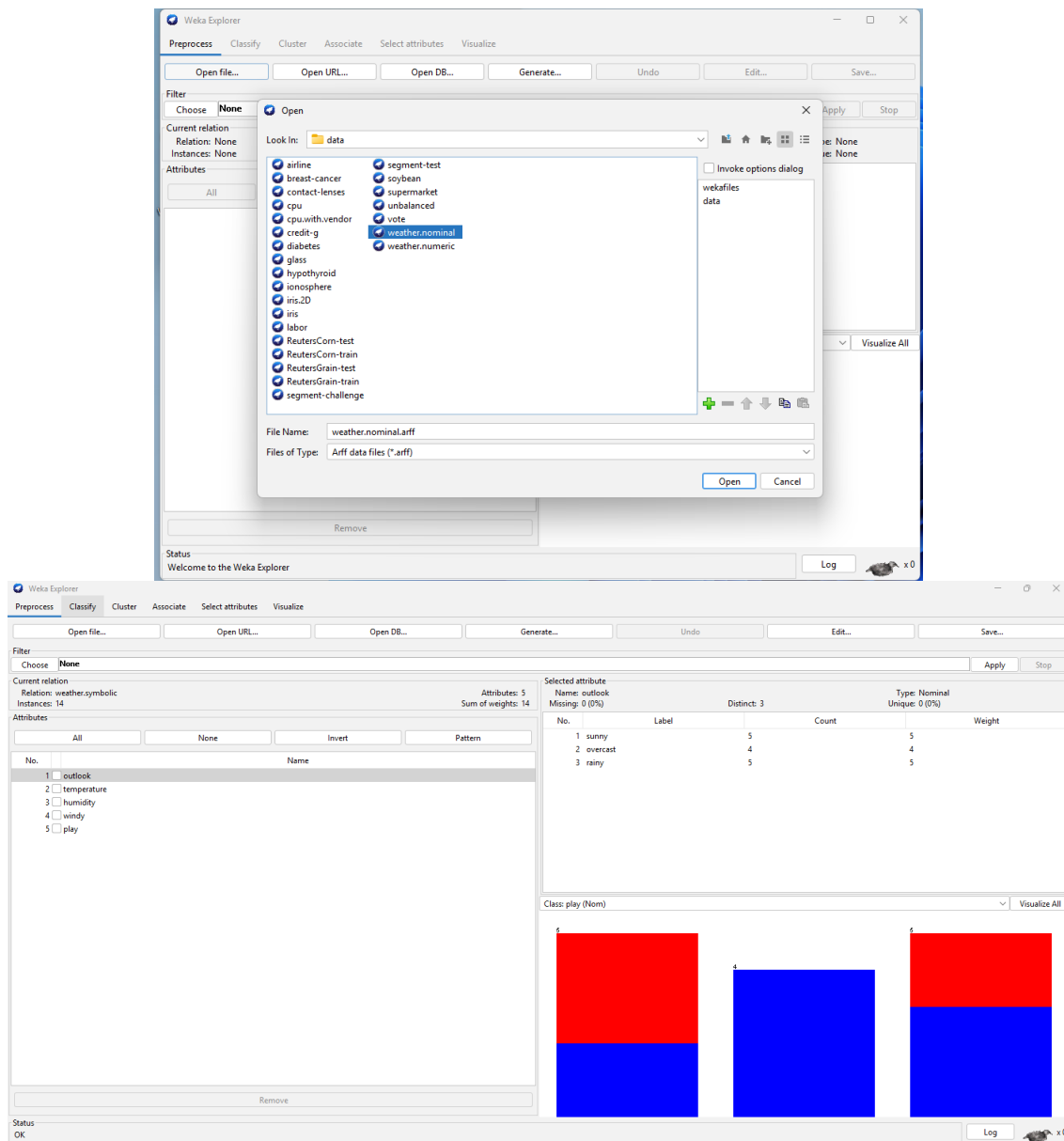
EXP NO: 5**DATE:****IMPLEMENTATION OF DATA MINING PROCESS
USING WEKA TOOL**

AIM: To implement and study various data mining processes using the WEKA tool, including dataset exploration, association rule mining using Apriori and FP-Growth, implementation of Apriori using WEKA KnowledgeFlow, and classification using the J48 Decision Tree algorithm.

QUESTIONS:**Q1. To familiarize with the WEKA interface and load a dataset****STEPS:**

1. Open WEKA GUI Chooser.
2. Click Explorer.
3. Select the Preprocess tab.
4. Click Open file.
5. Select the dataset: weather.nominal.arff
6. Click Open.
7. Observe the dataset information displayed in the Preprocess window.

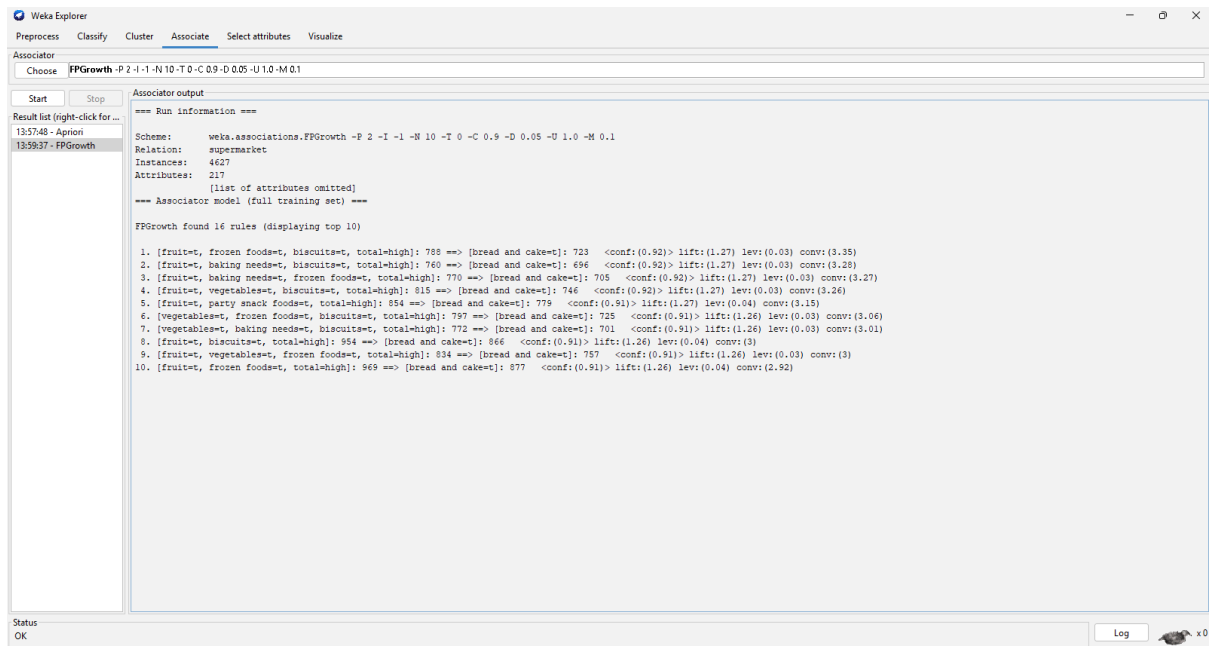
OUTPUT:



Q2. Association Rule Mining using Apriori

STEPS:

1. Open WEKA Explorer.
2. Go to Preprocess → Open file.
3. Load: weather.nominal.arff
4. Select the Associate tab.
5. Click Choose.
6. Select: weka → associations → Apriori
7. Keep the required/default parameters.
8. Click Start.
9. Observe the generated association rules.

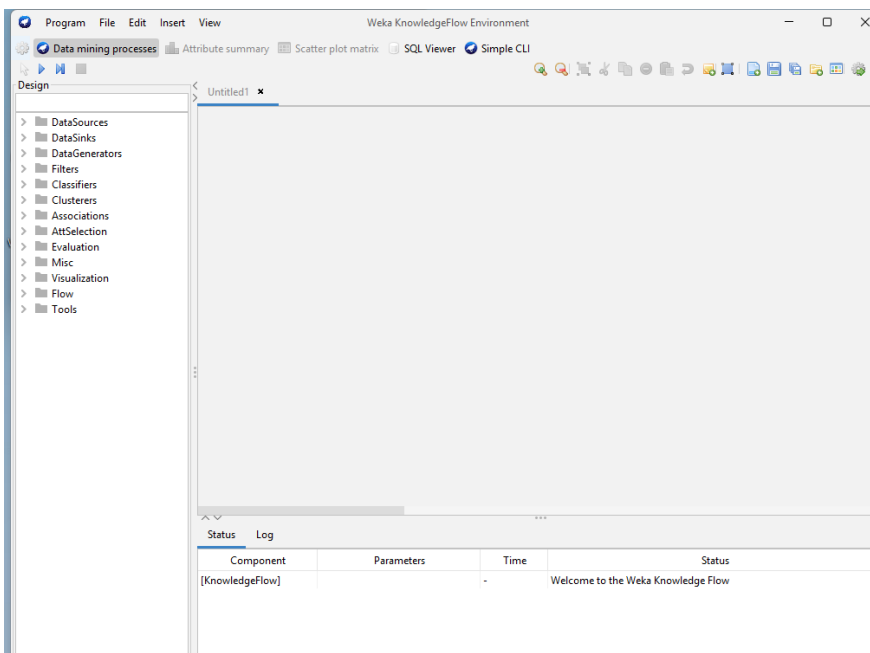


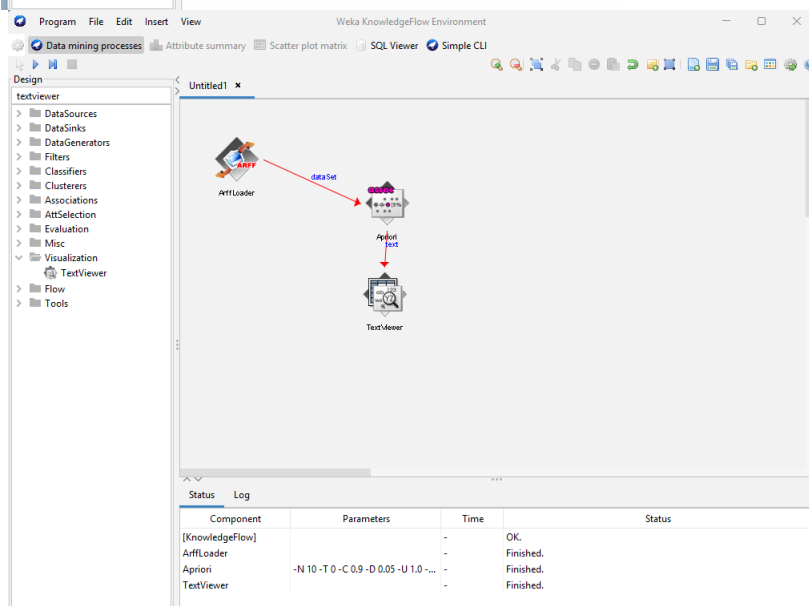
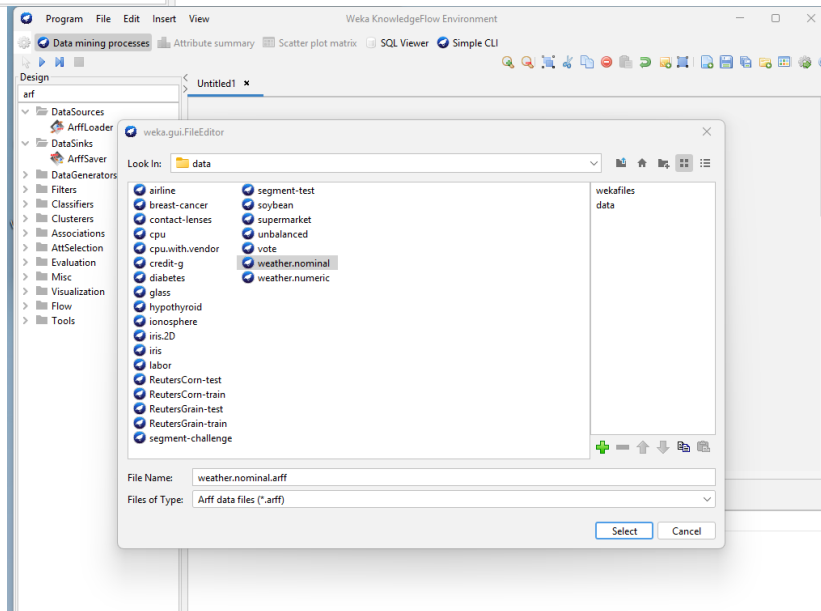
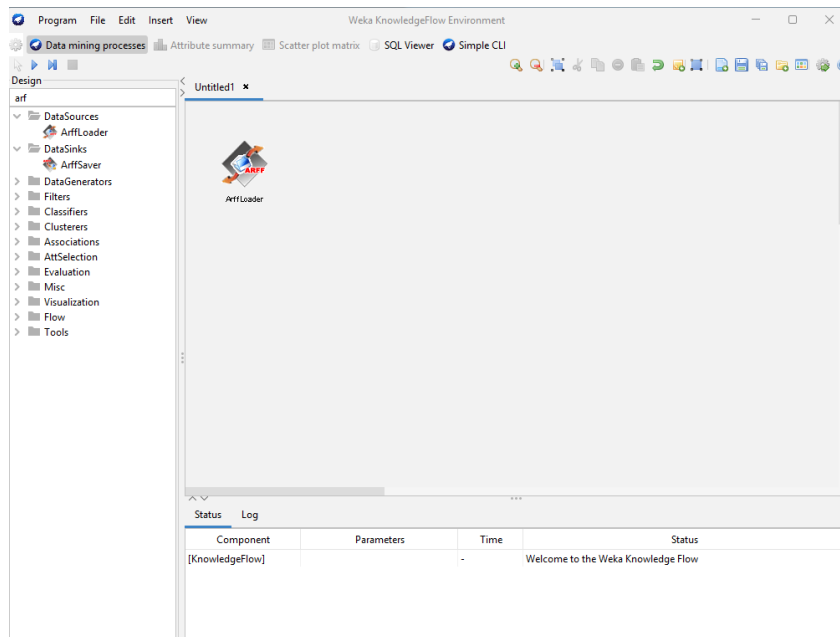
Q4. Implement Apriori Algorithm using WEKA KnowledgeFlow Layout

STEPS:

1. Open WEKA GUI Chooser.
2. Select KnowledgeFlow.
3. Add an ArffLoader component from Data Sources.
4. Configure ArffLoader and select: weather.nominal.arff
5. Add Apriori from the Associations section.
6. Add TextViewer from Visualization.
7. Connect the components as: ArffLoader -> Apriori -> TextViewer
8. Execute the KnowledgeFlow.
9. Open TextViewer to view the generated association rules.

OUTPUT:





```

Test
=== Associator model ===

Schema: Aggrider
Relation: weather.symbolic

Apriori
=====

Minimum support: 0.15 (2 instances)
Minimum metric (confidence): 0.9
Number of cycles performed: 17

Generated sets of large itemsets:

Size of set of large itemsets L(1): 12
Size of set of large itemsets L(2): 47
Size of set of large itemsets L(3): 39
Size of set of large itemsets L(4): 6
...

Best rules found:
1. outlook=overcast 4 ==> play=yes 4 <conf:(1)> lift:(1.56) lev:(0.1) [1] conv:(1.43)
2. temperature=cool 4 ==> humidity=normal 4 <conf:(1)> lift:(2) lev:(0.14) [2] conv:(2)
3. humidity=normal windy=FALSE 4 ==> play=yes 4 <conf:(1)> lift:(1.56) lev:(0.1) [1] conv:(1.43)
4. outlook=rainy play=no 3 ==> humidity=high 3 <conf:(1)> lift:(2) lev:(0.11) [1] conv:(1.5)
5. outlook=rainy humidity=high 3 ==> play=no 3 <conf:(1)> lift:(2.5) lev:(0.14) [1] conv:(1.93)
6. outlook=rainy play=yes 3 ==> windy=FALSE 3 <conf:(1)> lift:(1.78) lev:(0.09) [1] conv:(1.29)
7. outlook=rainy windy=FALSE 3 ==> play=yes 3 <conf:(1)> lift:(1.84) lev:(0.08) [1] conv:(1.07)
8. temperature=cool play=yes 3 ==> humidity=normal 3 <conf:(1)> lift:(2) lev:(0.11) [1] conv:(1.5)
9. outlook=rainy temperature=hot 2 ==> humidity=high 2 <conf:(1)> lift:(2) lev:(0.07) [1] conv:(1)
10. temperature=hot play=no 2 ==> outlook=rainy 2 <conf:(1)> lift:(2.8) lev:(0.08) [1] conv:(1.29)

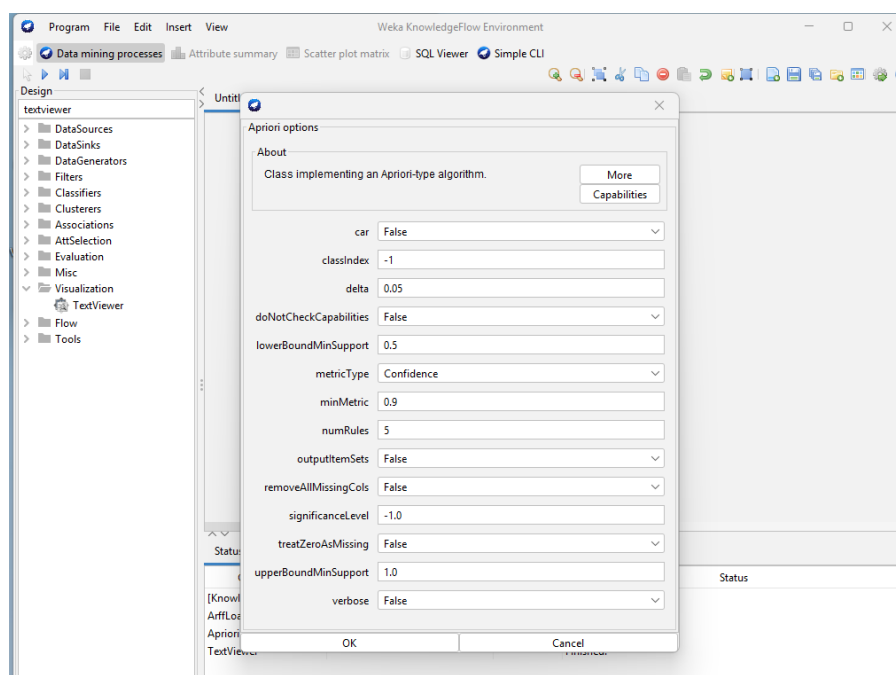
```

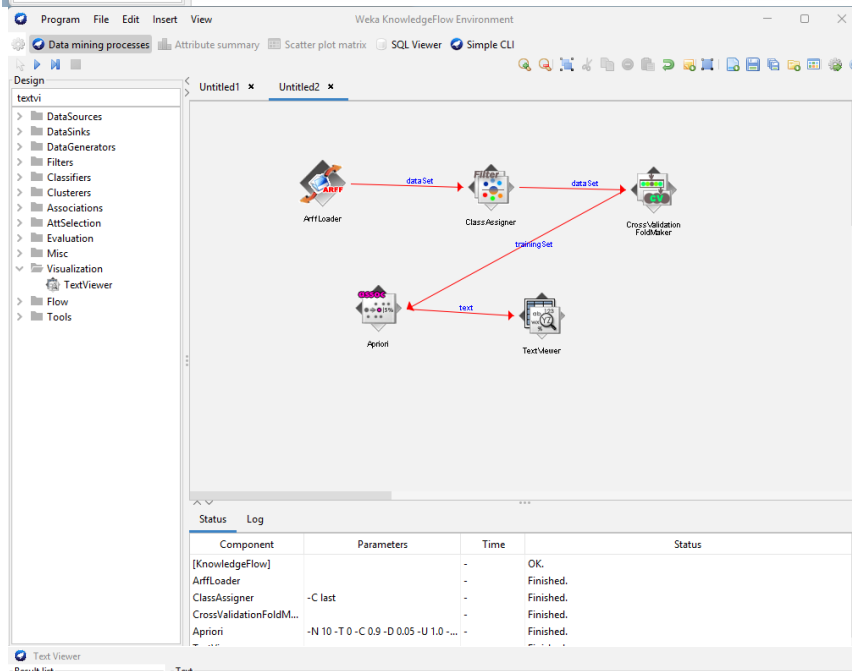
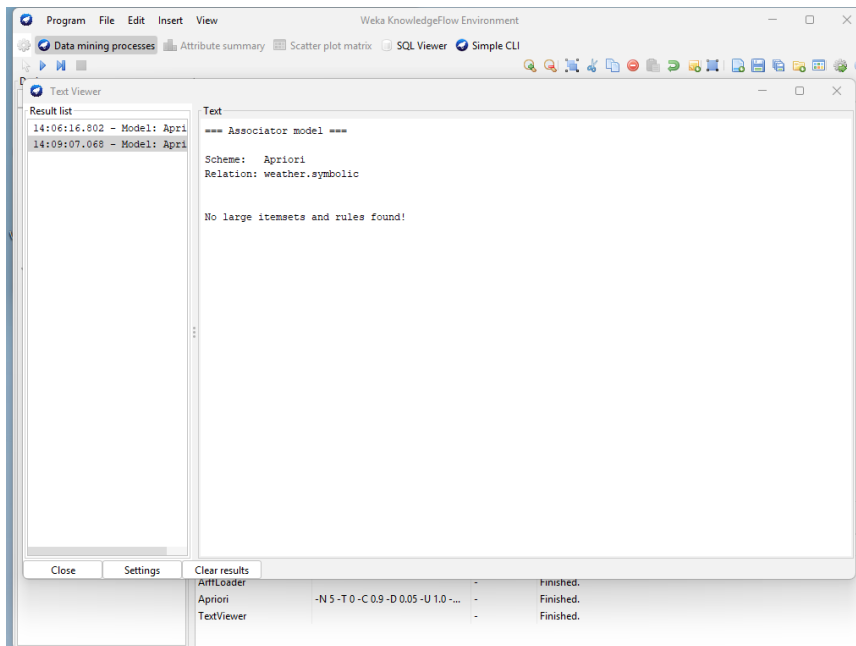
Q5. Implement Apriori with ClassAssigner and CrossValidationFoldMaker using WEKA KnowledgeFlow

STEPS:

1. Open WEKA KnowledgeFlow.
2. Add ArffLoader and load: weather.nominal.arff
3. Add a ClassAssigner component.
4. Configure the class attribute appropriately.
5. Add CrossValidationFoldMaker.
6. Add Apriori.
7. Add TextViewer.
8. Connect the components as: ArffLoader -> ClassAssigner -> CrossValidationFoldMaker -> Apriori -> TextViewer
9. Execute the KnowledgeFlow.
10. View the generated association rules using TextViewer.

OUTPUT:





Text Viewer

Result list

14:14:54.902 - Model: Apriori
14:14:54.902 - Model: Apriori
14:14:54.902 - Model: Apriori
14:14:54.904 - Model: Apriori
14:14:54.904 - Model: Apriori
14:14:54.904 - Model: Apriori
14:14:54.904 - Model: Apriori
14:14:54.907 - Model: Apriori
14:14:54.908 - Model: Apriori
14:14:54.908 - Model: Apriori
14:14:54.908 - Model: Apriori

Text

=== Associator model ===

Scheme: Apriori
Relation: weather.symbolic-weka.filters.unsupervised.attribute.ClassAssigner~Clast

Apriori
=====

Minimum support: 0.15 (2 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 17

Generated sets of large itemsets:

Size of set of large itemsets L(1): 12
Size of set of large itemsets L(2): 44
Size of set of large itemsets L(3): 35
Size of set of large itemsets L(4): 6

Best rules found:

1. humidity=normal windy=FALSE 4 ==> play=yes 4 <conf:(1)> lift:(1.63) lev:(0.12) [1] conv:(1.54)
2. outlook=overcast 3 ==> play=yes 3 <conf:(1)> lift:(1.69) lev:(0.09) [1] conv:(1.15)
3. temperature=cool 3 ==> humidity=normal 3 <conf:(1)> lift:(2.17) lev:(0.12) [1] conv:(1.62)
4. outlook=sunny play=no 3 ==> humidity=high 3 <conf:(1)> lift:(1.86) lev:(0.11) [1] conv:(1.38)
5. outlook=sunny humidity=high 3 ==> play=no 3 <conf:(1)> lift:(2.6) lev:(0.14) [1] conv:(1.85)
6. outlook=rainy play=yes 3 ==> windy=FALSE 3 <conf:(1)> lift:(1.63) lev:(0.09) [1] conv:(1.15)
7. outlook=rainy windy=FALSE 3 ==> play=yes 3 <conf:(1)> lift:(1.63) lev:(0.09) [1] conv:(1.15)
8. outlook=sunny temperature=hot 2 ==> humidity=high 2 <conf:(1)> lift:(1.86) lev:(0.07) [0] conv:(0.92)
9. temperature=hot play=no 2 ==> outlook=sunny 2 <conf:(1)> lift:(2.6) lev:(0.09) [1] conv:(1.23)
10. outlook=sunny temperature=hot 2 ==> play=no 2 <conf:(1)> lift:(2.6) lev:(0.09) [1] conv:(1.23)

Q6. Implement J48 Algorithm using WEKA KnowledgeFlow Layout

STEPS:

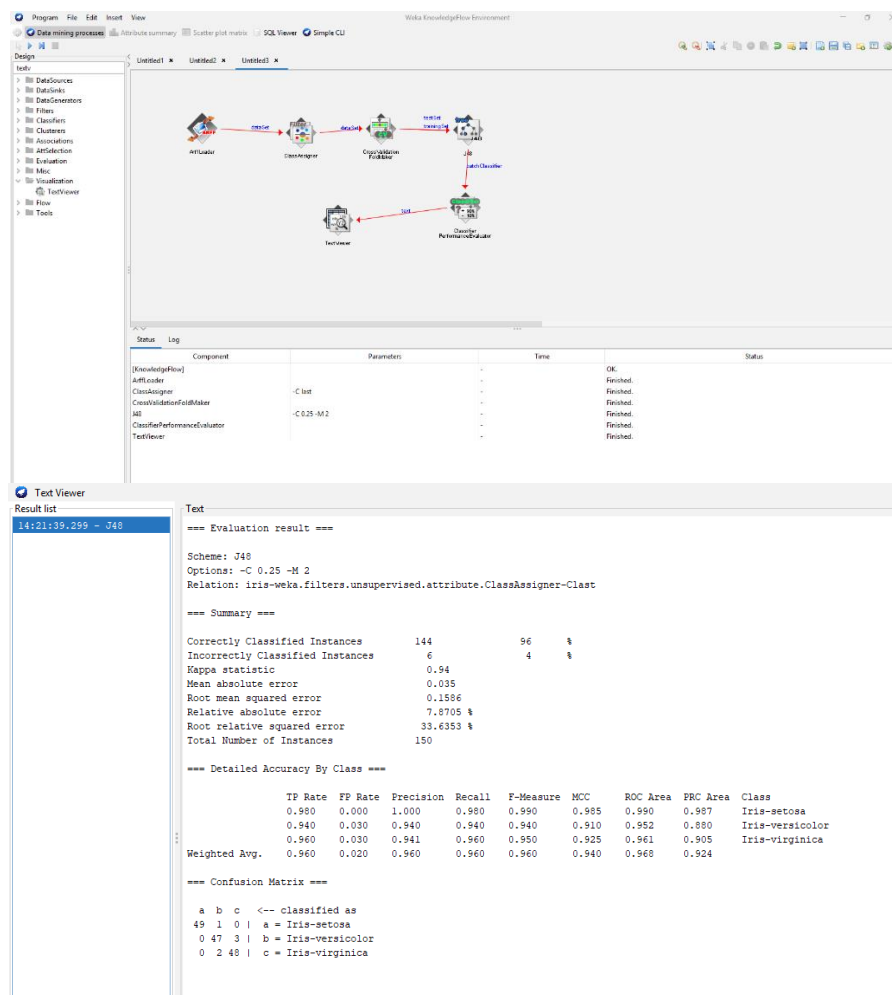
1. Open WEKA KnowledgeFlow.
2. Add ArffLoader.
3. Load the: iris.arff dataset.
4. Add ClassAssigner and assign the last attribute as the class.
5. Add CrossValidationFoldMaker.
6. Add the J48 classifier.
7. Add ClassifierPerformanceEvaluator.
8. Add TextViewer to display classification results.
9. Add GraphViewer to J48 to visualize the generated decision tree.
10. Add AttributeSummarizer and DataVisualizer to ArffLoader.
11. Add ModelPerformanceChart to ClassifierPerformanceEvaluator.
12. Connect the required components and execute the KnowledgeFlow.

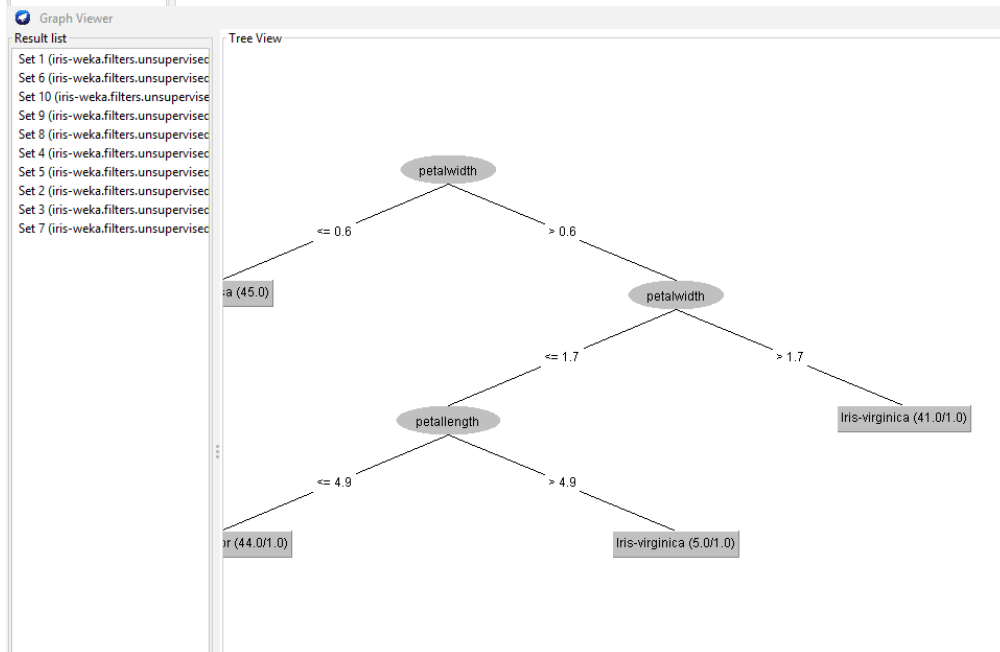
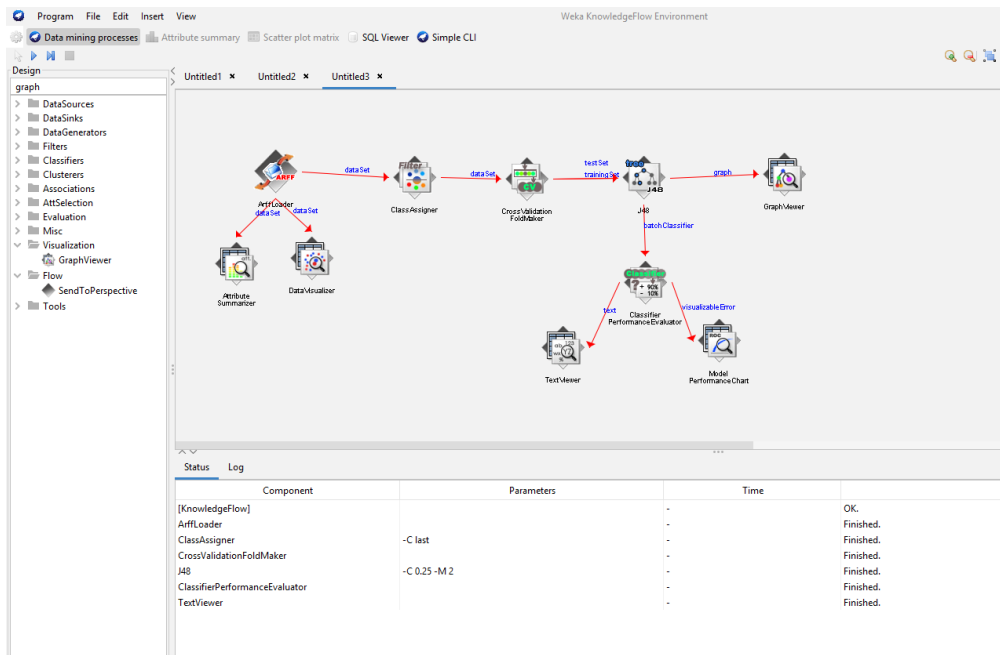
The main classification flow is:

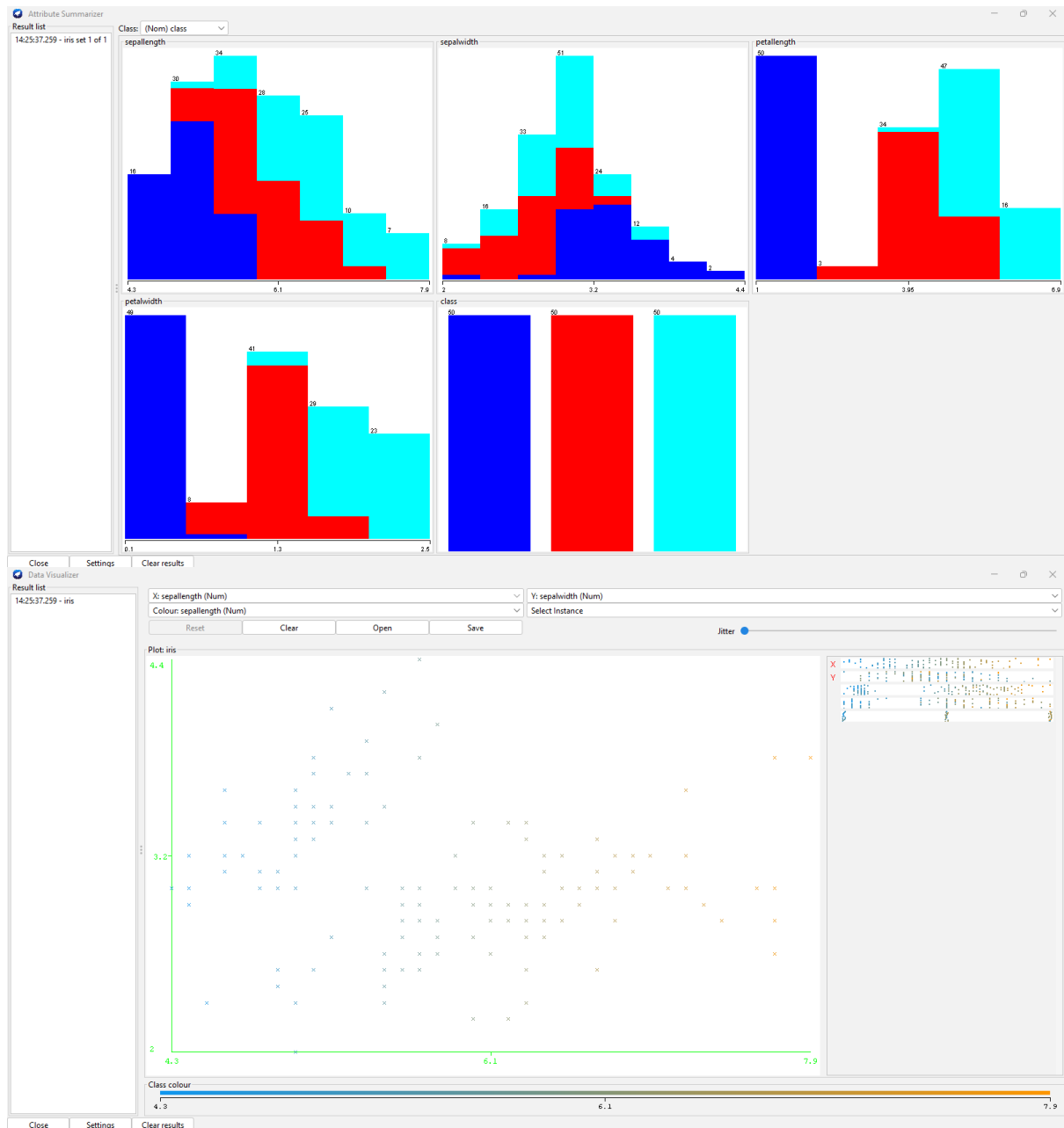
ArffLoader -> ClassAssigner -> CrossValidationFoldMaker -> J48 -> ClassifierPerformanceEvaluator -> TextViewer

Additional visualization connections are made to AttributeSummarizer, DataVisualizer, GraphViewer, and ModelPerformanceChart.

OUTPUT:







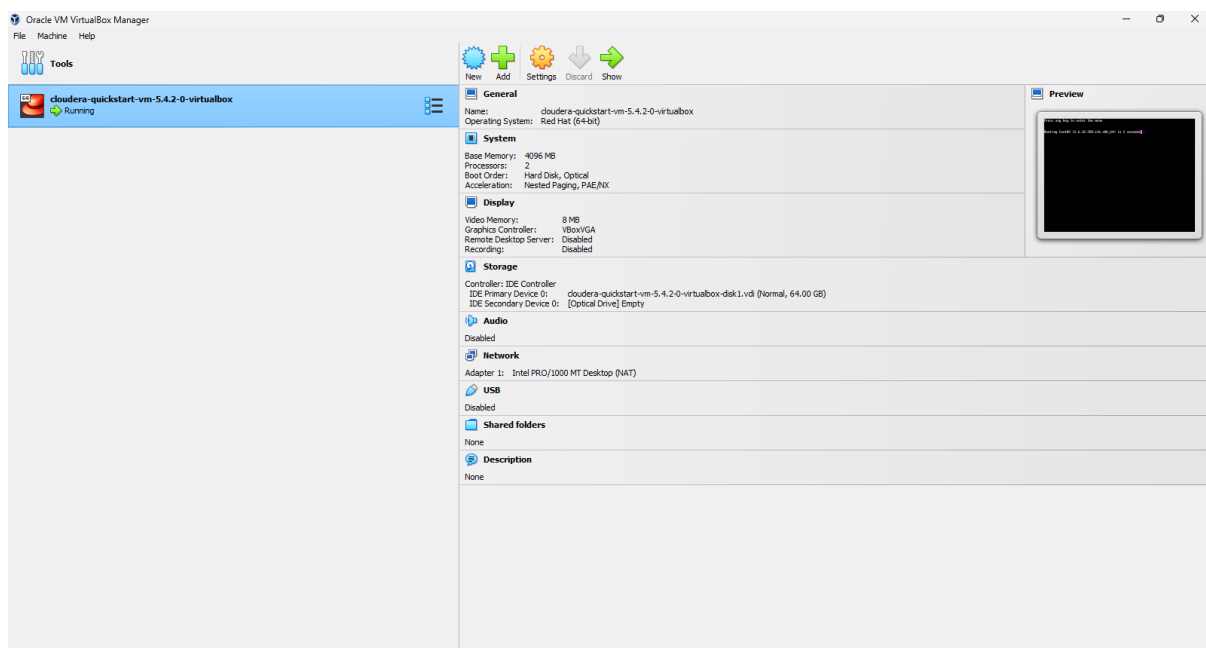
RESULT: Thus, various data mining processes were successfully implemented using the WEKA tool. Association rules were generated using Apriori and FP-Growth algorithms, Apriori was implemented using WEKA KnowledgeFlow, and the J48 Decision Tree classifier was applied to the Iris dataset. The J48 classifier achieved an accuracy of 96%, demonstrating successful classification of the Iris instances.

EXP NO: 6**DATE:****EXPLORING AND MANAGING HADOOP
USING CLOUDERA CLI AND WEBUI**

AIM: To explore the Cloudera Manager Web UI and to view, monitor, and analyse the status of Data Nodes in the Hadoop Distributed File System (HDFS).

QUESTIONS:**Q1. Identify the Hadoop Services that are Currently Running****PROCEDURE:**

1. Start the Cloudera QuickStart VM using Oracle VM VirtualBox.
2. Wait until the Cloudera environment completely boots.
3. Open the browser inside the VM.
4. Open Cloudera Manager and log in.
5. From the Cloudera Manager Home page, select the QuickStart Cluster.
6. Observe the list of Hadoop services available in the cluster.
7. Verify the status of services such as:
 - HDFS
 - YARN (MapReduce2)
 - Hue
 - Hive
 - HBase
 - Spark
 - Impala
 - Oozie
 - Zookeeper
8. Ensure that the required services are running.

OUTPUT:

The image displays three sequential screenshots of a Cloudera QuickStart VM environment, showing the progression from initial setup to login.

Top Screenshot: Cloudera Live Welcome Page

The browser window shows the Cloudera Live Welcome page. The navigation bar includes links for Cloudera, Hue, Hadoop, HBase, Impala, Spark, Solr, Oozie, Cloudera Manager, and Getting Started. The main content area displays "Welcome to Your Cloudera QuickStart VM!" and a table of cluster nodes:

Node	Address
Manager Node	10.0.2.15
Worker Node 1	10.0.2.15

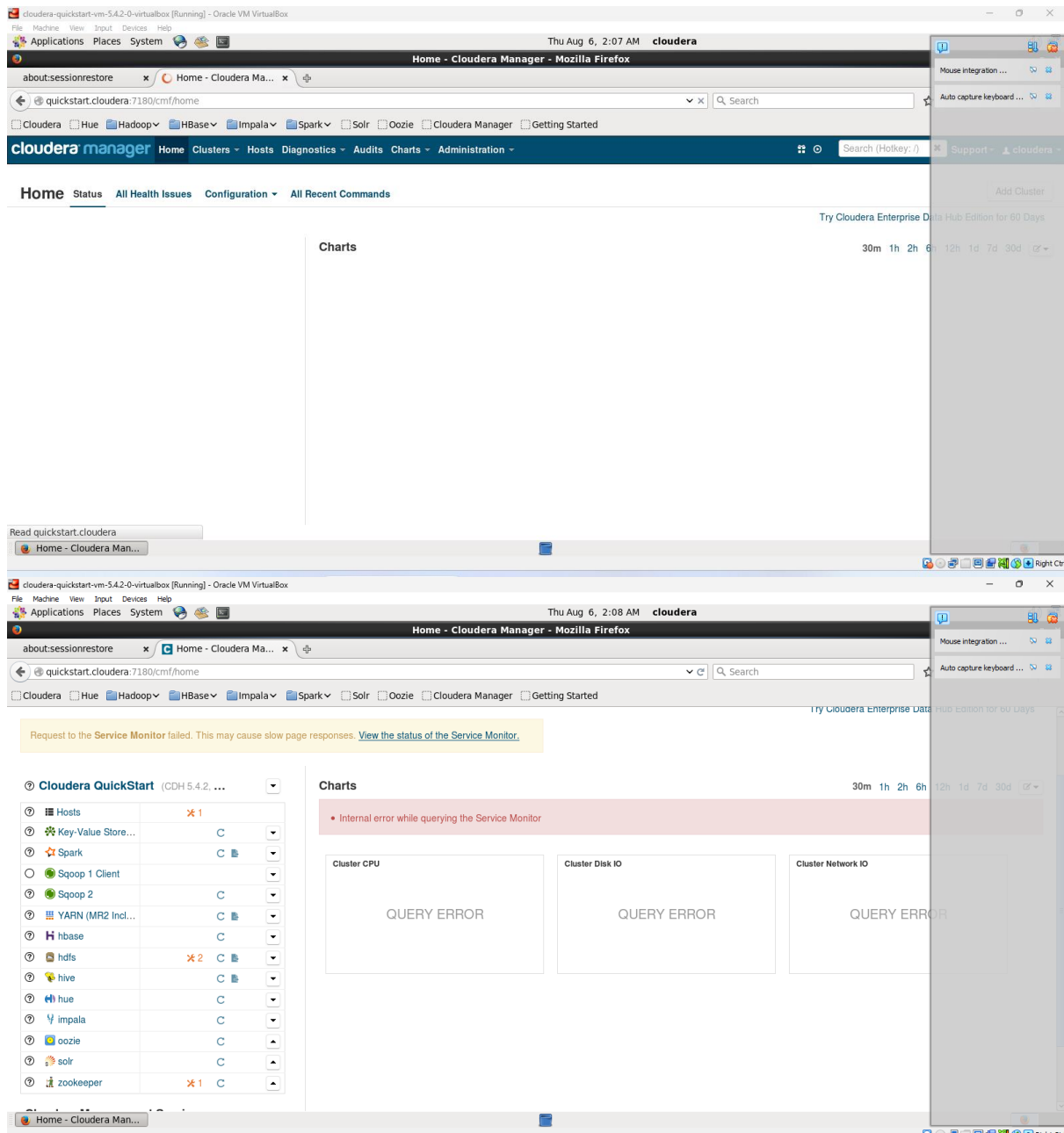
Below the table, there are sections for "Get Started" (with a "Start Tutorial" button) and "Analyze Your Data" (describing Hue as the open source web interface for Hadoop).

Middle Screenshot: Connecting to Cloudera Manager

The browser window shows the "Connecting to Cloudera Manager" page. The address bar displays the file path: `file:///home/cloudera/Documents/cloudera-manager.html`. The main content area displays the message "Attempting to connect to Cloudera Manager..." and provides instructions on how to migrate from Cloudera's Distribution including Apache Hadoop (CDH) to Cloudera Manager.

Bottom Screenshot: Cloudera Manager Login Page

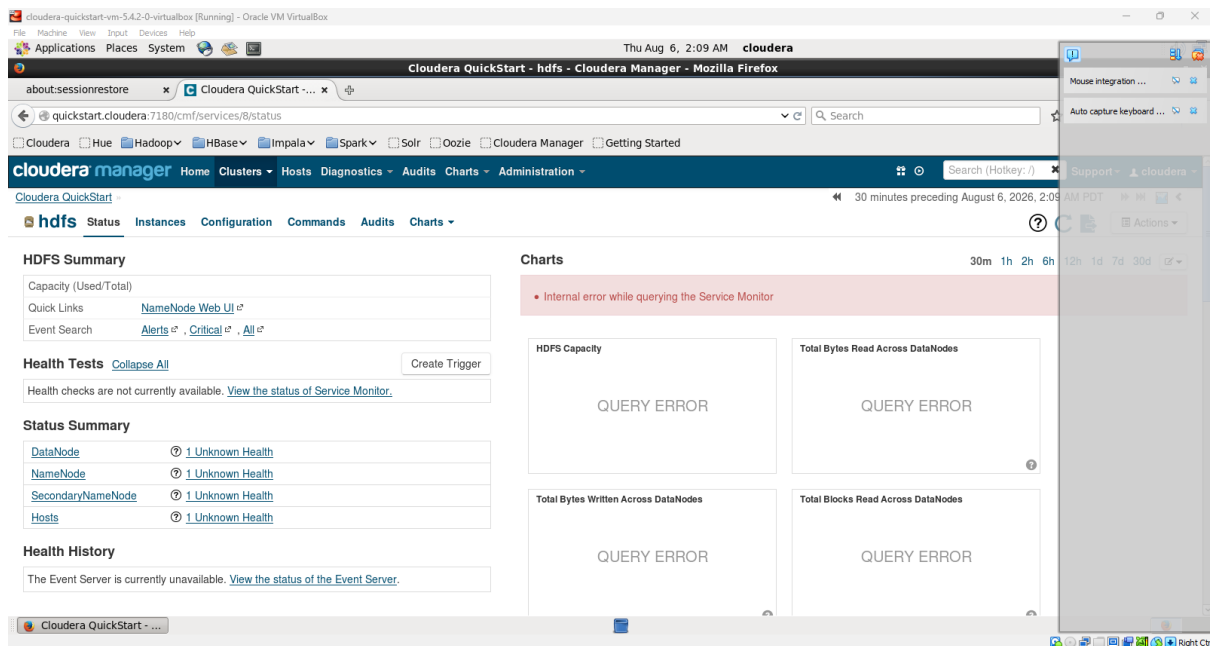
The browser window shows the Cloudera Manager Login page. The address bar displays the URL: `quickstart.cloudera:7180/cm/f/login`. The main content area displays the "Login" form with fields for Username (cloudera) and Password (*****), a "Remember me on this computer" checkbox, and a "Login" button.



Q2. Monitor HDFS Service

PROCEDURE:

1. From the Cloudera Manager cluster dashboard, click HDFS.
2. Open the Instances tab.
3. Identify the Name Node and Data Node instances.
4. Open the Charts section.
5. Observe HDFS-related information such as:
 - HDFS capacity
 - I/O activity
 - File operations
 - Data Node activity
6. Check the health and operational status of the HDFS service.

OUTPUT:**Q3. View Name Node and Data Node Details****PROCEDURE:**

1. Open the HDFS service in Cloudera Manager.
2. Go to the Instances tab.
3. Identify the Name Node and Data Node.
4. Open the Name Node Web UI.
5. Observe the Name Node information, including:
 - Hadoop version
 - Cluster ID
 - Block Pool ID
 - File-system summary
6. Open the Data Node Web UI.
7. Observe the Data Node information and operational status.
8. In Cloudera Manager, select a Data Node role to inspect:
 - Health status
 - CPU/Memory usage
 - Disk information
 - I/O activity
 - Logs.

OUTPUT:

The first screenshot shows the Cloudera QuickStart VM interface with the 'Namenode information' tab selected. The interface displays the following details for the active cluster 'quickstart.cloudera:8020':

Started:	Thu Aug 06 02:02:04 PDT 2026
Version:	2.6.0-cdh5.4.2, r15b703c8725733b7b2813d2325659eb7d57e7a3f
Compiled:	2015-05-20T00:03Z by jenkins from Unknown
Cluster ID:	CID-09038056-e581-4817-9ae2-18ad92cd7755
Block Pool ID:	BP-989008105-127.0.0.1-1433846136903

The second screenshot shows the 'DataNode on quickstart.cloudera:50075' page, which displays the text: 'Hadoop, 2014.'

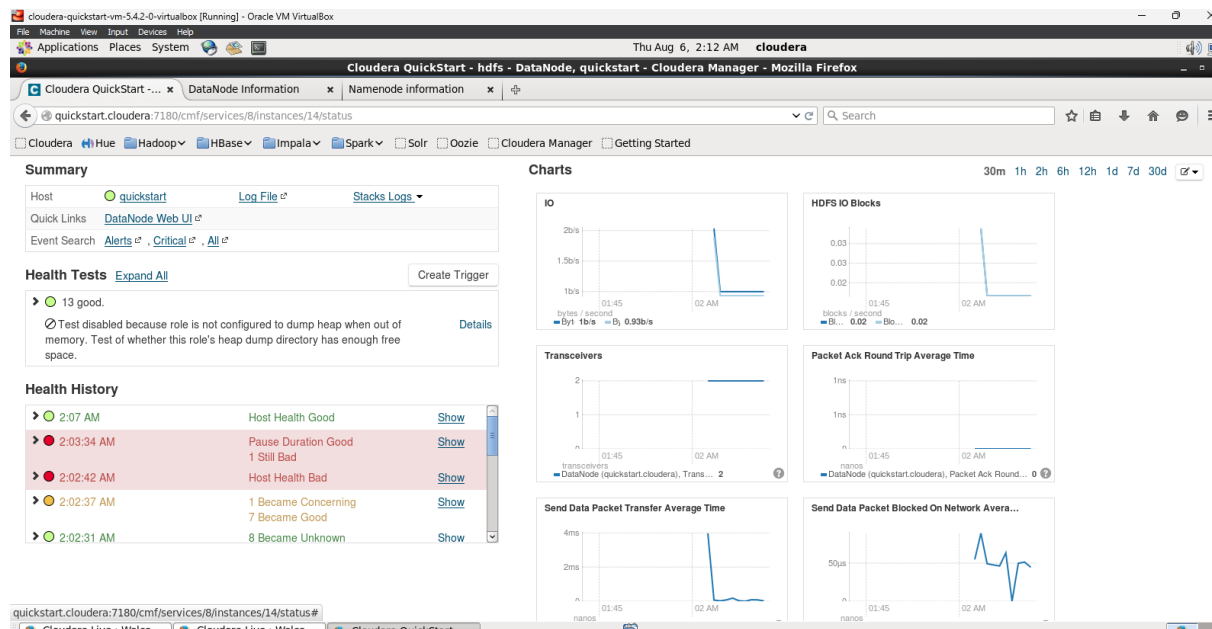
The third screenshot shows the Hue Quick Start wizard in the 'Check Configuration' step. The wizard is titled 'Quick Start Wizard - Hue™ 3.7.0 - The Hadoop UI' and includes a progress bar with four steps: Step 1: Check Configuration (active), Step 2: Examples, Step 3: Users, and Step 4: Get. The current step is 'Checking current configuration' and shows a loading spinner.

Q4. Access Hue to Use the Hadoop Web Interface

PROCEDURE:

1. Open Hue from the Cloudera environment.
2. Access the Hue Web UI.
3. Verify that the Hue interface loads successfully.
4. Use Hue to access Hadoop services.
5. Open the File Browser to browse HDFS.
6. Hue can be used to:
 - Browse HDFS directories.
 - Create directories.
 - Create files.
 - Upload files.
 - Upload directories.
7. Browse locations such as /user/Cloudera.

OUTPUT:



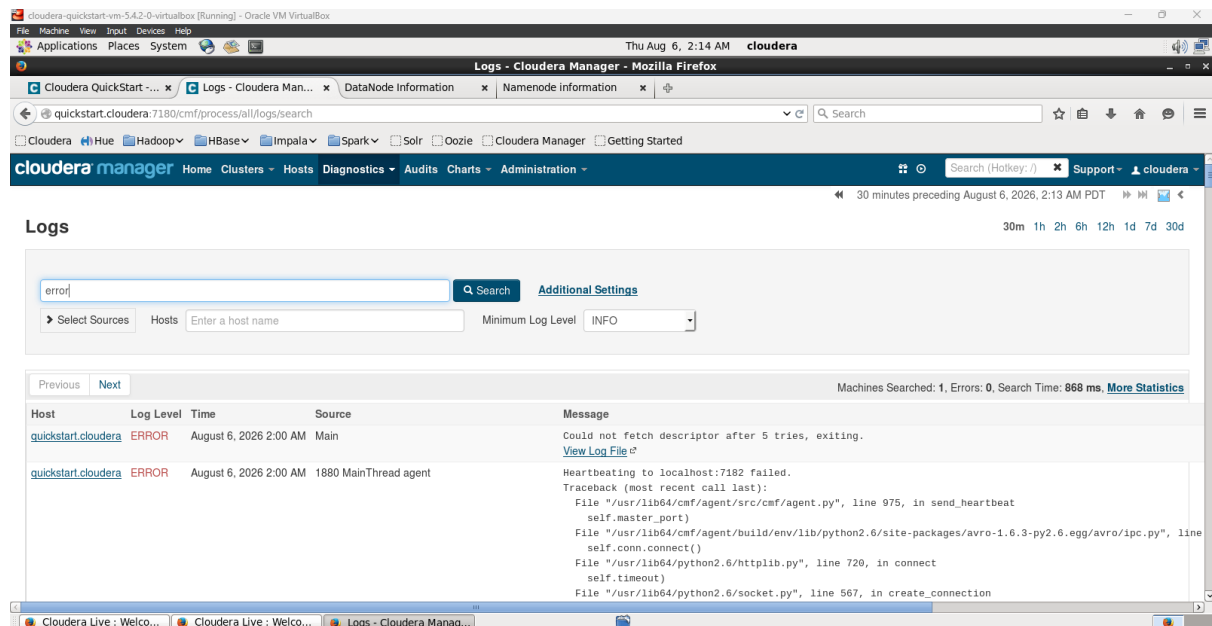
Q5. Monitor Cluster Health and Logs

PROCEDURE:

1. Open Cloudera Manager.
2. Go to Clusters → All Hosts.
3. Observe host resource information such as:
 - CPU usage
 - Memory usage
 - Disk usage
4. Open Diagnostics → Log Files.
5. Enter: error
6. Click Search.
7. Observe the ERROR-level log messages returned by Cloudera Manager.

8. Use these logs to identify Hadoop services or components reporting errors or health problems.

OUTPUT:



Q6. Run HDFS Commands using Cloudera CLI

PROCEDURE:

Open the terminal in the Cloudera VM.

First, list the existing HDFS contents: `hdfs dfs -ls`

Create a new HDFS directory named t1: `hdfs dfs -mkdir t1`

Create a local text file: `cat > sample.txt`

Enter some sample content into the file and press Ctrl+C as specified in your lab PPT.

Upload the file into the t1 directory in HDFS: `hdfs dfs -put sample.txt t1/`

Verify the uploaded file: `hdfs dfs -ls t1`

The uploaded contents can then be verified by refreshing the Hue File Browser.

OUTPUT:

```

Applications Places System cloudera@quickstart:~
cloudera@quickstart:~$ hdfs dfs -ls
Found 2 items
drwxr-xr-x - cloudera supergroup 0 2026-08-06 02:05 /user
drwxrwxrwt - cloudera supergroup 0 2026-08-04 19:50 /tmp
cloudera@quickstart:~$ hdfs dfs -mkdir t1
cloudera@quickstart:~$ hdfs dfs -ls
Found 3 items
drwxr-xr-x - cloudera supergroup 0 2026-08-06 02:05 /user
drwxrwxrwt - cloudera supergroup 0 2026-08-04 19:50 /tmp
drwxr-xr-x - cloudera supergroup 0 2026-08-06 02:18 /t1
cloudera@quickstart:~$ cat > sample.txt
This is a sample file.
It is created as part of Hadoop lab.
HDFS is awesome!
^C
cloudera@quickstart:~$ hdfs dfs -put sample.txt t1/
26/26 100%
cloudera@quickstart:~$ hdfs dfs -ls t1
Found 1 items
-rw-r--r-- 3 cloudera supergroup 78 2026-08-06 02:19 t1/sample.txt
cloudera@quickstart:~$ hdfs dfs -cat t1/sample.txt
This is a sample file.
It is created as part of Hadoop lab.
HDFS is awesome!
cloudera@quickstart:~$ hdfs dfs -ls -R t1
t1:
Found 1 items
-rw-r--r-- 3 cloudera supergroup 78 2026-08-06 02:19 t1/sample.txt
cloudera@quickstart:~$

```

RESULT: Thus, the Hadoop environment was successfully explored and managed using the Cloudera CLI and Web UI. The Hadoop services, HDFS Name Node and Data Node were viewed and monitored, Hue was accessed for HDFS operations, cluster health and logs were analysed, and basic HDFS file and directory operations were performed successfully.

EXP NO: 7**DATE:****DEMONSTRATION OF HADOOP MAPREDUCE
USING JAVA – WORD COUNT PROGRAM**

AIM: To develop and execute a Word Count MapReduce program using Java in Hadoop and verify the word-frequency output stored in HDFS.

STEPS:**Step 1: Create the Word Count directory**

Open the terminal in the Cloudera environment and create a directory named wordcount1.

```
mkdir wordcount1
```

```
cd wordcount1
```

Create the following three Java files:

```
vi WordCountDriver.java
```

```
vi WordCountMapper.java
```

```
vi WordCountReducer.java
```

WordCountMapper.java: The Mapper tokenizes each input line and emits every word with the value 1.

```
package wordcount;
```

```
import java.io.IOException;
```

```
import java.util.StringTokenizer;
```

```
import org.apache.hadoop.io.IntWritable;
```

```
import org.apache.hadoop.io.Text;
```

```
import org.apache.hadoop.io.LongWritable;
```

```
import org.apache.hadoop.mapreduce.Mapper;
```

```
public class WordCountMapper
```

```
    extends Mapper<LongWritable, Text, Text, IntWritable> {
```

```
    private final static IntWritable one = new IntWritable(1);
```

```
    private Text word = new Text();
```

```
    public void map(LongWritable key, Text value, Context context)
```

```
        throws IOException, InterruptedException {
```

```
        StringTokenizer tokenizer =
```

```
            new StringTokenizer(value.toString());
```

```
        while (tokenizer.hasMoreTokens()) {
```

```
            word.set(tokenizer.nextToken());
```

```
            context.write(word, one);
```

```
        }
```

```
    }
```

```
}
```

WordCountReducer.java: The Reducer receives each word and adds its occurrence values to obtain the final frequency.

```
package wordcount;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context)
        throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable value : values) {
            sum += value.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}
```

WordCountDriver.java: The Driver configures the MapReduce job, Mapper, Reducer, input path and output path.

```
package wordcount;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCountDriver {
```

```

public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "Word Count");

    job.setJarByClass(WordCountDriver.class);

    job.setMapperClass(WordCountMapper.class);
    job.setReducerClass(WordCountReducer.class);

    job.setCombinerClass(WordCountReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Step 2: Compile the Java files

Move back to the parent directory: `cd ..`

Compile all three Java files using the Hadoop classpath:

```
javac -classpath `hadoop classpath` -d . wordcount1/*.java
```

This creates the required .class files.

Step 3: Create the JAR file

Package the compiled classes into a JAR: `jar -cvf WordCount1.jar wordcount1/*.class`

The terminal should show the Mapper, Reducer and Driver .class files being added to the JAR.

```

[it23a47@quickstart ~]$ mkdir wordcount1
[it23a47@quickstart ~]$ cd wordcount1
[it23a47@quickstart wordcount1]$ vi WordCountDriver.java
[it23a47@quickstart wordcount1]$ vi WordCountMapper.java
[it23a47@quickstart wordcount1]$ vi WordCountReducer.java
[it23a47@quickstart wordcount1]$ cd ..
[it23a47@quickstart ~]$ javac -classpath `hadoop classpath` -d . wordcount1/*.java
[it23a47@quickstart ~]$ jar -cvf WordCount1.jar wordcount1/*.class
wordcount1/*.class : no such file or directory
added manifest
[it23a47@quickstart ~]$ ls
CreateHDFSFile.class  CreateHDFSFile.java  DeleteHDFSFile.jar  input.txt  ReadHDFSFile.jar  wordcount  WordCount1.jar  WordCount.jar
CreateHDFSFile.jar  DeleteHDFSFile.class  DeleteHDFSFile.java  ReadHDFSFile.class  ReadHDFSFile.java  wordcount1  wordcount.jar
[it23a47@quickstart ~]$ jar -cvf wordcount1.jar wordcount1/*.class
wordcount1/*.class : no such file or directory
added manifest
[it23a47@quickstart ~]$ cd wordcount1
[it23a47@quickstart wordcount1]$ vi WordCountDriver.java
[it23a47@quickstart wordcount1]$ vi WordCountMapper.java
[it23a47@quickstart wordcount1]$ vi WordCountReducer.java
[it23a47@quickstart wordcount1]$ cd ..
[it23a47@quickstart ~]$ javac -classpath `hadoop classpath` -d . wordcount1/*.java
[it23a47@quickstart ~]$ jar -cvf WordCount1.jar wordcount1/*.class
added manifest
adding: wordcount1/WordCountDriver.class(in = 1452) (out= 772) (deflated 46%)
adding: wordcount1/WordCountMapper.class(in = 1812) (out= 761) (deflated 58%)
adding: wordcount1/WordCountReducer.class(in = 1711) (out= 724) (deflated 57%)
[it23a47@quickstart ~]$ hdfs dfs -mkdir -p /user/it23a47/input.txt
[it23a47@quickstart ~]$ hdfs dfs -put input.txt /user/it23a47/input
put: /user/it23a47/input/input.txt: File exists

```

Step 4: Create the HDFS input directory and upload input file

Create the HDFS input directory: `hdfs dfs -mkdir -p /user/it23a47/input`

Upload the input file: `hdfs dfs -put input.txt /user/it23a47/input`

If the terminal displays: File exists

it means the input file was already uploaded previously, so it need not be uploaded again.

Step 5: Run the MapReduce job

Execute the Word Count program:

```
hadoop jar WordCount1.jar wordcount1.WordCountDriver /user/it23a47/input
/user/it23a47/output
```

If Hadoop reports: `FileAlreadyExistsException`

because the output directory already exists, delete the old output:

```
hdfs dfs -rm -r /user/it23a47/output
```

Then execute the MapReduce command again:

```
hadoop jar WordCount1.jar wordcount1.WordCountDriver /user/it23a47/input
/user/it23a47/output
```

```
[it23a47@quickstart ~]$ hadoop jar WordCount1.jar wordcount1.WordCountDriver /user/it23a47/input /user/it23a47/output
26/08/13 02:21:19 INFO client.RMProxy: Connecting to ResourceManager at quickstart.cloudera/192.168.19.105:8032
26/08/13 02:21:20 WARN security.UserGroupInformation: PrivilegedActionException as:it23a47 (auth:SIMPLE) cause:org.apache.hadoop.mapred.FileAlreadyExistsException: Output directory hdfs://quickstart.cloudera:8020/user/it23a47/output already exists
Exception in thread "main" org.apache.hadoop.mapred.FileAlreadyExistsException: Output directory hdfs://quickstart.cloudera:8020/user/it23a47/output already exists
    at org.apache.hadoop.mapreduce.lib.output.FileOutputFormat.checkOutputSpecs(FileOutputFormat.java:146)
    at org.apache.hadoop.mapreduce.JobSubmitter.checkSpecs(JobSubmitter.java:562)
    at org.apache.hadoop.mapreduce.JobSubmitter.submitJobInternal(JobSubmitter.java:432)
    at org.apache.hadoop.mapreduce.Job$10.run(Job.java:1306)
    at org.apache.hadoop.mapreduce.Job$10.run(Job.java:1303)
    at java.security.AccessController.doPrivileged(Native Method)
    at javax.security.auth.Subject.doAs(Subject.java:415)
    at org.apache.hadoop.security.UserGroupInformation.doAs(UserGroupInformation.java:1671)
    at org.apache.hadoop.mapreduce.Job.submit(Job.java:1303)
    at org.apache.hadoop.mapreduce.Job.waitForCompletion(Job.java:1324)
    at wordcount1.WordCountDriver.main(WordCountDriver.java:36)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:57)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.lang.reflect.Method.invoke(Method.java:606)
    at org.apache.hadoop.util.RunJar.run(RunJar.java:221)
    at org.apache.hadoop.util.RunJar.main(RunJar.java:136)
[it23a47@quickstart ~]$ hdfs dfs -rm -r /user/it23a47/output
26/08/13 02:22:53 INFO fs.TrashPolicyDefault: Namenode trash configuration: Deletion interval = 1440 minutes, Emptier interval = 0 minutes.
Moved: 'hdfs://quickstart.cloudera:8020/user/it23a47/output' to trash at: hdfs://quickstart.cloudera:8020/user/it23a47/.Trash/Current
[it23a47@quickstart ~]$ hadoop jar WordCount1.jar wordcount1.WordCountDriver /user/it23a47/input /user/it23a47/output
26/08/13 02:23:14 INFO client.RMProxy: Connecting to ResourceManager at quickstart.cloudera/192.168.19.105:8032
26/08/13 02:23:14 WARN mapreduce.JobSubmitter: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
26/08/13 02:23:15 INFO input.FileInputFormat: Total input paths to process : 1
26/08/13 02:23:15 INFO mapreduce.JobSubmitter: number of splits:1
26/08/13 02:23:15 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1786611919957_0005
26/08/13 02:23:15 INFO impl.YarnClientImpl: Submitted application application_1786611919957_0005
26/08/13 02:23:15 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1786611919957_0005/
26/08/13 02:23:15 INFO mapreduce.Job: Running job: job_1786611919957_0005
```

Step 6: Verify successful MapReduce execution

During execution, Hadoop performs the **Map** and **Reduce** phases.

It also displays MapReduce counters such as:

- Map input records
- Map output records
- Reduce input groups
- Reduce output records

```

26/08/13 02:23:21 INFO mapreduce.Job: Job job_1786611919957_0005 running in uber mode : false
26/08/13 02:23:21 INFO mapreduce.Job: map 0% reduce 0%
26/08/13 02:23:26 INFO mapreduce.Job: map 100% reduce 0%
26/08/13 02:23:31 INFO mapreduce.Job: map 100% reduce 100%
26/08/13 02:23:31 INFO mapreduce.Job: Job job_1786611919957_0005 completed successfully
26/08/13 02:23:31 INFO mapreduce.Job: Counters: 49
  File System Counters
    FILE: Number of bytes read=111
    FILE: Number of bytes written=227825
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=190
    HDFS: Number of bytes written=66
    HDFS: Number of read operations=6
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
  Job Counters
    Launched map tasks=1
    Launched reduce tasks=1
    Data-local map tasks=1
    Total time spent by all maps in occupied slots (ms)=353280
    Total time spent by all reduces in occupied slots (ms)=431360
    Total time spent by all map tasks (ms)=2760
    Total time spent by all reduce tasks (ms)=3370
    Total vcore-seconds taken by all map tasks=2760
    Total vcore-seconds taken by all reduce tasks=3370
    Total megabyte-seconds taken by all map tasks=353280
    Total megabyte-seconds taken by all reduce tasks=431360
  Map-Reduce Framework
    Map input records=2
    Map output records=11
    Map output bytes=108
    Map output materialized bytes=107
    Input split bytes=125
    Combine input records=11
    Combine output records=9
    Reduce input groups=9
    Reduce shuffle bytes=107
    Reduce input records=9
    Reduce output records=9
    Spilled Records=18
    Shuffled Maps =1
    Failed Shuffles=0
    Merged Map outputs=1
    GC time elapsed (ms)=86
    CPU time spent (ms)=1030
    Physical memory (bytes) snapshot=269918208
    Virtual memory (bytes) snapshot=1449623552
    Total committed heap usage (bytes)=95944704
  Shuffle Errors

```

Step 7: View the generated output

List the contents of the HDFS output directory:

```
hdfs dfs -ls /user/it23a47/output
```

The output directory contains files such as:

- _SUCCESS
- part-r-00000

Display the final Word Count result:

```
hdfs dfs -cat /user/it23a47/output/part-r-00000
```

```
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=65
File Output Format Counters
  Bytes Written=66
[it23a47@quickstart ~]$ hdfs dfs -ls /user/it23a47/output
Found 2 items
-rw-r--r--  1 it23a47 it23a47          0 2026-08-13 02:23 /user/it23a47/output/_SUCCESS
-rw-r--r--  1 it23a47 it23a47        66 2026-08-13 02:23 /user/it23a47/output/part-r-00000
[it23a47@quickstart ~]$ hdfs dfs -cat /user/it23a47/output/part-r-00000
I'm      1
as       1
calls    1
everybody 1
hi,      1
i'm      1
known    1
krithi.  3
me       1
[it23a47@quickstart ~]$
```

RESULT: Thus, the Word Count program was successfully implemented using Java MapReduce and executed in the Hadoop environment. The input file was processed through the Mapper and Reducer phases, and the frequency of each word was successfully generated and stored in HDFS.

EXP NO: 8**DATE:****IMPLEMENTATION OF CRUD OPERATIONS
IN A BIG DATA ENVIRONMENT**

AIM: To implement and demonstrate Create, Read and Append operations on files in HDFS using the Hadoop Java API.

STEPS:**Step 1 – CREATE Operation**

Create a Java file: vi CreateHDFSFile.java. Enter:

```
import java.io.IOException;
import java.net.URI;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.fs.FSDataOutputStream;

public class CreateHDFSFile {

    public static void main(String[] args) {

        try {

            String hdfsUri =
                "hdfs://quickstart.cloudera:8020";

            String filePath =
                "/user/john/sample.txt";

            Configuration conf =
                new Configuration();

            conf.addResource(
                new Path("/etc/hadoop/conf/core-site.xml")
            );

            conf.addResource(
                new Path("/etc/hadoop/conf/hdfs-site.xml")
            );

            FileSystem fs =
                FileSystem.get(
                    new URI(hdfsUri),
                    conf
                );
```

```

    Path path =
        new Path(filePath);

    FSDataOutputStream os =
        fs.create(path, true);

    String content =
        "Hello, this is a test file in HDFS.\n";

    os.writeBytes(content);

    os.close();

    System.out.println(
        "File created successfully in HDFS at: "
        + filePath
    );

    fs.close();

} catch (IOException e) {
    e.printStackTrace();

} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Compile: `javac -classpath `hadoop classpath` -d . CreateHDFSFile.java`

Create the JAR: `jar -cvf createfile.jar CreateHDFSFile.class`

Run: `hadoop jar createfile.jar CreateHDFSFile`

Verify: `hdfs dfs -cat /user/john/sample.txt`

OUTPUT:

```

[it23a41@quickstart ~]$ vi CreateHDFSFile.java
[it23a41@quickstart ~]$ javac -classpath `hadoop classpath` -d . CreateHDFSFile.java
[it23a41@quickstart ~]$ jar -cvf CreateHDFSFile.jar *.class
added manifest
adding: CreateHDFSFile.class(in = 1622) (out= 932) (deflated 42%)
[it23a41@quickstart ~]$ hadoop jar CreateHDFSFile.jar CreateHDFSFile
File created successfully in HDFS at: /user/it23a41/sample.txt
[it23a41@quickstart ~]$ hdfs dfs -ls
Found 4 items
drwx----- - it23a41 it23a41          0 2026-08-06 00:58 .Trash
drwx----- - it23a41 it23a41          0 2026-08-06 01:06 .staging
drwxr-xr-x - it23a41 it23a41          0 2026-08-06 00:57 input
-rw-r--r-- 1 it23a41 it23a41        36 2026-08-13 00:17 sample.txt
[it23a41@quickstart ~]$ hdfs dfs -cat sample.txt
Hello, this is a test file in HDFS.
[it23a41@quickstart ~]$

```

Step 2 – READ Operation

Create: vi ReadHDFSFile.java. Enter:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.URI;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.fs.FSDataInputStream;

public class ReadHDFSFile {

    public static void main(String[] args) {

        try {

            String hdfsUri =
                "hdfs://quickstart.cloudera:8020";

            String filePath =
                "/user/john/sample.txt";

            Configuration conf =
                new Configuration();

            conf.addResource(
                new Path("/etc/hadoop/conf/core-site.xml")
            );

            conf.addResource(
                new Path("/etc/hadoop/conf/hdfs-site.xml")
            );

            FileSystem fs =
                FileSystem.get(
                    new URI(hdfsUri),
                    conf
                );

            Path path =
                new Path(filePath);

            FSDataInputStream input =
                fs.open(path);

            BufferedReader reader =
                new BufferedReader(
                    new InputStreamReader(input)
```

```

        );

        String line;

        System.out.println(
            "Contents of HDFS file:"
        );

        while ((line = reader.readLine()) != null) {
            System.out.println(line);
        }

        reader.close();
        input.close();
        fs.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Compile: `javac -classpath `hadoop classpath` -d . ReadHDFSFile.java`

Create the JAR: `jar -cvf readfile.jar ReadHDFSFile.class`

Execute: `hadoop jar readfile.jar ReadHDFSFile`

OUTPUT:

```

[it23a41@quickstart ~]$ vi ReadHDFSFile.java
[it23a41@quickstart ~]$ javac -classpath `hadoop classpath` -d .
ReadHDFSFile.java
[it23a41@quickstart ~]$ jar -cvf ReadHDFSFile.jar *.class
added manifest
adding: CreateHDFSFile.class(in = 1622) (out= 932)(deflated 42%)
adding: ReadHDFSFile.class(in = 1903) (out= 1050)(deflated 44%)
[it23a41@quickstart ~]$ hadoop jar ReadHDFSFile.jar ReadHDFSFile
Hello, this is a test file in HDFS.
[it23a41@quickstart ~]$ 

```

Step 3 – UPDATE / APPEND Operation

Create: `vi AppendHDFSFile.java`. Enter:

```

import java.net.URI;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.fs.FSDataOutputStream;

```

```
public class AppendHDFSFile {  
  
    public static void main(String[] args) {  
  
        try {  
  
            String hdfsUri =  
                "hdfs://quickstart.cloudera:8020";  
  
            String filePath =  
                "/user/john/sample.txt";  
  
            Configuration conf =  
                new Configuration();  
  
            conf.addResource(  
                new Path("/etc/hadoop/conf/core-site.xml")  
            );  
  
            conf.addResource(  
                new Path("/etc/hadoop/conf/hdfs-site.xml")  
            );  
  
            FileSystem fs =  
                FileSystem.get(  
                    new URI(hdfsUri),  
                    conf  
                );  
  
            Path path =  
                new Path(filePath);  
  
            FSDataOutputStream os =  
                fs.append(path);  
  
            String newContent =  
                "This line is appended using Hadoop Java API.\n";  
  
            os.writeBytes(newContent);  
  
            os.close();  
  
            System.out.println(  
                "Data appended successfully to: "  
                + filePath  
            );  
  
            fs.close();  
  
        } catch (Exception e) {
```

```

        e.printStackTrace();
    }
}
}

```

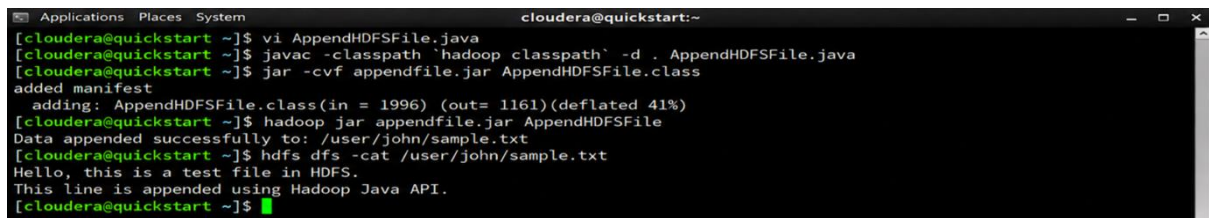
Compile: `javac -classpath `hadoop classpath` -d . AppendHDFSFile.java`

Create JAR: `jar -cvf appendfile.jar AppendHDFSFile.class`

Execute: `hadoop jar appendfile.jar AppendHDFSFile`

Verify: `hdfs dfs -cat /user/john/sample.txt`

OUTPUT:



```

[cloudera@quickstart ~]$ vi AppendHDFSFile.java
[cloudera@quickstart ~]$ javac -classpath `hadoop classpath` -d . AppendHDFSFile.java
[cloudera@quickstart ~]$ jar -cvf appendfile.jar AppendHDFSFile.class
added manifest
adding: AppendHDFSFile.class(in = 1996) (out= 1161)(deflated 41%)
[cloudera@quickstart ~]$ hadoop jar appendfile.jar AppendHDFSFile
Data appended successfully to: /user/john/sample.txt
[cloudera@quickstart ~]$ hdfs dfs -cat /user/john/sample.txt
Hello, this is a test file in HDFS.
This line is appended using Hadoop Java API.
[cloudera@quickstart ~]$

```

Step 4 – DELETE Operation

Create: `vi DeleteHDFSFile.java`. Enter:

```
import java.net.URI;
```

```
import org.apache.hadoop.conf.Configuration;
```

```
import org.apache.hadoop.fs.FileSystem;
```

```
import org.apache.hadoop.fs.Path;
```

```
public class DeleteHDFSFile {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            String hdfsUri =
```

```
                "hdfs://quickstart.cloudera:8020";
```

```
            String filePath =
```

```
                "/user/john /sample.txt";
```

```
            Configuration conf =
```

```
                new Configuration();
```

```
            conf.addResource(
```

```
                new Path("/etc/hadoop/conf/core-site.xml")
```

```
            );
```

```
            conf.addResource(
```

```
                new Path("/etc/hadoop/conf/hdfs-site.xml")
```

```

    );

    FileSystem fs =
        FileSystem.get(
            new URI(hdfsUri),
            conf
        );

    Path path =
        new Path(filePath);

    boolean deleted =
        fs.delete(path, false);

    if (deleted) {

        System.out.println(
            "File deleted successfully from HDFS."
        );

    } else {

        System.out.println(
            "File could not be deleted."
        );

    }

    fs.close();

} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Compile: `javac -classpath `hadoop classpath` -d . DeleteHDFSFile.java`

Create JAR: `jar -cvf deletefile.jar DeleteHDFSFile.class`

Execute: `hadoop jar deletefile.jar DeleteHDFSFile`

Verify that the file has been removed: `hdfs dfs -ls /user/john/`

OUTPUT:

```

[it23a41@quickstart ~]$ vi ReadHDFSFile.java
[it23a41@quickstart ~]$ javac -classpath `hadoop classpath` -d .
ReadHDFSFile.java
[it23a41@quickstart ~]$ jar -cvf ReadHDFSFile.jar *.class
added manifest
adding: CreateHDFSFile.class(in = 1622) (out= 932)(deflated 42%)
adding: ReadHDFSFile.class(in = 1903) (out= 1050)(deflated 44%)
[it23a41@quickstart ~]$ hadoop jar ReadHDFSFile.jar ReadHDFSFile
Hello, this is a test file in HDFS.
[it23a41@quickstart ~]$ vi DeleteHDFSFile.java
[it23a41@quickstart ~]$ javac -classpath `hadoop classpath` -d . DeleteHDFSFile.java
[it23a41@quickstart ~]$ jar -cvf DeleteHDFSFile.jar *.class
added manifest
adding: CreateHDFSFile.class(in = 1622) (out= 932)(deflated 42%)
adding: DeleteHDFSFile.class(in = 1602) (out= 933)(deflated 41%)
adding: ReadHDFSFile.class(in = 1903) (out= 1050)(deflated 44%)
[it23a41@quickstart ~]$ hadoop jar DeleteHDFSFile.jar DeleteHDFSFile
File deleted successfully from HDFS: /user/it23a41/sample.txt
[it23a41@quickstart ~]$ hdfs dfs -ls
Found 6 items
drwx----- - it23a41 it23a41          0 2026-08-06 00:58 .Trash
drwx----- - it23a41 it23a41          0 2026-08-13 00:35 .staging
drwxr-xr-x - it23a41 it23a41          0 2026-08-06 00:57 input
drwxr-xr-x - it23a41 it23a41          0 2026-08-13 00:35 oozie-oozi
drwxr-xr-x - it23a41 it23a41          0 2026-08-13 00:33 output
-rw-r--r-- 1 it23a41 it23a41        173 2026-08-13 00:30 student.csv
[it23a41@quickstart ~]$ █

```

RESULT: Thus, the file operation in HDFS was successfully implemented using the Hadoop Java API. The Java program successfully connected to HDFS, performed CRUD operations and its contents were verified successfully.

EXP NO: 9
DATE:

EXECUTION OF PIG LATIN QUERIES FOR DATA ANALYSIS USING APACHE PIG

AIM: To execute Pig Latin queries on a student dataset using Apache Pig in the Cloudera environment and perform basic data-analysis operations such as loading, filtering, projection, grouping, counting, and aggregation.

QUESTIONS:

Step 1: Prepare the Student Dataset

Create a file named student.csv containing:

```
1,adarsh,int,83
2,aswin,cs,101
3,kumar,mec,105
4,bershey,bio,106
```

Upload the file to HDFS.

OUTPUT:

Name	Size	User	Group	Permissions	Date
hdfs		hdfs	supergroup	drwxr-xr-x	August 06, 2026 12:42 AM
.		it23a41	it23a41	drwxr-xr-x	August 13, 2026 12:33 AM
.Trash		it23a41	it23a41	drwxr-xr-x	August 06, 2026 12:58 AM
.staging		it23a41	it23a41	drwxr-xr-x	August 13, 2026 12:35 AM
input		it23a41	it23a41	drwxr-xr-x	August 06, 2026 12:57 AM
oozie-oozi		it23a41	it23a41	drwxr-xr-x	August 13, 2026 12:35 AM
output		it23a41	it23a41	drwxr-xr-x	August 13, 2026 12:33 AM
sample.txt	36 bytes	it23a41	it23a41	-rw-r--r--	August 13, 2026 12:17 AM
student.csv	173 bytes	it23a41	it23a41	-rw-r--r--	August 13, 2026 12:30 AM

Step 2: Start Apache Pig

Open the terminal in the Cloudera VM and select Pig from Query Editors

Q1: Load and Display/Save All Records

```
students = LOAD '/user/john/student.csv'
USING PigStorage(',')
AS (roll:int,name:chararray,dept:chararray,marks:int);
STORE students
INTO '/user/john/output/q1'
USING PigStorage(',');
```

OUTPUT:

```
1,adarsh,int,83
2,aswin,cs,101
3,kumar,mec,105
4,bershey,bio,106
```

Q2: Filter Students Scoring More Than 85

```
high = FILTER students BY marks > 85;
STORE high
INTO '/user/john/output/q2'
USING PigStorage(',');
```

OUTPUT:

```
2,aswin,cs,101
3,kumar,mec,105
4,bershey,bio,106
```

Q3: Display Only Student Name and Marks

```
result1 = FOREACH students GENERATE name, marks;
STORE result1
INTO '/user/john/output/q3'
USING PigStorage(',');
```

OUTPUT:

```
adarsh,83
aswin,101
kumar,105
bershey,106
```

Q4: Group Students Department-wise

```
grp = GROUP students BY dept;
result2 = FOREACH grp
GENERATE group, COUNT(students);
STORE result2
```

```
INTO '/user/john/output/q4'  
USING PigStorage(',');
```

OUTPUT:

```
cs,1  
bio,1  
int,1  
mec,1
```

Q5: Find Average Marks Department-wise

```
result3 = FOREACH grp  
GENERATE group, AVG(students.marks);  
STORE result3  
INTO '/user/john/output/q5'  
USING PigStorage(',');
```

OUTPUT:

```
cs,101.0  
bio,106.0  
int,83.0  
mec,105.0
```

RESULT: Thus, the student dataset was successfully loaded and analysed using Apache Pig in the Cloudera environment. Pig Latin operations such as LOAD, FILTER, FOREACH, GROUP, COUNT, AVG, DUMP, and STORE were performed successfully to filter student records, project selected fields, group students department-wise, count students, and calculate average marks.

EXP NO: 10**DATE:****IMPLEMENTATION OF BASIC HIVEQL DATABASE,
TABLE, DATA INSERTION AND QUERY**

AIM: To create a Hive database and employee table, insert employee records, and perform basic HiveQL queries on the inserted data.

QUESTIONS:**Step 1: Open Hive Editor**

Open the Cloudera Hue Web UI and select: Query Editors → Hive

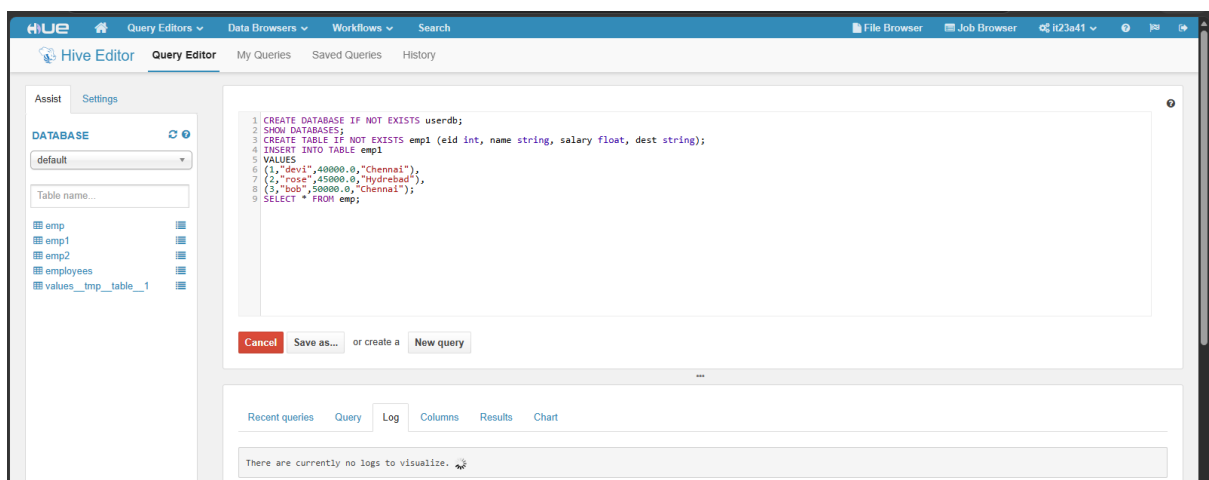
Step 2: Create the Database

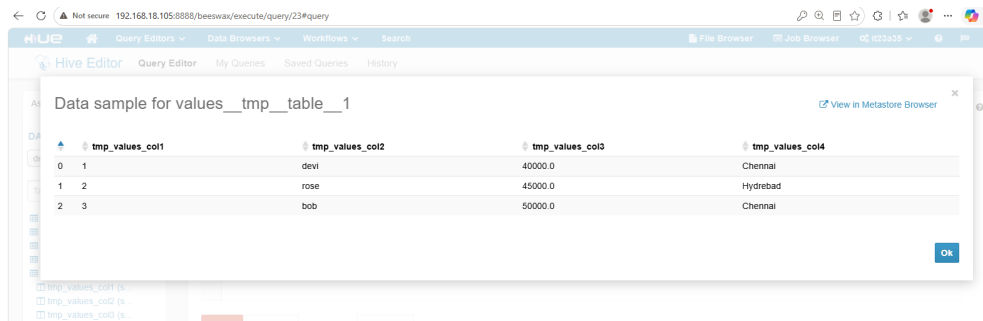
```
CREATE DATABASE IF NOT EXISTS userdb;
SHOW DATABASES;
USE userdb;
```

Q1: Create the Employee Table

```
CREATE TABLE IF NOT EXISTS emp1 (
    eid INT,
    name STRING,
    salary FLOAT,
    dest STRING
);
```

```
INSERT INTO TABLE emp1
VALUES
(1,"devi",40000.0,"Chennai"),
(2,"rose",45000.0,"Hydrebad"),
(3,"bob",50000.0,"Chennai");
Display all records:
SELECT * FROM emp1;
```

OUTPUT:

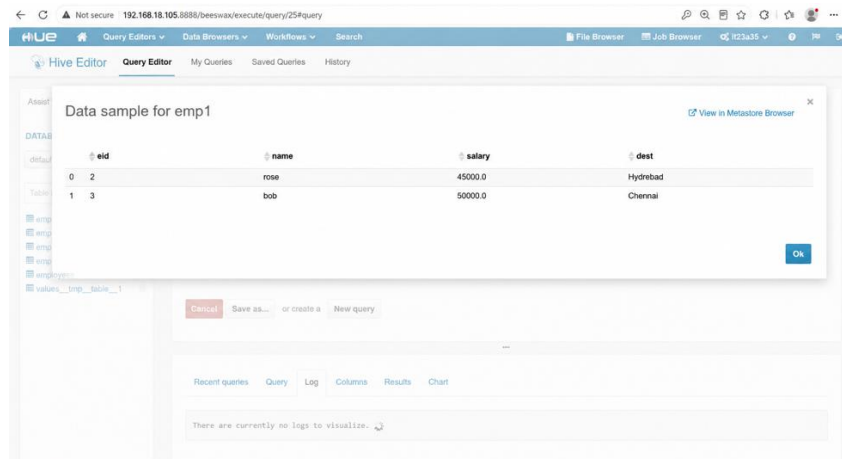


	tmp_values_col1	tmp_values_col2	tmp_values_col3	tmp_values_col4
0	1	devi	40000.0	Chennai
1	2	rose	45000.0	Hyderabad
2	3	bob	50000.0	Chennai

Q2: Display Employees with Salary Greater Than 40,000

```
SELECT *
FROM emp1
WHERE salary > 40000;
```

OUTPUT:



	eid	name	salary	dest
0	2	rose	45000.0	Hyderabad
1	3	bob	50000.0	Chennai

Q3: Display Employee Name and Destination

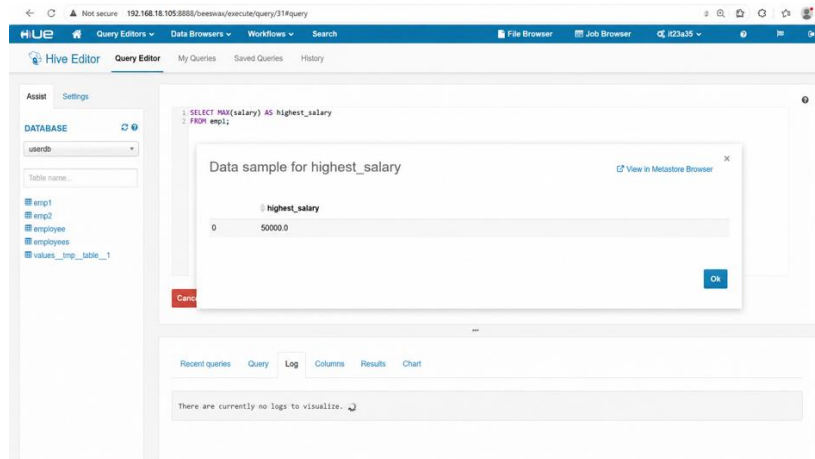
```
SELECT name, dest
FROM emp1;
```

OUTPUT:

	name	dest
0	devi	Chennai
1	rose	Hyderabad
2	bob	Chennai

Q4: Find the Highest Salary

```
SELECT MAX(salary) AS highest_salary
FROM emp1;
```

OUTPUT:**Q5: Display Employees in Descending Order of Salary**

```
SELECT eid, name, salary, dest
FROM emp1
ORDER BY salary DESC;
```

OUTPUT:

Data sample for emp1 [View in Metastore Browser](#)

	eid	name	salary	dest
0	3	bob	50000.0	Chennai
1	2	rose	45000.0	Hydreb主
2	1	devi	40000.0	Chennai

Ok

RESULT: Thus, a Hive database and employee table were successfully created using HiveQL in the Cloudera environment. Employee records were inserted and retrieved successfully, and various HiveQL operations such as filtering, projection, aggregate function MAX(), and sorting using ORDER BY were performed successfully.

EXP NO: 11**WORD COUNT USING SPARK 1.3.0 AND HDFS****DATE:**

AIM: To implement a simple Word Count program in Apache Spark using RDDs by reading input data from HDFS and writing the output back to HDFS.

STEPS:

1. Open the Cloudera environment and connect through the terminal/Putty.
2. Create an input text file in HDFS using Hue WebUI under the user directory.
Example content: hello spark hello Hadoop spark makes big data processing easy
3. Start the Spark shell: spark-shell
A Scala REPL should open, and Spark should indicate that the Spark context is available as sc and SQL context as sqlContext.
4. Load the input file from HDFS:

```
val textFile = sc.textFile("hdfs:///user/john/input.txt")
```

 To verify that the file is loaded: `textFile.first()`
5. Split each line into individual words:

```
val words = textFile.flatMap(line => line.split("\\s+"))
```
6. Convert each word into a key-value pair (word, 1):

```
val wordPairs = words.map(word => (word, 1))
```
7. Count the occurrences of each word using `reduceByKey`:

```
val wordCounts = wordPairs.reduceByKey(_ + _)
```
8. Display the result:

```
wordCounts.collect().foreach { case (word, count) =>
  println(s"$word: $count")
}
```
9. Save the result back to HDFS:

```
wordCounts.saveAsTextFile("hdfs:///user/john/output_wordcount")
```
10. Exit the Scala shell: `:quit` or: `exit`


```

        tracking URL: http://quickstart.cloudera:8088/proxy/application_1/85826
660745_0018/
        user: cloudera
26/08/31 00:19:56 INFO YarnClientSchedulerBackend: Application application_17858
26660745_0018 has started running.
26/08/31 00:19:58 INFO NettyBlockTransferService: Server created on 48388
26/08/31 00:19:58 INFO BlockManagerMaster: Trying to register BlockManager
26/08/31 00:19:59 INFO BlockManagerMasterActor: Registering block manager quicks
tart.cloudera:48388 with 265.4 MB RAM, BlockManagerId(<driver>, quickstart.cloud
era, 48388)
26/08/31 00:20:01 INFO BlockManagerMaster: Registered BlockManager
26/08/31 00:20:06 INFO EventLoggingListener: Logging events to hdfs://quickstart
.cloudera:8020/user/spark/applicationHistory/application_1785826660745_0018
26/08/31 00:20:07 INFO YarnClientSchedulerBackend: SchedulerBackend is ready for
scheduling beginning after waiting maxRegisteredResourcesWaitingTime: 30000(ms)
26/08/31 00:20:07 INFO SparkILoop: Created spark context..
Spark context available as sc.
26/08/31 00:20:11 INFO SparkILoop: Created sql context (with Hive support)..
SQL context available as sqlContext.

```

```

scala> 26/08/31 00:20:49 INFO YarnClientSchedulerBackend: Registered executor: Ac
26/08/31 00:20:52 INFO BlockManagerMasterActor: Registering block manager quickst

```

```
scala> █
```

```

scala> val textfile1 = sc.textFile("hdfs:///user/cloudera/input.txt")
26/08/31 00:23:34 INFO MemoryStore: ensureFreeSpace(283068) called with curMem=305316, maxMem=278302556
26/08/31 00:23:34 INFO MemoryStore: Block broadcast_1 stored as values in memory (estimated size 276.4
26/08/31 00:23:34 INFO MemoryStore: ensureFreeSpace(22296) called with curMem=588384, maxMem=278302556
26/08/31 00:23:34 INFO MemoryStore: Block broadcast_1_piece0 stored as bytes in memory (estimated size
26/08/31 00:23:34 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on quickstart.cloudera:4838
26/08/31 00:23:34 INFO BlockManagerMaster: Updated info of block broadcast_1_piece0
26/08/31 00:23:34 INFO SparkContext: Created broadcast 1 from textFile at <console>:21
textfile1: org.apache.spark.rdd.RDD[String] = hdfs:///user/cloudera/input.txt MapPartitionsRDD[3] at te

```

```

scala> textfile1.first()
26/08/31 00:23:47 INFO FileInputFormat: Total input paths to process : 1
26/08/31 00:23:48 INFO SparkContext: Starting job: first at <console>:24
26/08/31 00:23:48 INFO DAGScheduler: Got job 0 (first at <console>:24) with 1 output partitions (allow
26/08/31 00:23:48 INFO DAGScheduler: Final stage: Stage 0(first at <console>:24)
26/08/31 00:23:48 INFO DAGScheduler: Parents of final stage: List()
26/08/31 00:23:48 INFO DAGScheduler: Missing parents: List()
26/08/31 00:23:48 INFO DAGScheduler: Submitting Stage 0 (hdfs:///user/cloudera/input.txt MapPartitionsF
26/08/31 00:23:48 INFO MemoryStore: ensureFreeSpace(2680) called with curMem=610680, maxMem=278302556
26/08/31 00:23:48 INFO MemoryStore: Block broadcast_2 stored as values in memory (estimated size 2.6 Kf
26/08/31 00:23:48 INFO MemoryStore: ensureFreeSpace(1656) called with curMem=613360, maxMem=278302556
26/08/31 00:23:48 INFO MemoryStore: Block broadcast_2_piece0 stored as bytes in memory (estimated size
26/08/31 00:23:48 INFO BlockManagerInfo: Added broadcast_2_piece0 in memory on quickstart.cloudera:4838
26/08/31 00:23:48 INFO BlockManagerMaster: Updated info of block broadcast_2_piece0
26/08/31 00:23:48 INFO SparkContext: Created broadcast 2 from broadcast at DAGScheduler.scala:839
26/08/31 00:23:48 INFO DAGScheduler: Submitting 1 missing tasks from Stage 0 (hdfs:///user/cloudera/inp
26/08/31 00:23:48 INFO YarnScheduler: Adding task set 0.0 with 1 tasks
26/08/31 00:23:49 INFO TaskSetManager: Starting task 0.0 in stage 0.0 (TID 0, quickstart.cloudera, NODE
26/08/31 00:23:53 INFO BlockManagerInfo: Added broadcast_2_piece0 in memory on quickstart.cloudera:3886
26/08/31 00:23:56 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on quickstart.cloudera:3886
26/08/31 00:24:13 INFO TaskSetManager: Finished task 0.0 in stage 0.0 (TID 0) in 24415 ms on quickstart
26/08/31 00:24:13 INFO DAGScheduler: Stage 0 (first at <console>:24) finished in 24.516 s
26/08/31 00:24:13 INFO YarnScheduler: Removed TaskSet 0.0, whose tasks have all completed, from pool
26/08/31 00:24:13 INFO DAGScheduler: Job 0 finished: first at <console>:24, took 25.511302 s
res1: String = hello spark hello hadoop spark makes big data processing easy

```

```

scala> val words = textfile1.flatMap(line => line.split("\\s+"))
words: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[4] at flatMap at <console>:23

scala> val wordpairs = words.map(word => (word,1))
wordpairs: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[5] at map at <console>:25

scala> val wordcounts = wordpairs.reduceByKey(_+_ )
wordcounts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[6] at reduceByKey at <console>:27

scala> wordcounts.collect().foreach {case (word,count) => println(s"$word: $count")}
26/08/31 00:24:19 INFO SparkContext: Starting job: collect at <console>:30
26/08/31 00:24:19 INFO DAGScheduler: Registering RDD 5 (map at <console>:25)
26/08/31 00:24:19 INFO DAGScheduler: Got job 1 (collect at <console>:30) with 2 output partitions (a
26/08/31 00:24:19 INFO DAGScheduler: Final stage: Stage 2(collect at <console>:30)
26/08/31 00:24:19 INFO DAGScheduler: Parents of final stage: List(Stage 1)
26/08/31 00:24:19 INFO DAGScheduler: Missing parents: List(Stage 1)
26/08/31 00:24:19 INFO DAGScheduler: Submitting Stage 1 (MapPartitionsRDD[5] at map at <console>:25)
26/08/31 00:24:19 INFO MemoryStore: ensureFreeSpace(3752) called with curMem=615016, maxMem=27830255
26/08/31 00:24:19 INFO MemoryStore: Block broadcast_3 stored as values in memory (estimated size 3.7
26/08/31 00:24:19 INFO MemoryStore: ensureFreeSpace(2158) called with curMem=618768, maxMem=27830255
26/08/31 00:24:19 INFO MemoryStore: Block broadcast_3_piece0 stored as bytes in memory (estimated si
26/08/31 00:24:19 INFO BlockManagerInfo: Added broadcast_3_piece0 in memory on quickstart.cloudera:4
26/08/31 00:24:19 INFO BlockManagerMaster: Updated info of block broadcast_3_piece0
26/08/31 00:24:19 INFO SparkContext: Created broadcast 3 from broadcast at DAGScheduler.scala:839
26/08/31 00:24:19 INFO DAGScheduler: Submitting 2 missing tasks from Stage 1 (MapPartitionsRDD[5] at
26/08/31 00:24:23 INFO SparkContext:
26/08/31 00:24:23 INFO DAGScheduler:
26/08/31 00:24:23 INFO YarnScheduler:
26/08/31 00:24:23 INFO TaskSetManager
26/08/31 00:24:23 INFO BlockManagerIr
26/08/31 00:24:23 INFO MapOutputTrack
26/08/31 00:24:23 INFO MapOutputTrack
26/08/31 00:24:24 INFO TaskSetManager
26/08/31 00:24:24 INFO TaskSetManager
26/08/31 00:24:24 INFO DAGScheduler:
26/08/31 00:24:24 INFO DAGScheduler:
26/08/31 00:24:24 INFO TaskSetManager
26/08/31 00:24:24 INFO YarnScheduler:
big: 1
data: 1
hello: 2
easy: 1
spark: 2
hadoop: 1
processing: 1
makes: 1

```

```

scala> wordcounts.saveAsTextFile("hdfs:///user/cloudera/output_wordcount")
26/08/31 00:26:26 INFO BlockManager: Removing broadcast 4
26/08/31 00:26:26 INFO BlockManager: Removing block broadcast_4
26/08/31 00:26:26 INFO MemoryStore: Block broadcast_4 of size 2128 dropped from memory (free 27768032)
26/08/31 00:26:26 INFO BlockManager: Removing block broadcast_4_piece0
26/08/31 00:26:26 INFO MemoryStore: Block broadcast_4_piece0 of size 1310 dropped from memory (free 27768032)
26/08/31 00:26:26 INFO BlockManagerInfo: Removed broadcast_4_piece0 on quickstart.cloudera:48388 in memory
26/08/31 00:26:26 INFO BlockManagerMaster: Updated info of block broadcast_4_piece0
26/08/31 00:26:26 INFO BlockManagerInfo: Removed broadcast_4_piece0 on quickstart.cloudera:38801 in memory
26/08/31 00:26:26 INFO ContextCleaner: Cleaned broadcast 4
26/08/31 00:26:26 INFO BlockManager: Removing broadcast 3
26/08/31 00:26:26 INFO BlockManager: Removing block broadcast_3_piece0
26/08/31 00:26:26 INFO MemoryStore: Block broadcast_3_piece0 of size 2158 dropped from memory (free 27768032)
26/08/31 00:26:26 INFO BlockManagerInfo: Removed broadcast_3_piece0 on quickstart.cloudera:48388 in memory
26/08/31 00:26:26 INFO BlockManagerMaster: Updated info of block broadcast_3_piece0
26/08/31 00:26:26 INFO BlockManager: Removing block broadcast_3
26/08/31 00:26:26 INFO MemoryStore: Block broadcast_3 of size 3752 dropped from memory (free 27768032)
26/08/31 00:26:26 INFO BlockManagerInfo: Removed broadcast_3_piece0 on quickstart.cloudera:38801 in memory
26/08/31 00:26:26 INFO ContextCleaner: Cleaned broadcast 3
26/08/31 00:26:26 INFO deprecation: mapred.tip.id is deprecated. Instead, use mapreduce.task.id
26/08/31 00:26:26 INFO deprecation: mapred.task.id is deprecated. Instead, use mapreduce.task.attemptid
26/08/31 00:26:26 INFO deprecation: mapred.task.is.map is deprecated. Instead, use mapreduce.task.ismap
26/08/31 00:26:26 INFO deprecation: mapred.task.partition is deprecated. Instead, use mapreduce.task.partition
26/08/31 00:26:26 INFO deprecation: mapred.job.id is deprecated. Instead, use mapreduce.job.id
26/08/31 00:26:26 INFO FileOutputCommitter: File Output Committer Algorithm version is 1
26/08/31 00:26:27 INFO SparkContext: Starting job: saveAsTextFile at <console>:30
26/08/31 00:26:27 INFO DAGScheduler: Got job 2 (saveAsTextFile at <console>:30) with 2 output partitions
26/08/31 00:26:27 INFO DAGScheduler: Final stage: Stage 4(saveAsTextFile at <console>:30)
26/08/31 00:26:27 INFO DAGScheduler: Parents of final stage: List(Stage 3)
26/08/31 00:26:27 INFO DAGScheduler: Missing parents: List()
26/08/31 00:26:27 INFO DAGScheduler: Submitting Stage 4 (MapPartitionsRDD[7] at saveAsTextFile at <console>:30)
26/08/31 00:26:27 INFO MemoryStore: ensureFreeSpace(137608) called with curMem=615016, maxMem=27830256
26/08/31 00:26:27 INFO MemoryStore: Block broadcast_5 stored as values in memory (estimated size 134.0 MB)
26/08/31 00:26:27 INFO MemoryStore: ensureFreeSpace(47774) called with curMem=752624, maxMem=27830256
26/08/31 00:26:27 INFO MemoryStore: Block broadcast_5_piece0 stored as bytes in memory (estimated size 46.0 MB)
26/08/31 00:26:27 INFO BlockManagerInfo: Added broadcast_5_piece0 in memory on quickstart.cloudera:48388
26/08/31 00:26:27 INFO BlockManagerMaster: Updated info of block broadcast_5_piece0
26/08/31 00:26:27 INFO SparkContext: Created broadcast 5 from broadcast at DAGScheduler.scala:839

scala> :quit
Stopping spark context.
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/metrics/json,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/stages/stage/kill,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/static,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/executors/threadDump/json,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/executors/threadDump,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/executors/json,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/executors,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/environment/json,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/environment,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/logs,null}
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/output_wordcount
Found 3 items
-rw-r--r-- 1 cloudera cloudera 0 2026-08-31 00:26 /user/cloudera/output_wordcount/_SUCCESS
-rw-r--r-- 1 cloudera cloudera 36 2026-08-31 00:26 /user/cloudera/output_wordcount/part-00000
-rw-r--r-- 1 cloudera cloudera 46 2026-08-31 00:26 /user/cloudera/output_wordcount/part-00001
[cloudera@quickstart ~]$ hdfs dfs -cat '/user/cloudera/output_wordcount/part-*'
(big,1)
(data,1)
(hello,2)
(easy,1)
(spark,2)
(hadoop,1)
(processing,1)
(makes,1)
[cloudera@quickstart ~]$ █

```

RESULT: Thus, the Word Count program was successfully implemented using Apache Spark RDDs. The input file was read from HDFS, split into words, converted into key-value pairs, reduced to obtain word frequencies, displayed successfully, and the output was stored back into HDFS.

EXP NO: 12
DATE:

WORKING WITH HBASE IN CLOUDERA MANAGER

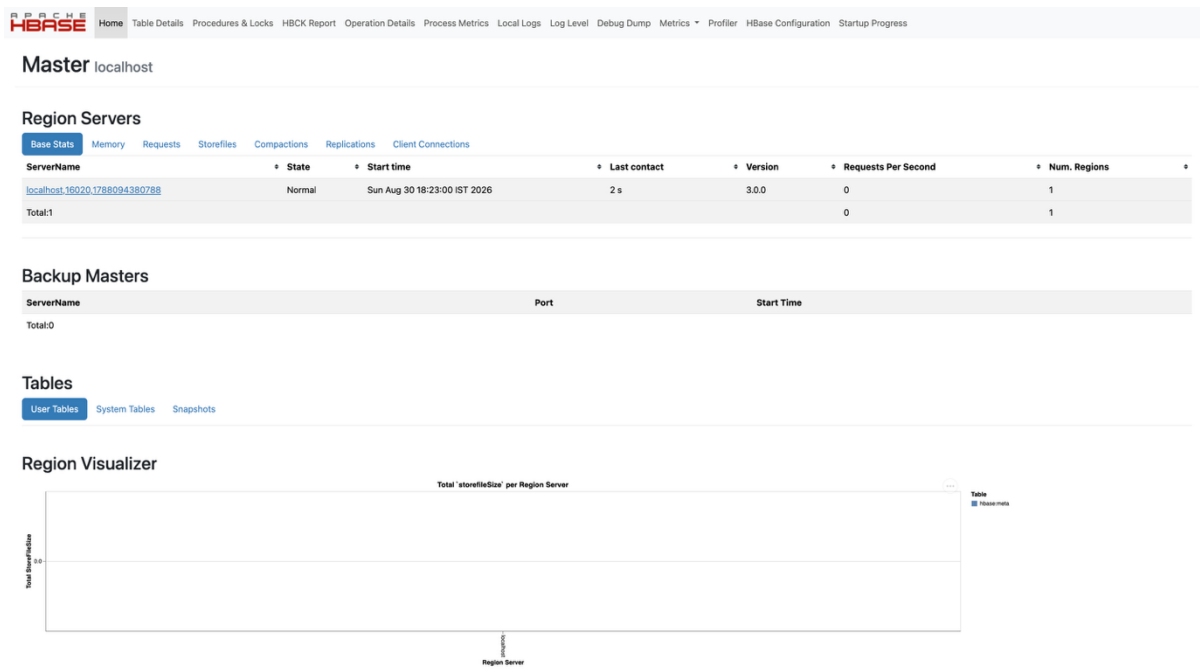
AIM: To explore the features of HBase using Cloudera Manager and perform basic operations such as creating a table, inserting data, and retrieving data.

QUESTIONS:

Q1: Explore HBase in Cloudera Manager

1. Start the Cloudera environment.
2. Open Cloudera Manager.
3. Select the cluster.
4. Click the HBase service.
5. Observe whether HBase is running.
6. Open the Charts section.
7. Observe HBase metrics such as:
 - Requests per second
 - Reads and writes
 - Compactions
 - Other available performance indicators
8. Open the Configuration section.
9. Search for:
 - heap
 - hbase.rootdir
10. Record the configured heap size and HDFS storage path.

OUTPUT:



APACHE

HBASE

Home

Table Details

Procedures & Locks

HBase Report

Operation Details

Process Metrics

Local Logs

Log Level

Debug Dump

Metrics ▾

Profiler

HBase Configuration

Startup Progress

Peers

Peer Id

Cluster Key

Endpoint

State

IsSerial

Remote WAL

Sync Replication State

Bandwidth

ReplicateAll

Namespaces

Exclude Namespaces

Table Cfs

Exclude Table Cfs

Total: 0

Tasks

Show All Monitored Tasks

Show non-RPC Tasks

Show All RPC Handler Tasks

Show Active RPC Calls

Show Client Operations

[View as JSON](#)

Start Time

Description

State

Status

Completion Time

Sun Aug 30 18:23:01 IST 2026

Closing region 1595e783b53d99cd5eeef43b6debb2682 as it is being closed

COMPLETE (since 33sec ago)

Closed (since 33sec ago)

Sun Aug 30 18:23:01 IST 2026 (since 33sec ago)

Sun Aug 30 18:23:01 IST 2026

Initializing region master:store,,11595e783b53d99cd5eeef43b6debb2682.

COMPLETE (since 32sec ago)

Region opened successfully (since 32sec ago)

Sun Aug 30 18:23:01 IST 2026 (since 32sec ago)

Sun Aug 30 18:23:01 IST 2026

Initializing region hbase:meta,,11588230740

COMPLETE (since 32sec ago)

Region opened successfully (since 32sec ago)

Sun Aug 30 18:23:01 IST 2026 (since 32sec ago)

Sun Aug 30 18:23:01 IST 2026

Closing region 1588230740 as it is being closed

COMPLETE (since 32sec ago)

Closed (since 32sec ago)

Sun Aug 30 18:23:01 IST 2026 (since 32sec ago)

Sun Aug 30 18:23:05 IST 2026

Initializing region hbase:meta,,11588230740

COMPLETE (since 29sec ago)

Region opened successfully (since 29sec ago)

Sun Aug 30 18:23:05 IST 2026 (since 29sec ago)

Sun Aug 30 18:23:00 IST 2026

Master startup

COMPLETE (since 24sec ago)

Initialization successful (since 24sec ago)

Sun Aug 30 18:23:09 IST 2026 (since 24sec ago)

Sun Aug 30 18:23:09 IST 2026

Progress after master initialized

COMPLETE (since 24sec ago)

Progress after master initialized complete (since 24sec ago)

Sun Aug 30 18:23:09 IST 2026 (since 24sec ago)

Software Attributes

Attribute Name

Value

Description

JVM Version

Eclipse Adoptium 17.0.15-17.0.15+6

JVM vendor and version

HBase Version

3.0.0, revision=d418afa6973bf422231c1d233410d8c18ac7ec

HBase version and revision

HBase Compiled

Fri Jul 17 15:43:49 UTC 2026, work

When HBase version was compiled and by whom

HBase Source Checksum

63f026b7bcc4d89a7d710cb962270650549bffdcc78e6cfc6a206bd5e7e919be092d5e359ed42798656aebd47b1317bc4a871e584f1c36d8f64c33ed71e4b

HBase source SHA512 checksum

Hadoop Version

3.4.3, revision=9d50c6884666e794e45102260a4017bb31802e1b

Hadoop version and revision

Hadoop Compiled

2026-02-12T15:31Z, stevel

When Hadoop version was compiled and by whom

Hadoop Source Checksum

2331238c4c2929e66645316a32a8613

Hadoop source MD5 checksum

ZooKeeper Client Version

3.8.6, revision=6df26081269769c160c8c3a24929c60c91cd19c3

ZooKeeper client version and revision hash

ZooKeeper Client Compiled

2026-01-28 20:28 UTC

When ZooKeeper client version was compiled

Q2: Create and Manage a Table in HBase Shell

1. Open a terminal in the Cloudera VM.
2. Start HBase shell: hbase shell
3. Create a table named student with column family info: create 'student', 'info'
4. Insert sample records:


```
put 'student', '101', 'info:name', 'Anu'
put 'student', '101', 'info:dept', 'IT'
put 'student', '102', 'info:name', 'Bala'
put 'student', '102', 'info:dept', 'CSE'
```
5. Retrieve all records from the table: scan 'student'
6. Retrieve the record corresponding to row key 101: get 'student', '101'

OUTPUT:

```
(base) john@ohns-MacBook-Air-2 ~ % hbase shell
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/homebrew/Cellar/hbase/3.0.0/libexec/lib/client-facing-thirdparty/log4j-slf4j-impl-2.25.4.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/homebrew/Cellar/hadoop/3.5.0/libexec/share/hadoop/common/lib/slf4j-reloadj-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: https://hbase.apache.org/docs/shell
Version 3.0.0, rda418afa6973bf4222231cd233410d8c18ac7ec, Fri Jul 17 15:43:49 UTC 2026
Took 0.0004 seconds
hbase:001:0> status
2026-08-30T18:26:26.690 INFO [main] util.log: Logging initialized @865ms to org.eclipse.jetty.util.log.Slf4jLog
1 active master, 0 backup masters,
    1 servers,
    0 decommissioned,
    0 dead,
    1.0000 average load
Took 0.4939 seconds
hbase:002:0> list
TABLE
0 row(s)
Took 0.0153 seconds
=> []
hbase:003:0> create 'student', 'info'
Created table student
Took 0.6996 seconds
=> Hbase::Table - student
hbase:004:0> put 'student', '101', 'info:name', 'Anu'
hbase:005:0> put 'student', '101', 'info:dept', 'IT'
Took 0.0832 seconds
hbase:006:0> put 'student', '102', 'info:name', 'Bala'
hbase:007:0> put 'student', '102', 'info:dept', 'CSE'
Took 0.0067 seconds
Took 0.0023 seconds
hbase:008:0> scan 'student'
ROW COLUMN+CELL
101 column=info:dept, timestamp=2026-08-30T18:27:11.207, value=IT
101 column=info:name, timestamp=2026-08-30T18:27:11.199, value=Anu
102 column=info:dept, timestamp=2026-08-30T18:27:18.722, value=CSE
102 column=info:name, timestamp=2026-08-30T18:27:18.716, value=Bala
2 row(s)
Took 0.0591 seconds
hbase:009:0> get 'student', '101'
COLUMN CELL
info:dept timestamp=2026-08-30T18:27:11.207, value=IT
info:name timestamp=2026-08-30T18:27:11.199, value=Anu
1 row(s)
Took 0.0172 seconds
hbase:010:0> exit
(base) john@ohns-MacBook-Air-2 ~ % hbase org.apache.hadoop.hbase.util.HBaseConfTool hbase.rootdir
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/homebrew/Cellar/hbase/3.0.0/libexec/lib/client-facing-thirdparty/log4j-slf4j-impl-2.25.4.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/homebrew/Cellar/hadoop/3.5.0/libexec/share/hadoop/common/lib/slf4j-reloadj-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
file:/opt/homebrew/var/hbase
(base) john@ohns-MacBook-Air-2 ~ %
```

RESULT: Thus, HBase was successfully explored using Cloudera Manager, including its service status, performance metrics, and configuration parameters. A student table was created using HBase Shell, sample records were inserted successfully, and the stored records were retrieved using scan and get commands.

EXP NO: 13**DATA VISUALIZATION USING TABLEAU****DATE:**

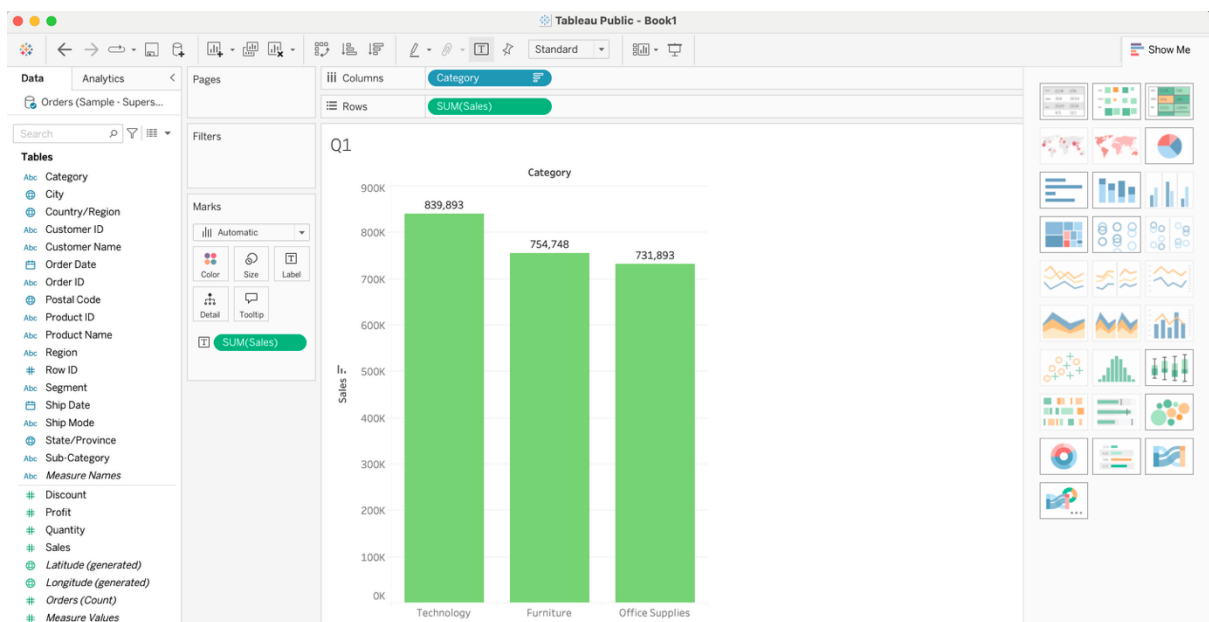
AIM: To analyse and visualize the Sample Superstore dataset using Tableau by creating different charts, performing sales and profit analysis, applying filters and time hierarchy drill-down, and designing an interactive business intelligence dashboard.

Q1. Sales Across Product Categories

QUESTION: Create a bar chart to compare sales across different product categories.

PROCEDURE:

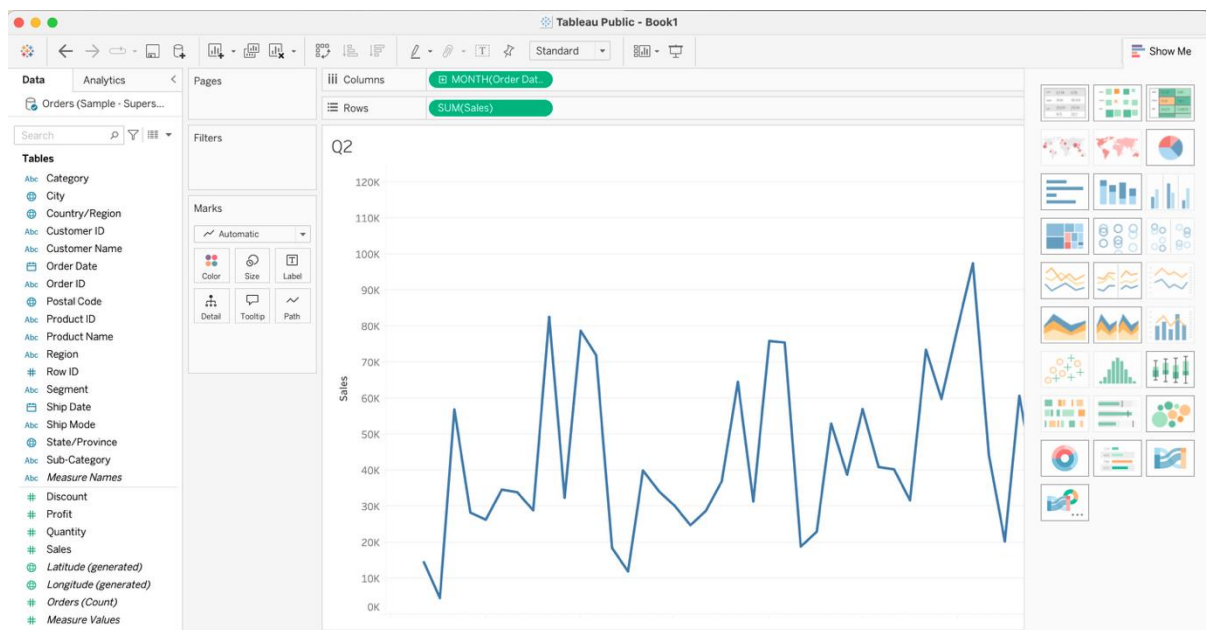
1. Open Tableau Public.
2. Connect to the Sample – Superstore dataset.
3. Create a new worksheet and rename it Q1.
4. Drag Category to Columns.
5. Drag Sales to Rows.
6. Select Bar Chart from Show Me.
7. Drag Sales to Label in the Marks card to display the sales values.
8. Compare the total sales of Technology, Furniture, and Office Supplies.

OUTPUT:**Q2. Monthly Sales Trend**

QUESTION: Visualize how sales change over time using a line chart and identify the month with the highest sales.

PROCEDURE:

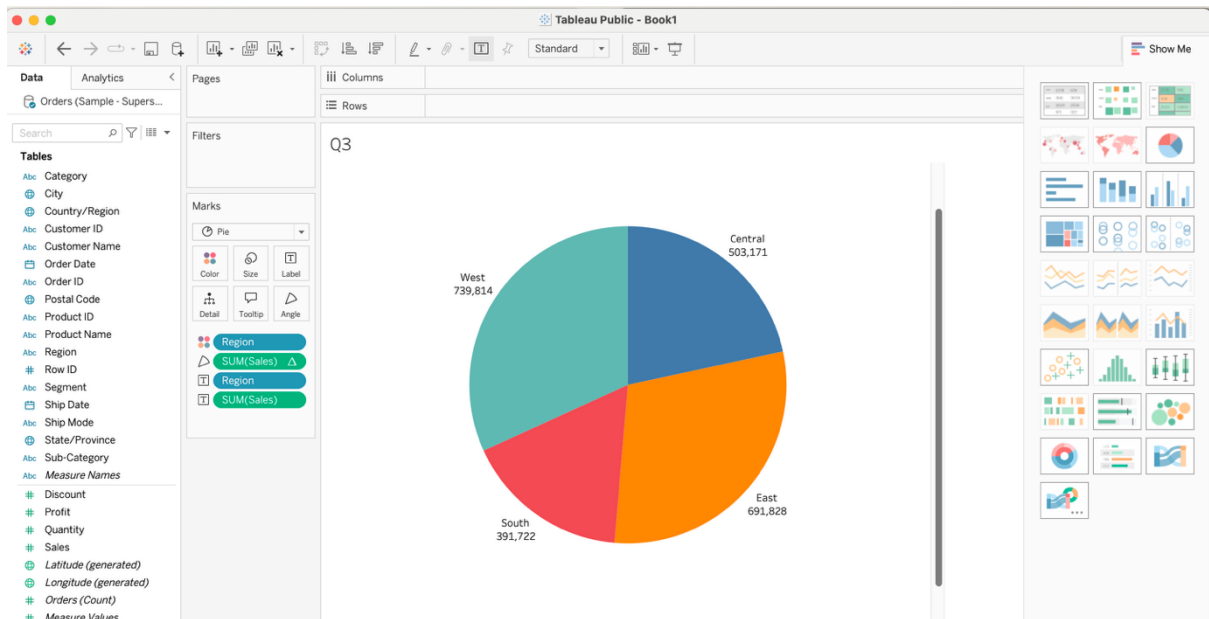
1. Create a new worksheet named Q2.
2. Drag Order Date to Columns.
3. Change the date level to Month.
4. Drag Sales to Rows.
5. Select Line Chart from Show Me.
6. Observe the variation in sales over different months.
7. Locate the highest point on the line chart to identify the month having maximum sales.

OUTPUT:**Q3. Regional Contribution to Sales**

QUESTION: Show the contribution of different regions to total sales using a pie chart.

PROCEDURE:

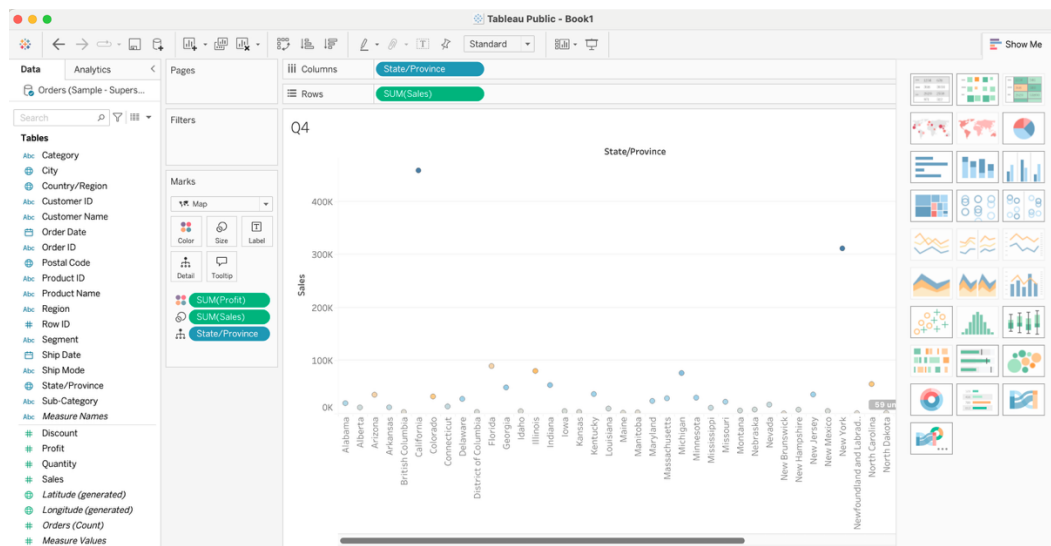
1. Create a new worksheet named Q3.
2. Select Pie as the Marks type.
3. Drag Region to Colour.
4. Drag Sales to Angle.
5. Drag Region and Sales to Label.
6. Adjust the size of the pie chart for clear visualization.
7. Compare the contribution of Central, East, South, and West regions.

OUTPUT:**Q4. Geographical Sales and Profit Analysis**

QUESTION: Visualize geographical differences in sales and profit using a geographic visualization.

PROCEDURE:

1. Create a new worksheet named Q4.
2. Use State/Province as the geographical dimension.
3. Drag Sales and Profit to the appropriate Marks properties.
4. Use Sales to represent the magnitude/size of the marks.
5. Use Profit to distinguish the profitability of different states.
6. Observe variations in sales and profit across different states.

OUTPUT:

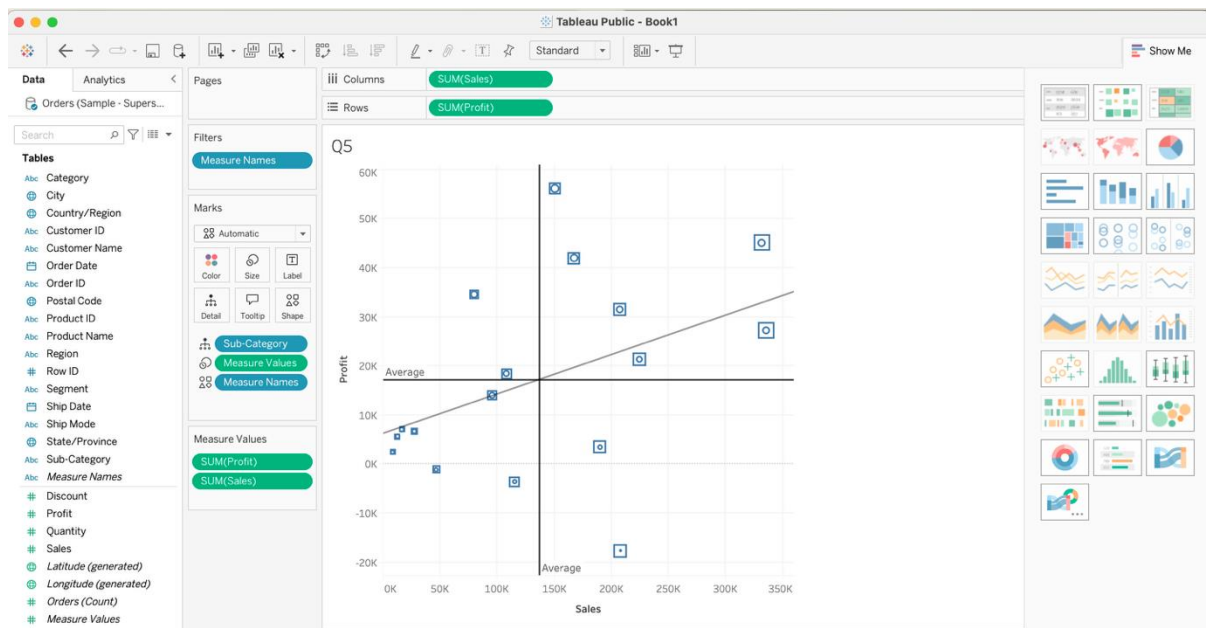
Q5. Relationship Between Sales and Profit

QUESTION: Determine the relationship between Sales and Profit using a scatter plot. Identify sub-categories having high/low sales and profit and add a trend line.

PROCEDURE:

1. Create a worksheet named Q5.
2. Drag Sales to Columns.
3. Drag Profit to Rows.
4. Drag Sub-Category to Detail in the Marks card.
5. Create the resulting Scatter Plot.
6. Add average reference lines for Sales and Profit.
7. The reference lines divide the scatter plot into four regions:
 - High Sales – High Profit
 - High Sales – Low Profit
 - Low Sales – High Profit
 - Low Sales – Low Profit
8. Open the Analytics pane.
9. Add a Trend Line to study the overall relationship between Sales and Profit.

OUTPUT:



Q6. Interactive Business Intelligence Dashboard

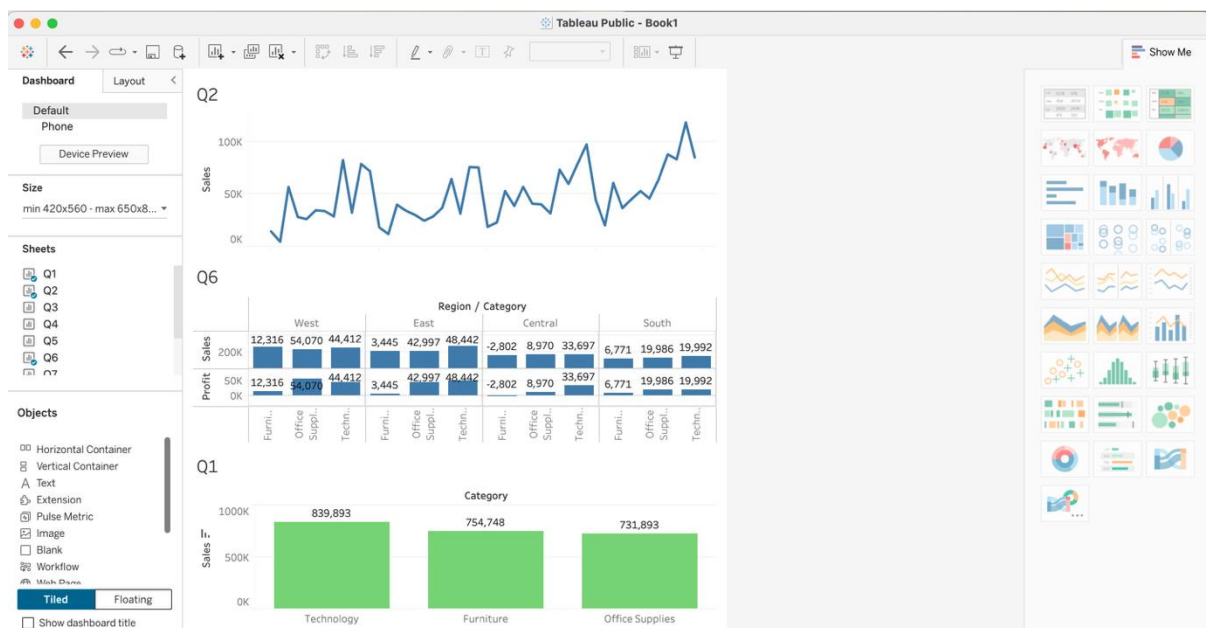
QUESTION: Combine multiple visualizations into a single dashboard and design a complete interactive business intelligence dashboard.

PROCEDURE:

1. Click New Dashboard in Tableau.
2. Select an appropriate dashboard size.

3. Drag the required worksheets from the Sheets panel onto the dashboard.
4. Arrange the visualizations clearly using tiled containers.
5. Resize each visualization for proper readability.
6. Include multiple analyses such as:
 - Category/Region-wise Sales
 - Monthly Sales Trend
 - Profit analysis
 - Sales comparison
7. Arrange them into a single dashboard for consolidated analysis.

OUTPUT:

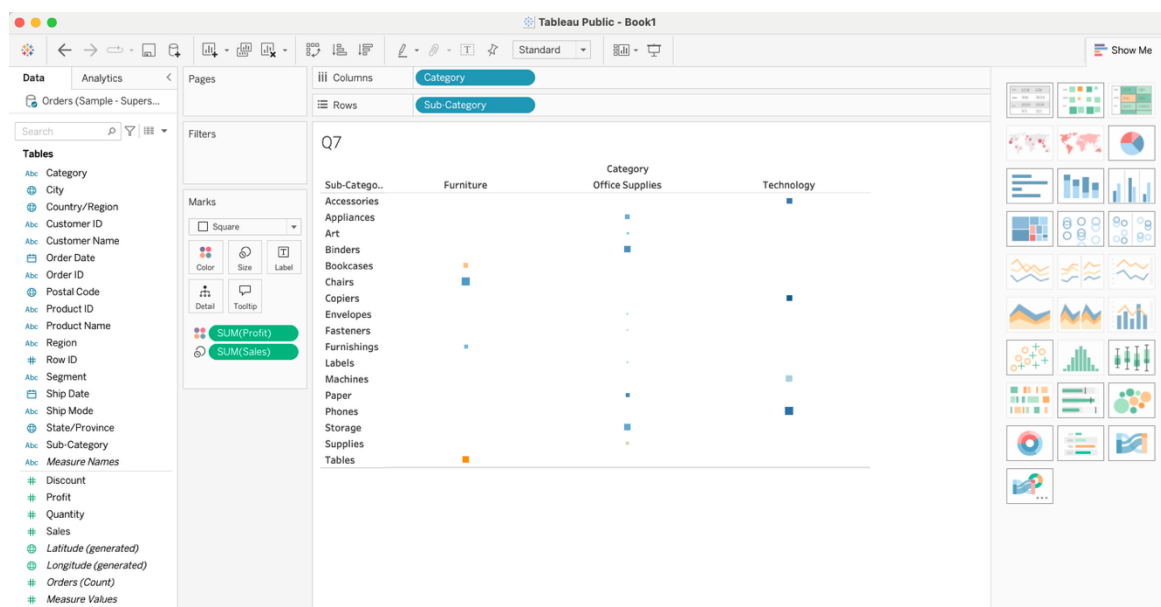


Q7. Profitable and Loss-Making Product Combinations

QUESTION: Identify profitable and loss-making product combinations using a heat map.

PROCEDURE:

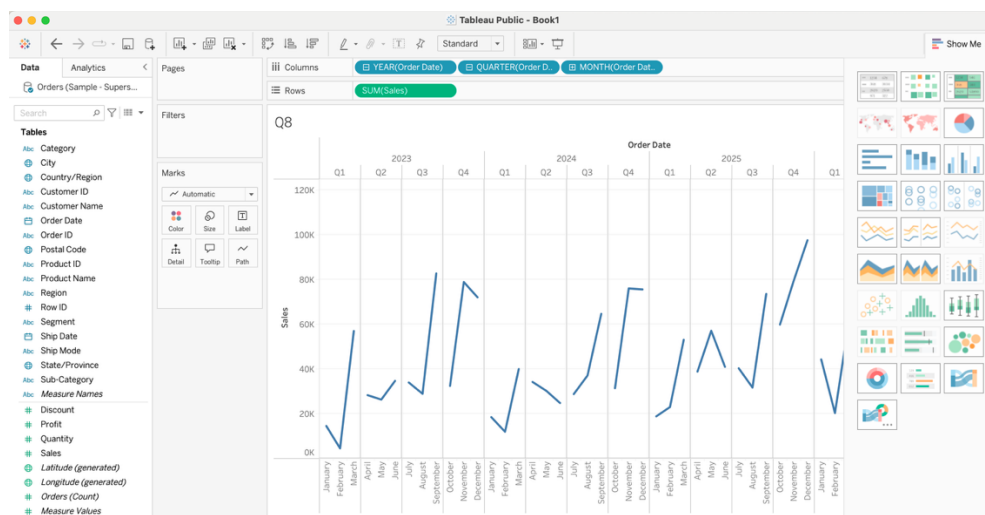
1. Create a worksheet named Q7.
2. Drag Category to Columns.
3. Drag Sub-Category to Rows.
4. Change the Marks type to Square.
5. Drag Profit to Colour.
6. Drag Sales to Size.
7. Observe the different Category–Sub-Category combinations.
8. Use the colour variation to distinguish profitable and loss-making combinations and mark size to compare sales.

OUTPUT:**Q8. Time Hierarchy Drill-Down Analysis**

QUESTION: Analyse sales at different levels of the time hierarchy using drill-down analysis for Year, Quarter and Month.

PROCEDURE:

1. Create a worksheet named Q8.
2. Drag Order Date to Columns.
3. Drag Sales to Rows.
4. Expand the Order Date hierarchy from: Year → Quarter → Month
5. Keep YEAR(Order Date), QUARTER(Order Date) and MONTH(Order Date) visible.
6. Use a line chart to visualize Sales.
7. Observe sales variations at yearly, quarterly, and monthly levels.

OUTPUT:

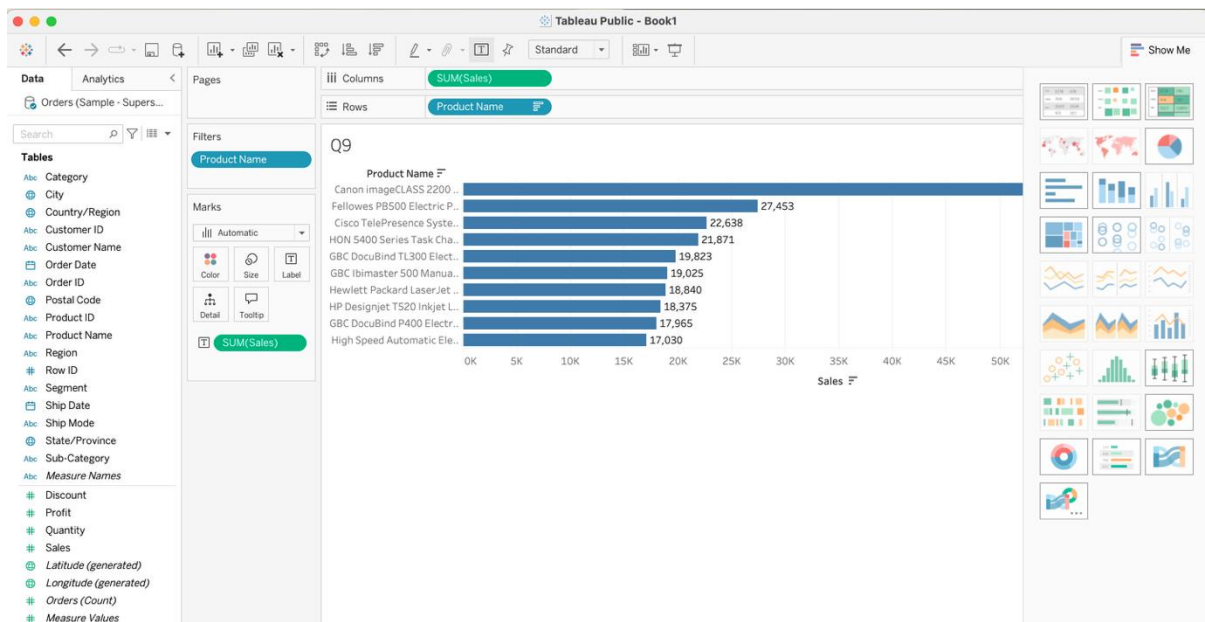
Q9. Top-Performing Products

QUESTION: Identify the top-performing products using a filter and draw a bar chart.

PROCEDURE:

1. Create a worksheet named Q9.
2. Drag Product Name to Rows.
3. Drag Sales to Columns.
4. Select a horizontal Bar Chart.
5. Drag Product Name to the Filters shelf.
6. Apply a Top N filter based on SUM(Sales).
7. Sort the products in descending order of Sales.
8. Drag Sales to Label to display values.
9. Analyse the products generating the highest sales.

OUTPUT:



RESULT: Thus, the Sample Superstore dataset was successfully analysed and visualized using Tableau. Various visualizations such as bar chart, line chart, pie chart, geographical visualization, scatter plot, heat map, time hierarchy drill-down, and top-product analysis were created. Sales and profit patterns across categories, regions, states, time periods, and products were analysed, and multiple visualizations were combined to form an interactive business intelligence dashboard.

EXP NO: 14
DATE:

**MINI PROJECT: RETAIL SALES ANALYTICS USING
HADOOP HDFS, APACHE HIVE AND TABLEAU**

AIM: To develop a Retail Sales Analytics system using Hadoop HDFS and Apache Hive for storing and analysing retail transaction data, and to visualize key sales trends and insights using Tableau.

PROBLEM STATEMENT

Retail organizations generate large volumes of transactional data containing information about products, categories, regions, sales, quantity and profit. Analysing such data manually becomes difficult as the volume of data increases. The objective of this mini-project is to store retail sales data using Hadoop Distributed File System (HDFS), analyse the data using Apache Hive, and visualize important sales patterns using Tableau. The analysis helps identify high-performing categories, regions, products and monthly sales trends.

OBJECTIVES:

1. To store retail sales transaction data using Hadoop HDFS.
2. To create a structured Hive table for querying the retail dataset.
3. To analyse category-wise and region-wise sales using HiveQL.
4. To identify profitable categories and top-selling products.
5. To analyse monthly sales trends and visualize the results using Tableau.

DATASET DESCRIPTION

A synthetic retail sales dataset containing 50 transaction records was used for the experiment. The dataset contains the following attributes:

Attribute	Description
Order ID	Unique identifier for each transaction
Order Date	Date of the transaction
Product	Name of the product
Category	Product category
Region	Sales region
Quantity	Number of units purchased
Price	Price of the product
Sales	Total sales amount
Profit	Profit obtained from the transaction

The dataset contains transactions belonging to three major categories: Electronics, Furniture and Stationery, distributed across the North, South, East and West regions.

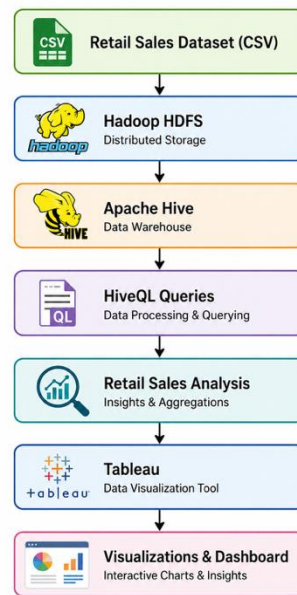
BIG DATA TECHNOLOGIES USED

1. Hadoop HDFS: Hadoop Distributed File System was used to store the retail sales dataset in a distributed storage environment.

2. Apache Hive: Apache Hive was used to create a structured table over the HDFS dataset and perform analytical operations using HiveQL.

Visualization Tool – Tableau: Tableau was additionally used to visualize the analytical results in the form of bar charts, pie charts, line charts and a dashboard.

ARCHITECTURE / WORKFLOW



PROCEDURE:

1. A retail sales dataset containing 50 transaction records was prepared in CSV format.
2. Hadoop HDFS services were started and verified.
3. A directory named /retail was created in HDFS.
4. The retail sales CSV file was uploaded into HDFS.
5. Apache Hive was initialized and a database named retaildb was created.
6. An external Hive table named retail_sales was created over the HDFS dataset.
7. HiveQL queries were executed to analyse sales and profit across categories, regions, products and months.
8. The retail dataset was connected to Tableau.
9. Category-wise sales, region-wise sales and monthly sales trends were visualized.
10. A Retail Sales Analytics Dashboard was created using Tableau.

HDFS OPERATIONS

1. Verify Hadoop Services

```
jps
```

2. Create HDFS Directory

```
hdfs dfs -mkdir -p /retail
```

3. Upload Retail Dataset to HDFS

```
hdfs dfs -put -f retail_sales.csv /retail/retail_sales.csv
```

4. Verify the File

```
hdfs dfs -ls /retail
```

5. Display Dataset Contents

```
hdfs dfs -cat /retail/retail_sales.csv | head
```

OUTPUT:

```

Last login: Sun Aug 30 17:53:17 from ::1
(base) john@ohns-MacBook-Air-2 ~ % >...
1029, 2026-04-22, Tablet, Electronics, South, 2, 25000, 50000, 6200
1030, 2026-04-25, Marker, Stationery, North, 15, 80, 1200, 250
1031, 2026-05-02, Laptop, Electronics, East, 2, 45000, 90000, 9000
1032, 2026-05-06, Sofa, Furniture, South, 1, 30000, 30000, 4800
1033, 2026-05-09, Keyboard, Electronics, West, 5, 2000, 10000, 1500
1034, 2026-05-12, Pen, Stationery, South, 30, 50, 1500, 350
1035, 2026-05-15, Phone, Electronics, North, 3, 20000, 60000, 6500
1036, 2026-05-19, Bookshelf, Furniture, East, 2, 10000, 20000, 3100
1037, 2026-05-22, Muse, Electronics, South, 8, 1000, 8000, 1100
1038, 2026-06-01, Table, Furniture, West, 4, 6000, 24000, 3500
1039, 2026-06-04, Printer, Electronics, North, 2, 12000, 24000, 2900
1040, 2026-06-08, Notebook, Stationery, East, 25, 100, 2500, 500
1041, 2026-06-11, Monitor, Electronics, South, 4, 15000, 60000, 7000
1042, 2026-06-15, Chair, Furniture, West, 7, 2500, 17500, 2700
1043, 2026-06-18, Headphones, Electronics, North, 6, 3000, 18000, 2600
1044, 2026-06-21, Desk, Furniture, South, 2, 8000, 16000, 2100
1045, 2026-06-24, Tablet, Electronics, East, 3, 25000, 75000, 8000
1046, 2026-07-02, Laptop, Electronics, North, 2, 45000, 90000, 9500
1047, 2026-07-06, Sofa, Furniture, West, 1, 30000, 30000, 4700
1048, 2026-07-10, Phone, Electronics, East, 5, 20000, 100000, 10500
1049, 2026-07-15, Notebook, Stationery, South, 18, 100, 1800, 380
1050, 2026-07-20, Chair, Furniture, North, 5, 2500, 12500, 1900
EOF
(base) john@ohns-MacBook-Air-2 ~ % head retail_sales.csv
1001, 2026-01-05, Laptop, Electronics, South, 2, 45000, 90000, 8000
1002, 2026-01-06, Chair, Furniture, North, 4, 2500, 10000, 1500
1003, 2026-01-07, Phone, Electronics, West, 3, 20000, 60000, 6000
1004, 2026-01-10, Table, Furniture, South, 2, 6000, 12000, 1800
1005, 2026-01-12, Headphones, Electronics, East, 5, 3000, 15000, 2500
1006, 2026-01-15, Notebook, Stationery, North, 10, 100, 1000, 200
1007, 2026-01-18, Printer, Electronics, South, 2, 12000, 24000, 3000
1008, 2026-01-20, Desk, Furniture, West, 2, 8000, 16000, 2200
1009, 2026-02-02, Pen, Stationery, East, 20, 50, 1000, 250
1010, 2026-02-05, Monitor, Electronics, North, 3, 15000, 45000, 5500
(base) john@ohns-MacBook-Air-2 ~ % wc -l retail_sales.csv
50 retail_sales.csv
(base) john@ohns-MacBook-Air-2 ~ % █

(base) john@ohns-MacBook-Air-2 ~ % jps
10193 DataNode
10326 SecondaryNameNode
12118 HMaster
14413 Jps
10093 NameNode
(base) john@ohns-MacBook-Air-2 ~ % hdfs dfs -mkdir -p /retail
2026-08-30 20:50:39,434 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
(base) john@ohns-MacBook-Air-2 ~ % hdfs dfs -put -f retail_sales.csv /retail/retail_sales.csv
2026-08-30 20:50:50,992 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
(base) john@ohns-MacBook-Air-2 ~ % hdfs dfs -ls /retail
2026-08-30 20:51:03,592 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r-- 1 john supergroup 2882 2026-08-30 20:50 /retail/retail_sales.csv
(base) john@ohns-MacBook-Air-2 ~ % hdfs dfs -cat /retail/retail_sales.csv | head
2026-08-30 20:51:13,736 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
1001, 2026-01-05, Laptop, Electronics, South, 2, 45000, 90000, 8000
1002, 2026-01-06, Chair, Furniture, North, 4, 2500, 10000, 1500
1003, 2026-01-07, Phone, Electronics, West, 3, 20000, 60000, 6000
1004, 2026-01-10, Table, Furniture, South, 2, 6000, 12000, 1800
1005, 2026-01-12, Headphones, Electronics, East, 5, 3000, 15000, 2500
1006, 2026-01-15, Notebook, Stationery, North, 10, 100, 1000, 200
1007, 2026-01-18, Printer, Electronics, South, 2, 12000, 24000, 3000
1008, 2026-01-20, Desk, Furniture, West, 2, 8000, 16000, 2200
1009, 2026-02-02, Pen, Stationery, East, 20, 50, 1000, 250
1010, 2026-02-05, Monitor, Electronics, North, 3, 15000, 45000, 5500
(base) john@ohns-MacBook-Air-2 ~ % █

```

HIVE OPERATIONS

1. Create and Select Database

```
CREATE DATABASE IF NOT EXISTS retaildb;
USE retaildb;
```

2. Create External Hive Table

```
CREATE EXTERNAL TABLE IF NOT EXISTS retail_sales (
  order_id INT,
  order_date STRING,
  product STRING,
  category STRING,
  region STRING,
  quantity INT,
  price DOUBLE,
  sales DOUBLE,
  profit DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/retail';
```

3. Verify Table

```
SHOW TABLES;
```

4. Display Sample Records

```
SELECT * FROM retail_sales LIMIT 10;
```

OUTPUT:

```
Connected to: Apache Hive (version 4.2.1)
Driver: Hive JDBC (version 4.2.1)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 4.2.1 by Apache Hive
0: jdbc:hive2://> CREATE DATABASE IF NOT EXISTS retaildb;
> USE retaildb;
26/08/30 21:01:36 [HiveServer2-Background-Pool: Thread-75]
use external tables to use to managed tablespace.
No rows affected (1.544 seconds)
No rows affected (0.019 seconds)
0: jdbc:hive2://> █
```

```

0: jdbc:hive2://> CREATE EXTERNAL TABLE IF NOT EXISTS retail_sales (
>   order_id INT,
>   order_date STRING,
>   product STRING,
>   category STRING,
>   region STRING,
>   quantity INT,
>   price DOUBLE,
>   sales DOUBLE,
>   profit DOUBLE
> )
> ROW FORMAT DELIMITED
> FIELDS TERMINATED BY ','
> STORED AS TEXTFILE
> LOCATION '/retail';
No rows affected (0.377 seconds)
0: jdbc:hive2://> SHOW TABLES;
+-----+
| tab_name |
+-----+
| retail_sales |
+-----+
1 row selected (0.228 seconds)
0: jdbc:hive2://> SELECT * FROM retail_sales LIMIT 10;
26/08/30 21:02:55 [27c2ca31-202e-4a7e-830c-972c3ef9ca5b min]: WARN calcite.RelOptHiveTable: No Stats for retaildb@etail_sales, Columns: order_date, product, quantity, price, category, region, order_id, profit, sales
No Stats for retaildb@etail_sales, Columns: order_date, product, quantity, price, category, region, order_id, profit, sales
26/08/30 21:02:55 [27c2ca31-202e-4a7e-830c-972c3ef9ca5b min]: WARN optimizer.SimpleFetchOptimizer: Table retaildb@etail_sales is external table, falling back to filesystemscan
+-----+
| retail_sales.order_id | retail_sales.order_date | retail_sales.product | retail_sales.category | retail_sales.region | retail_sales.quantity | retail_sales.price | retail
| sales.sales | retail_sales.profit |
+-----+
+-----+
| 1001 | 2026-01-05 | Laptop | Electronics | South | 2 | 45000.0 | 90000
| 0 | 8000.0 |
| 1002 | 2026-01-06 | Chair | Furniture | North | 4 | 2500.0 | 10000
| 0 | 1500.0 |
| 1003 | 2026-01-07 | Phone | Electronics | West | 3 | 20000.0 | 60000
| 0 | 6000.0 |
| 1004 | 2026-01-10 | Table | Furniture | South | 2 | 6000.0 | 12000
| 0 | 1800.0 |
| 1005 | 2026-01-12 | Headphones | Electronics | East | 5 | 3000.0 | 15000
| 0 | 2500.0 |
| 1006 | 2026-01-15 | Notebook | Stationery | North | 10 | 100.0 | 1000
| 0 | 200.0 |
| 1007 | 2026-01-18 | Printer | Electronics | South | 2 | 12000.0 | 24000
| 0 | 3000.0 |
| 1008 | 2026-01-20 | Desk | Furniture | West | 2 | 8000.0 | 16000
| 0 | 2200.0 |
| 1009 | 2026-02-02 | Pen | Stationery | East | 20 | 50.0 | 1000
| 0 | 250.0 |
| 1010 | 2026-02-05 | Monitor | Electronics | North | 3 | 15000.0 | 45000
| 0 | 5500.0 |
+-----+
10 rows selected (1.357 seconds)
0: jdbc:hive2://>

```

HIVEQL ANALYSIS

1. Category-wise Sales Analysis

```

SELECT category, SUM(sales) AS total_sales
FROM retail_sales
GROUP BY category
ORDER BY total_sales DESC;

```

Output:

Category	Total Sales
Electronics	1,098,000
Furniture	307,500
Stationery	14,710

```

Job running in-process (local Hadoop)
26/08/30 21:07:05 [pool-18-thread-1]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
2026-08-30 21:07:06,072 Stage-2 map = 100% reduce = 100%
Ended Job = job_local872502104_0002
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 11528 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Stage-Stage-2: HDFS Read: 11528 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
+-----+
| category | total_sales |
+-----+
| Electronics | 1098000.0 |
| Furniture | 307500.0 |
| Stationery | 14710.0 |
+-----+
3 rows selected (4.747 seconds)

```

Inference: Electronics generated the highest total sales among all product categories.

2. Region-wise Sales Analysis

```
SELECT region, SUM(sales) AS total_sales
FROM retail_sales
GROUP BY region
ORDER BY total_sales DESC;
```

Output:

Region	Total Sales
South	411,300
East	385,460
North	312,950
West	310,500

```
2026-08-30 21:08:59,897 Stage-2 map = 100% reduce = 100%
Ended Job = job_local687765936_0004
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 17292 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Stage-Stage-2: HDFS Read: 17292 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
+-----+-----+
| region | total_sales |
+-----+-----+
| South  | 411300.0    |
| East   | 385460.0    |
| North  | 312950.0    |
| West   | 310500.0    |
+-----+-----+
4 rows selected (4.257 seconds)
0: jdbc:hive2://>
```

The actual Hive execution showed South as the highest-performing region, followed by East, North and West.

Inference: The South region recorded the highest overall sales.

3. Category-wise Profit Analysis

```
SELECT category, SUM(profit) AS total_profit
FROM retail_sales
GROUP BY category
ORDER BY total_profit DESC;
```

Output:

Category	Total Profit
Electronics	122,200
Furniture	46,000
Stationery	3,110

```

2026-08-30 21:09:57,757 Stage-2 map = 100%   reduce = 100%
Ended Job = job_local420439078_0006
MapReduce Jobs Launched:
Stage-Stage-1:   HDFS Read: 23056 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Stage-Stage-2:   HDFS Read: 23056 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
+-----+-----+
| category | total_profit |
+-----+-----+
| Electronics | 122200.0 |
| Furniture   | 46000.0 |
| Stationery  | 3110.0 |
+-----+-----+
3 rows selected (4.262 seconds)

```

These are the values obtained from the successful Hive execution.

Inference: Electronics was both the highest-selling and most profitable product category.

4. Top Five Products Based on Sales

```

SELECT product, SUM(sales) AS total_sales
FROM retail_sales
GROUP BY product
ORDER BY total_sales DESC
LIMIT 5;

```

Output:

Product	Total Sales
Laptop	315,000
Phone	300,000
Tablet	175,000
Monitor	135,000
Sofa	90,000

```

2026-08-30 21:10:43,108 Stage-2 map = 100%   reduce = 100%
Ended Job = job_local1124517802_0008
MapReduce Jobs Launched:
Stage-Stage-1:   HDFS Read: 28820 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Stage-Stage-2:   HDFS Read: 28820 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
+-----+-----+
| product | total_sales |
+-----+-----+
| Laptop  | 315000.0 |
| Phone   | 300000.0 |
| Tablet  | 175000.0 |
| Monitor | 135000.0 |
| Sofa    | 90000.0 |
+-----+-----+
5 rows selected (4.213 seconds)

```

The Hive result identified these five products in the same descending order.

Inference: Laptop was the highest-selling individual product, followed closely by Phone.

5. Monthly Sales Trend

```
SELECT substr(order_date,1,7) AS month,
       SUM(sales) AS total_sales
FROM retail_sales
GROUP BY substr(order_date,1,7)
ORDER BY month;
```

Output:

Month	Total Sales
January 2026	228,000
February 2026	148,000
March 2026	171,210
April 2026	182,200
May 2026	219,500
June 2026	237,000
July 2026	234,300

```
2026-08-30 21:12:18,071 Stage-2 map = 100%   reduce = 100%
Ended Job = job_local2092519048_0010
MapReduce Jobs Launched:
Stage-Stage-1:   HDFS Read: 34584 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Stage-Stage-2:   HDFS Read: 34584 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
+-----+-----+
| month | total_sales |
+-----+-----+
| 2026-01 | 228000.0 |
| 2026-02 | 148000.0 |
| 2026-03 | 171210.0 |
| 2026-04 | 182200.0 |
| 2026-05 | 219500.0 |
| 2026-06 | 237000.0 |
| 2026-07 | 234300.0 |
+-----+-----+
7 rows selected (4.304 seconds)
```

Inference: Sales reached their highest value in June 2026, while February recorded the lowest sales.

DATA VISUALIZATION USING TABLEAU

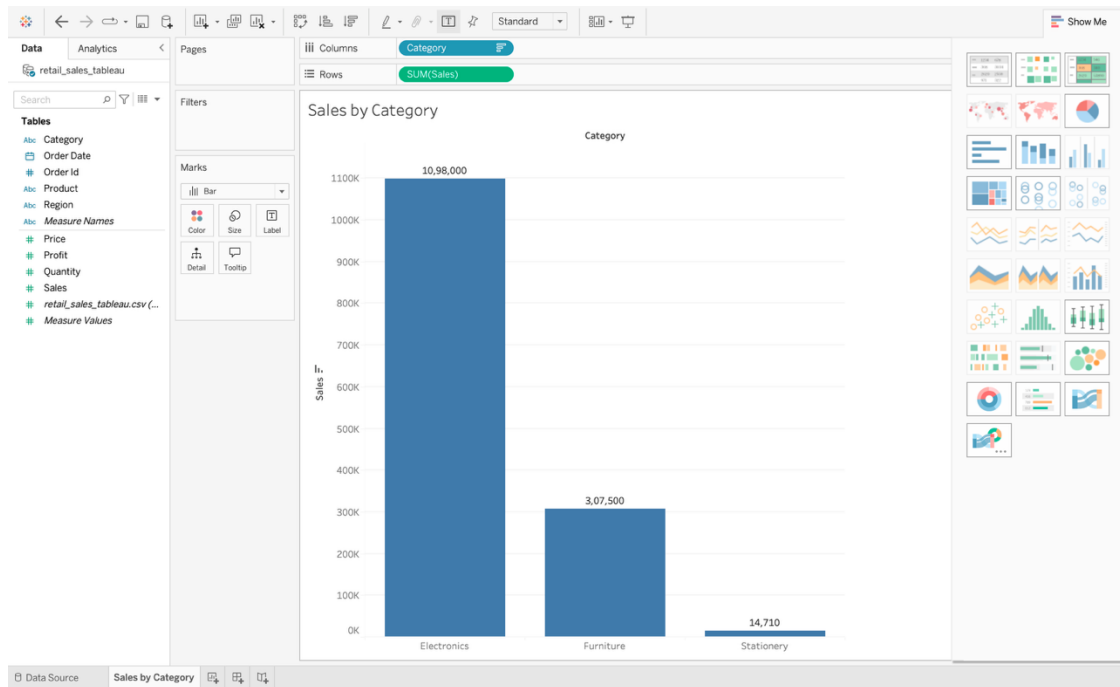
The retail sales dataset was imported into Tableau to visually represent the results obtained through Hive analysis.

1. Category-wise Retail Sales

A bar chart was created using:

- Category → Columns
- SUM(Sales) → Rows

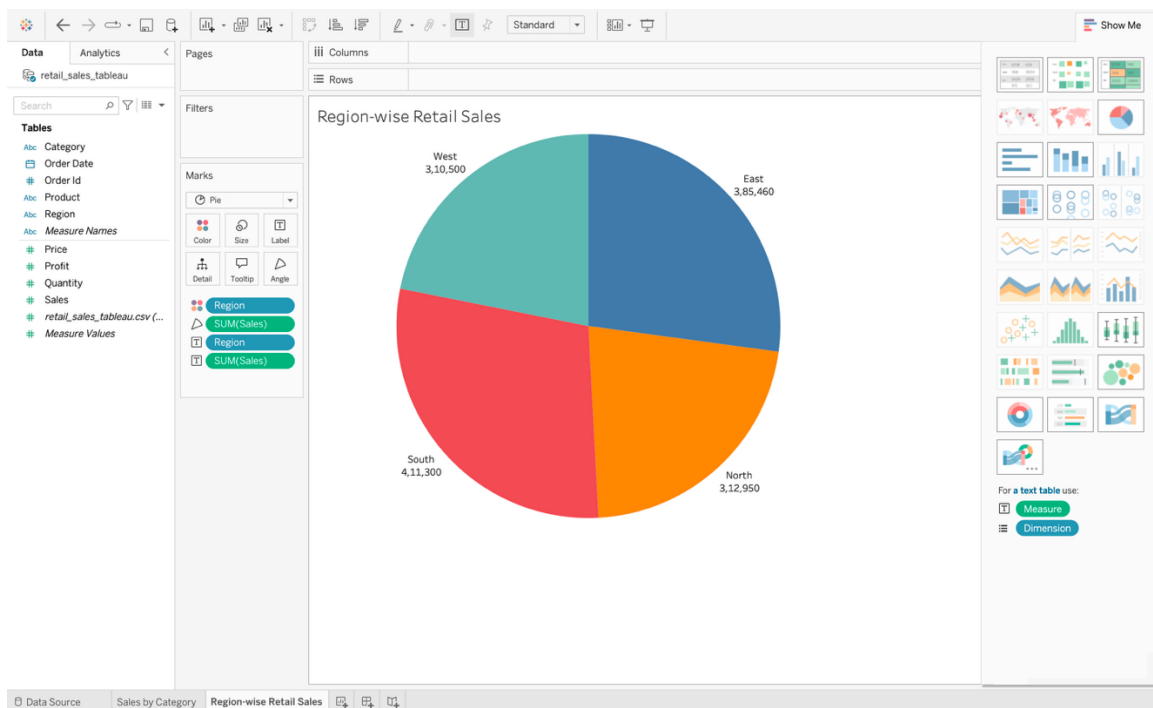
The visualization shows that Electronics contributed the largest portion of total sales.

OUTPUT:**2. Region-wise Retail Sales**

A pie chart was created using:

- Region → Color and Label
- SUM(Sales) → Angle and Label

The South region recorded the highest sales, followed by East, North and West.

OUTPUT:

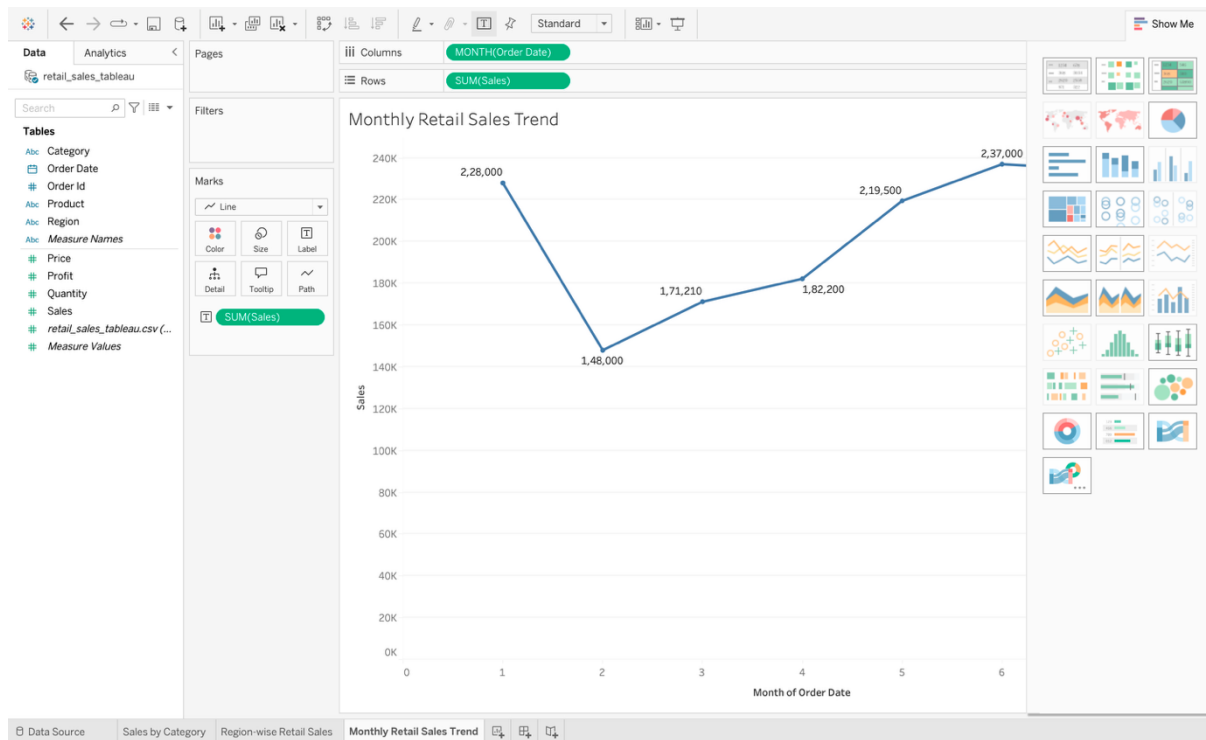
3. Monthly Retail Sales Trend

A line chart was created using:

- MONTH(Order Date) → Columns
- SUM(Sales) → Rows

The visualization shows a fall in sales during February followed by a generally increasing trend, reaching the maximum in June.

OUTPUT:



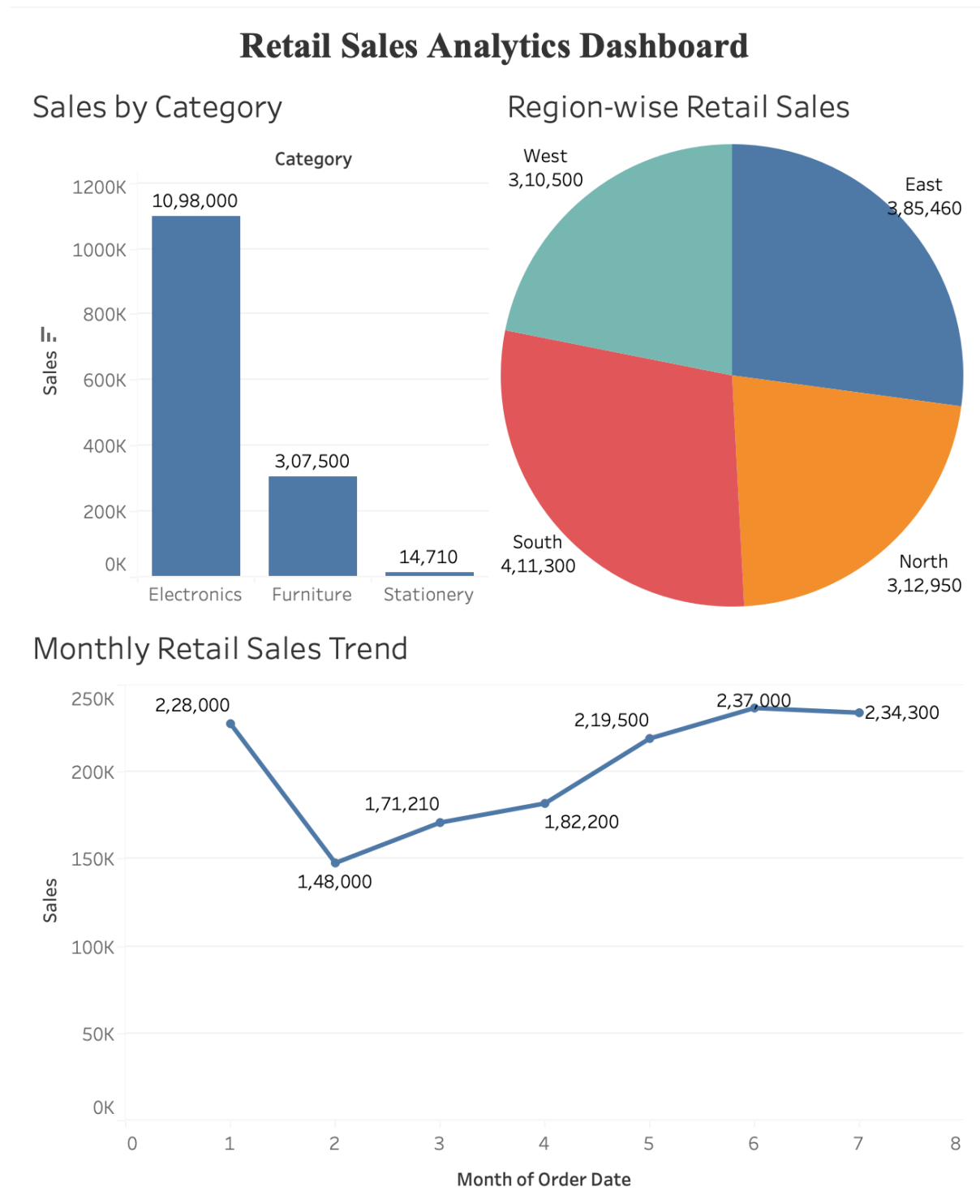
RETAIL SALES ANALYTICS DASHBOARD

The three visualizations were combined into a single Tableau dashboard consisting of:

- Category-wise Retail Sales
- Region-wise Retail Sales
- Monthly Retail Sales Trend

The dashboard provides a consolidated view of important sales patterns and allows the performance of categories, regions and months to be understood easily.

OUTPUT:



FINDINGS

The analysis revealed that Electronics was the highest-selling category with total sales of 1,098,000 and was also the most profitable category with a profit of 122,200. The South region recorded the highest regional sales of 411,300. Among individual products, Laptop generated the highest sales of 315,000, followed by Phone with 300,000. Monthly analysis showed that June 2026 recorded the highest sales of 237,000.

CONCLUSION

Thus, a Retail Sales Analytics system was successfully implemented using Hadoop HDFS and Apache Hive. The retail dataset was stored in HDFS and analysed using HiveQL to identify category-wise sales, regional performance, category-wise profit, top-selling products and monthly sales trends. Tableau was used to visualize the analytical results and develop an interactive retail sales dashboard.

FUTURE ENHANCEMENT

The system can be extended to process larger real-time retail datasets and integrate additional attributes such as customer demographics, discounts, payment methods and inventory information. Machine learning techniques can also be incorporated for sales forecasting, customer segmentation, demand prediction and product recommendation. Real-time streaming technologies such as Apache Kafka and Spark can further be integrated for live retail analytics.

RESULT: The retail sales dataset was successfully stored and processed using Hadoop HDFS and Apache Hive. HiveQL queries were used to derive useful sales insights. The analytical results were successfully visualized using Tableau through bar, pie and line charts and consolidated into a Retail Sales Analytics Dashboard.